

RĪGAS VALSTS TEHNIKUMS

DATORIKAS NODAĻA

Izglītības programma: Programmēšana

KVALIFIKĀCIJAS DARBS

“Daudzfunkcionālas e-komercijas platforma datortehnikas un tās komponentu tirdzniecībai”

Paskaidrojošais raksts 106 lpp.

Audzēknis:

Daniils Onufrijuks

Prakses vadītāja:

Jeļena Matvejeva

Nodaļas vadītājs:

Normunds Barbāns

Rīga, 2026

ANOTĀCIJA

Kvalifikācijas darba tēma ir “Daudzfunkcionāla e-komercijas platforma datortehnikas un tās komponentu tirdzniecībai”. Izstrādātā sistēma ir paredzēta datoru, klēpj datoru un to komponentu tirdzniecībai, nodrošinot lietotājiem ne tikai preču iegādi, bet arī dalību izsolēs, piekļuvi informatīvam bloga saturam un interaktīvu saziņu ar mākslīgā intelekta asistentu. Būtiska platformas inovācija ir ģimenes konta funkcionalitāte, kas ļauj apvienot vairākus lietotājus vienotā grupā, pārvaldīt kopīgas maksājuma metodes un centralizēti kontrolēt ģimenes locekļu veiktos pirkumus un izdevumus. Sistēmas realizācijai tika izmantotas mūsdienīgas tehnoloģijas: HTML5, CSS (Bootstrap), JavaScript un Vue.js klienta puses izstrādei, savukārt servera puses loģika tika izstrādāta ar PHP 8.2 un Laravel 11 ietvaru, izmantojot MySQL 8.0 relāciju datu bāzi. Drošu maksājumu apstrādi nodrošina Stripe sistēma, bet mākslīgā intelekta asistenta funkcijas realizētas, integrējot DeepSeek mākslīgā intelekta risinājumu.

Kvalifikācijas darbs ietver ievadu, uzdevuma nostādni, prasību specifikāciju, uzdevuma risināšanas līdzekļu izvēles pamatojumu, programmatūras produkta modelēšanas un projektēšanas aprakstu, datu struktūru aprakstu, lietotāja ceļvedi, testēšanas piemērus un secinājumus. Ievadā ir pamatota e-komercijas platformas aktualitāte Latvijas tirgū un nepieciešamība pēc integrētiem tehnoloģiju risinājumiem. Uzdevuma nostādnē ir definēti sistēmai izvirzītie mērķi, tostarp ģimenes konta mehānisma izstrāde un funkcionālo lomu sadalījums. Prasību specifikācija sastāv no detalizēta ieejas un izejas informācijas apraksta, kā arī sistēmas funkcionālajām un nefunkcionālajām prasībām. Uzdevuma risināšanas līdzekļu izvēles pamatojumā ir analizētas izvēlētās tehnoloģijas, to priekšrocības un loma drošas sistēmas izveidē. Programmatūras produkta modelēšanas un projektēšanas apraksts ietver modulāru sistēmas arhitektūru ar sešām apakšsistēmām, entītiņu relāciju (ER) modeli un datu plūsmu diagrammas galvenajiem procesiem. Datu struktūru aprakstā tiek parādīta datu bāzes tabulu struktūra, relācijas starp tām, kā arī datu tipi un lauku ierobežojumi. Lietotāja ceļvedī ir norādītas sistēmas prasības aparatūrai un programmatūrai, instalācijas un palaišanas instrukcijas, kā arī detalizēta programmas lietošanas pamācība dažādām lietotāju lomām. Testa piemērā ir dots detalizēts reģistrācijas, izsoļu dalības un ģimenes konta pārvaldības apraksts ar vizuāliem attēliem un rezultātu analīzi. Nobeigumā ir apkopoti darba izstrādes rezultāti un sniegti priekšlikumi sistēmas turpmākai attīstībai.

Kvalifikācijas darbs sastāv no 106 lappusēm, kurās ietilpst 35 attēls, 24 tabulas un 14 pielikumi. Pielikumi satur programmas pirmkoda fragmentus un citus papildu tehniskos materiālus.

ANNOTATION

The theme of the qualification work is "Multifunctional E-commerce Platform for Trading Computer Hardware and Components". The developed system is designed for the trade of computers, laptops, and components, providing users not only with product purchases but also participation in auctions, access to informative blog content, and interactive communication with an artificial intelligence assistant. A significant innovation of the platform is the family account functionality, which allows combining several users into a unified group, managing shared payment methods, and centrally controlling purchases and expenses made by family members. For the implementation of the system, modern technologies were used: HTML5, CSS (Bootstrap), JavaScript, and Vue.js for frontend development, while the server-side logic was developed with PHP 8.2 and the Laravel 11 framework, utilizing a MySQL 8.0 relational database. Secure payment processing is ensured by the Stripe system, and the AI assistant functions are implemented by integrating the DeepSeek AI solution.

The qualification work includes an introduction, task statement, requirements specification, justification of development tools, software product modeling and design description, description of data structures, user guide, testing examples, and conclusion. The introduction justifies the relevance of the e-commerce platform in the Latvian market and the need for integrated technology solutions. The task setting defines the goals set for the system, including the development of the family account mechanism and the distribution of functional roles. The requirements specification consists of a detailed description of input and output information, as well as the system's functional and non-functional requirements. The justification of tools analyzes the chosen technologies, their advantages, and their role in creating a secure system. The software product modeling and design description includes a modular system architecture with six subsystems, an entity-relationship (ER) model, and data flow diagrams for the main processes. The data structure description shows the structure of the database tables, the relations between them, as well as data types and field constraints. The user guide specifies the system requirements for hardware and software, installation and startup instructions, as well as a detailed program usage guide for various user roles. The testing example provides a detailed description of registration, auction participation, and family account management with visual images and result analysis. In the conclusion, the results of the work development are summarized, and proposals for further system development are provided.

The qualification work consists of 106 pages, including 35 images, 24 tables, and 14 appendices. The appendices contain program source code fragments and other additional technical materials.

SATURS

IEVADS	6
1. UZDEVUMA NOSTĀDNE	8
2. PRASĪBU SPECIFIKĀCIJA	12
2.1. Ieejas, izejas un ārējās informācijas apraksts	12
2.1.1. Ieejas informācijas apraksts	12
2.1.2. Izejas informācijas apraksts	14
2.2. Funkcionālās prasības	19
2.3. Nefunkcionālās prasības	24
3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS	28
4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA	31
4.1. Sistēmas struktūras modelis	31
4.1.1. Sistēmas arhitektūra	31
4.1.2. Sistēmas ER modelis	34
4.2. Funkcionālais sistēmas modelis	36
4.2.1. Datu plūsmu modelis	36
5. DATU STRUKTŪRU APRAKSTS	41
6. LIETOTĀJA CEĻVEDIS	52
6.1. Sistēmas prasības aparatūrai un programmatūrai	52
6.2. Sistēmas instalācija un palaišana	52
6.3. Programmas apraksts	54
7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA	62
7.1. Testēšanas mērķis	63
7.2. Testēšanas metodika	63
7.3. Testēšanas scenāriji	64
7.4. Testēšanas rezultāti	64
7.5. Testēšanas rezultātu analīze	65
7.6. Ieteiktie uzlabojumi	65
7.7. Secinājumi par lietojamību	65
NOBEIGUMS	66
INFORMĀCIJAS AVOTI	67
PIELIKUMI	68
1. pielikums Lietojumgadījumu diagramma	69
1.2. pielikums Lietojumgadījumu diagramma	70
2. pielikums Funkcionālās dekompozīcijas diagramma	71

3. pielikums ER diagramma	72
4. pielikums Fiziskā struktūra	73
5. pielikums Lietotāja kontrolieris	74
6. pielikums Izsoles kontrolieris	75
7. pielikums Pasūtījumu kontrolieris	76
8. pielikums Preču kontrolieris	78
9. pielikums Autentifikācijas kontrolieri	78
10. pielikums Komentaru kontrolieris	82
11. pielikums Parvāldnieka kontrolieris	82
12. pielikums MI (DeepSeek) kontrolieris	96
13. pielikums PDF izdruka	104

IEVADS

Mūsdienu digitālajā vidē e-komercijai ir būtiska nozīme gan uzņēmējdarbībā, gan ikdienas lietotāju paradumos. Tehnoloģiju attīstība un interneta pieejamība ir būtiski mainījusi to, kā cilvēki iegādājas preces un pakalpojumus, arvien vairāk dodot priekšroku tiešsaistes risinājumiem. Īpaši strauji attīstās specializētās e-komercijas platformas. Tas piedāvā ne tikai preču iegādi, bet arī papildu funkcionalitāti, piemēram, izsoļu mehānismus, ģimenes pakalpojumus, informatīvu saturu, MI un automatizētus atbalsta rīkus.

Datortehnikas un tās komponentu tirgus ir viens no dinamiskākajiem tehnoloģiju sektorā, jo datoru lietošana ir neatņemama ikdienas sastāvdaļa gan izglītībā, gan profesionālajā darbībā. Lietotāji arvien biežāk meklē ne tikai iespēju iegādāties datorus vai to detaļas, bet arī saņemt kvalitatīvu informāciju par produktu īpašībām, savietojamību un piemērotību konkrētām vajadzībām. Tomēr Latvijas e-komercijas vidē šobrīd trūkst vienotas, specializētas platformas, kas apvienotu datortehnikas tirdzniecību ar izglītojošu saturu, elastīgu cenu noteikšanu un personalizētu lietotāju atbalstu.

Pašlaik lielākā daļa pieejamo interneta veikalu piedāvā tikai klasisku preču katalogu ar fiksētām cenām, savukārt izsoļu platformas un tehnoloģiju forumi darbojas kā atsevišķi, savstarpēji nesaistīti risinājumi. Tas rada situāciju, kurā lietotājam ir jāizmanto vairākas dažādas tīmekļa vietnes – viena preču iegādei, cita cenu salīdzināšanai, vēl cita pieredzes un ieteikumu iegūšanai. Šāda sadrumstalotība palielina laika patēriņu un samazina lietošanas ērtumu, kā arī var novest pie neprecīziem lēmumiem preču izvēlē.

Papildus tam arvien aktuālāks kļūst jautājums par drošu un pārskatāmu maksājumu pārvaldību, īpaši gadījumos, kad digitālās platformas izmanto vairāki vienas ģimenes locekļi. Tiešsaistes iepirkšanās procesā bieži iesaistās gan pieaugušie, gan jaunieši, tādēļ ir svarīgi nodrošināt mehānismus, kas ļauj kontrolēt izdevumus, pārvaldīt maksājuma metodes un nodrošināt caurspīdīgu darījumu uzskaiti. Centralizēta ģimenes konta ieviešana e-komercijas platformā ļauj apvienot vairākus lietotājus vienotā struktūrā, nodrošinot kopīgu maksājumu pārvaldību un darījumu uzraudzību.

Lai mazinātu minētās problēmas, ir nepieciešams izstrādāt modernu, integrētu e-komercijas platformu, kas nodrošinātu centralizētu piekļuvi visiem ar datortehnikas iegādi saistītajiem procesiem. Šādai sistēmai būtu jāapvieno preču pārdošana, izsoļu funkcionalitāte, bloga vide ar lietotāju radītu saturu, ģimenes konta mehānisms ar kopīgu maksājumu pārvaldību,

kā arī mākslīgā intelekta asistents, kas spēj sniegt ātras un personalizētas atbildes uz lietotāju jautājumiem. Mākslīgā intelekta izmantošana e-komercijā ļauj uzlabot klientu apkalpošanu, samazināt administratīvo slogu un veicināt efektīvāku lietotāju iesaisti platformas darbībā.

Izstrādājamā e-komercijas platforma ir vērsta uz tehnoloģiju entuziastiem, IT speciālistiem, studentiem un citiem lietotājiem, kuri ikdienā strādā ar datoriem vai interesējas par datortehnikas jaunumiem. Vienlaikus sistēma paredzēta arī ģimenēm, kurās vairāki lietotāji izmanto vienotu maksājumu vidi, nodrošinot pārskatāmu izdevumu uzskaiti un drošu maksājumu apstrādi. Platforma paredzēta kā daudzfunkcionāls risinājums, kas ne tikai atvieglo pirkšanas un pārdošanas procesu, bet arī veicina zināšanu apmaiņu, lietotāju savstarpējo komunikāciju un atbildīgu digitālo finanšu pārvaldību. Vienotas sistēmas ieviešana ļautu uzlabot lietotāju pieredzi, paaugstināt informācijas pieejamību un sekmēt mūsdienīgu e-komercijas risinājumu attīstību Latvijā.

1. UZDEVUMA NOSTĀDNE

Kvalifikācijas darba uzdevums ir izstrādāt lietotājam draudzīgu, ātru un drošu e-komercijas platformu, kas specializējas datoru, klēpj datoru un to komponentu tirdzniecībā, vienlaikus nodrošinot izsoļu, bloga un mākslīgā intelekta asistenta funkcionalitāti.

Izstrādātajai sistēmai jānodrošina iespēja lietotājiem apskatīt, meklēt un iegādāties datortehniku un komponentes, piedalīties izsolēs, lasīt un komentēt bloga ierakstus, kā arī izmantot mākslīgā intelekta asistentu jautājumu uzdošanai un konsultāciju saņemšanai. Platformai jābūt pieejamai gan neregistrētiem, gan reģistrētiem lietotājiem, piedāvājot personalizētu un ērtu lietošanas pieredzi.

Sistēmas izstrāde ir aktuāla, jo Latvijas tirgū trūkst specializētu e-komercijas risinājumu. Platformas mērķauditorija ir tehnoloģiju entuziasti, IT speciālisti un studenti, kā arī citi lietotāji, kuri vēlas iegādāties vai pārdot datortehniku un vienlaikus iegūt aktuālu informāciju par tehnoloģijām.

Papildus sistēmā jāparedz ģimenes kontu funkcionalitāte, kas ļauj vienam galvenajam lietotājam izveidot koplietojamu kontu ar vairākiem apakšlietotājiem. Šāda funkcionalitāte nodrošina iespēju pārvaldīt kopīgus pasūtījumus, sekot līdzi pirkumu vēsturei, noteikt piekļuves tiesības atsevišķiem ģimenes locekļiem, kā arī kontrolēt izdevumus, piemēram, nosakot pirkumu apstiprināšanas mehānismu vai limitus. Ģimenes kontu sistēma veicina drošāku un pārskatāmāku platformas izmantošanu mājsaimniecības ietvaros, īpaši gadījumos, kad platformu izmanto vairāki lietotāji no vienas ģimenes.

Izstrādātajā sistēmā ir plānots realizēt šādas galvenās funkcijas:

- lietotājam iespēju apskatīt un iegādāties datorus, klēpj datorus un komponentes;
- izsoļu funkcionalitāti, kas nodrošina elastīgu un konkurētspējīgu cenu noteikšanu;
- bloga sadaļu ar iespēju lasīt rakstus par tehnoloģiju aktualitātēm;
- mākslīgā intelekta asistenta integrāciju, kas sniedz lietotājam tūlītējas atbildes un palīdz orientēties platformā;
- administrācijas iespēju pārvaldīt preces, izsoles, lietotājus un publicēto saturu.
- ģimenes konta funkcionalitāti

Izstrādātā e-komercijas platforma atšķiras no citiem tirgū pieejamiem risinājumiem ar savu specializāciju datortehnikas jomā, vairāku funkciju apvienošanu vienotā sistēmā un modernu, lietotājam ērtu saskarni, kas uzlabo kopējo lietošanas pieredzi. Sistēmā darbojas piecas lietotāju lomas: Pārvaldnieks, Pircējs (Reģistrēts lietotājs), Viesis, Stripe sistēma un DeepSeek MI. Ir plānotas vairākas funkcijas un lomas (skat. 1. un 1.2 pielikumus):

1) Pārvaldnieks ir sistēmas pārzinis, kurš nodrošina platformas darbību un uzrauga lietotājus. Viņam ir pilnīga piekļuve visiem sistēmas aspektiem, lai nodrošinātu izsoļu un preču tirdzniecības godīgumu un drošību. Pārvaldnieks var pārvaldīt lietotāju kontus, ģimenes ieskaitot to bloķēšanu, atbloķēšanu un dzēšanu, kā arī uzraudzīt izsoles un preču sarakstus. Viņa atbildībā ir arī jauno preču apstiprināšana vai noraidīšana, maksājumu un transakciju pārbaude, sūdzību risināšana un komentāru moderēšana. Pārvaldnieks var arī pārvaldīt preces un izsoles, mainot vai dzēšot ierakstus pēc nepieciešamības. Lomas funkcijas ietver:

- **lietotāju kontu pārvaldīšanu** – var bloķēt vai atbloķēt lietotājus neatbilstošas rīcības gadījumā, kā arī dzēst kontus, kas pārkāpj platformas noteikumus vai netiek izmantoti;
- **izsoles un preču sarakstu uzraudzību** – regulāri pārskata aktīvās izsoles un pievienotās preces, lai nodrošinātu, ka tās atbilst platformas prasībām;
- **jaunu izsoļu preču apstiprināšanu vai noraidīšanu** – izvērtē lietotāju pievienotās preces pirms to publicēšanas, novēršot neatbilstoša vai maldinoša satura parādīšanos;
- **maksājumu un transakciju pārbaudīšanu** – uzrauga maksājumu statusus, identificē kļūdas vai aizdomīgus darījumus un nepieciešamības gadījumā veic papildu pārbaudes;
- **sūdzību risināšanu un komentāru moderēšanu** – izskata lietotāju iesniegtās sūdzības, dzēš neatbilstošus komentārus un nodrošina korektu komunikāciju platformā;
- **preču un izsoļu pārvaldīšanu** – var rediģēt, apturēt vai dzēst preces un izsoles, ja tiek konstatēti pārkāpumi vai tehniskas problēmas;
- **preču katalogu un izsoļu pārlūkošanu** – izmanto tādas pašas pārlūkošanas iespējas kā lietotāji, bet ar papildu administratīvām tiesībām.

2) Reģistrēts lietotājs ir tas, kurš reģistrējies un pierakstījies sistēmā, lai izmantotu visas platformas iespējas. Viņš var pārlūkot preču katalogu un izsoles, iegādāties preces par fiksētu cenu, pievienot tās grozam un noformēt pasūtījumus. Pircējam ir iespēja piedalīties izsolēs, iesniedzot savus cenu piedāvājumus, kā arī izmantot integrēto maksājumu sistēmu, lai veiktu apmaksu un izvēlētos sev piemērotu piegādes veidu. Papildus pircējs var izmantot mākslīgā intelekta asistentu, sazināties ar pārvaldnieku caur “Contact” sadaļu un piedalīties bloga komentāru sadaļā. Lomas funkcijas ietver:

- **preču pārlūkošanu un meklēšanu katalogā** – var apskatīt visas pieejamās preces, izmantot meklēšanu un filtrus, lai ātri atrastu sev interesējošo;
- **preču pievienošanu grozam un pasūtījumu noformēšanu** – izvēlas preces, pievieno tās grozam un veic pirkumu, izmantojot integrēto maksājumu sistēmu;
- **izsoļu dalību** – var pievienot savus cenu piedāvājumus izsolēs vai sekot līdzi citu lietotāju piedāvājumiem reāllaikā;

- **mākslīgā intelekta asistenta izmantošanu** – uzdod jautājumus par platformas funkcionalitāti, precēm vai citiem tematiem, lai ātri saņemtu atbildes;
- **bloga komentāru rakstīšanu un saglabāšanu** – var piedalīties diskusijās, izteikt viedokli un vēlāk rediģēt vai dzēst savus komentārus;
- **sazināšanos ar pārvaldnieku** – izmanto “Contact” sadaļu, lai uzdotu jautājumus, ziņotu par problēmām vai iesniegtu sūdzības;
- **sava profila pārvaldīšanu** – var mainīt personīgos datus, atjaunot paroli un pārvaldīt konta iestatījumus;
- **ģimenes konta pārvaldīšanu** – var aktivizēt ģimenes kontus, pievienot apakšlietotājus (ģimenes locekļus) un noteikt viņiem piekļuves tiesības;
- **ģimenes locekļu atļauju pārvaldīšanu** – nosaka, kuri lietotāji drīkst veikt pirkumus patstāvīgi, piedalīties izsolēs vai tikai pārlūkot saturu;
- **kopējo izdevumu kontrolēšanu** – var iestatīt pirkumu limitus, apstiprināšanas mehānismu (piemēram, vecāku apstiprinājumu) un pārskatīt visu ģimenes locekļu pasūtījumu vēsturi;
- **ģimenes aktivitāšu uzraudzību** – redz ģimenes konta ietvaros veiktos pasūtījumus, izsoļu piedāvājumus un maksājumu informāciju vienotā pārskatā.

3) Viesis ir neautenticēts lietotājs, kurš var pārlūkot sistēmas saturu ar ierobežotām iespējām. Viņam ir atļauts skatīt pieejamās preces un izsoles, filtrēt un kārtot preces, kā arī lasīt bloga saturu. Viesis var pievienot preces grozam, bet nevar veikt pasūtījumu vai piedalīties izsolēs. Ja viņš vēlas izmantot papildu iespējas, viesim vispirms jāreģistrējas un jāpierakstās sistēmā. Papildus viesim ir iespēja sazināties ar administratoru jautājumu vai problēmu gadījumā. Lomas funkcijas ietver:

- **preču pārlūkošanu un meklēšanu katalogā** – var apskatīt piedāvājumu bez nepieciešamības veidot kontu;
- **preču filtrēšanu un kārtošānu** – izmanto filtrus (piemēram, pēc cenas vai nosaukuma), lai ērtāk atrastu interesējošās preces;
- **bloga saturu skatīšanu** – lasa rakstus un komentārus, bet nevar tos pievienot vai rediģēt.
- **preču pievienošanu grozam** – var sagatavot grozu, taču pasūtījuma noformēšanai nepieciešama reģistrācija;
- **izsoles piedāvājumu skatīšanu** – var sekot līdzi izsoļu norisei, bet nevar piedalīties cenu solīšanā;

- **reģistrēšanos vai pieslēgšanos** – var izveidot kontu vai autorizēties, lai iegūtu pilnas platformas iespējas;
- **ģimenes konta izveidošanu** – var aktivizēt/izveidot ģimenes konta režīmu.

4) Stripe sistēma ir ārējais maksājumu apstrādātājs, kas nodrošina finanšu darījumu drošību un precizitāti. Tā validē lietotāja ievadītos maksājuma datus, apstrādā maksājumus un sniedz atgriezenisko saiti par darījuma veiksmīgumu vai kļūdu. Stripe integrācija garantē, ka pircēju maksājumi tiek veikti droši un atbilstoši starptautiskiem standartiem. Lomas funkcijas ietver:

- **lietotāju maksājumu apstrādi** – nodrošina drošu norēķinu veikšanu par pasūtījumiem;
- **maksājumu datu validēšanu** – pārbauda kartes vai citu maksājumu metožu korektumu un drošību;
- **darījuma statusu sniegšanu** – nosūta platformai informāciju par veiksmīgu maksājumu vai radušos kļūdu.

5) DeepSeek AI ir mākslīgā intelekta asistents, kas palīdz lietotājiem atrast atbildes uz jautājumiem un sniedz personalizētu atbalstu platformā. Tas saņem lietotāja jautājumu, apstrādā pieprasījumu un sniedz atbildi, kas tiek parādīta sistēmas saskarnē. Ja jautājumu nav iespējams apstrādāt, MI paziņo par kļūdu. Šī funkcionalitāte uzlabo lietotāja pieredzi, padarot sistēmu interaktīvāku un ērtāku. Lomas funkcijas ietver:

- **lietotāja jautājumu saņemšanu** – lietotājs ievada jautājumu platformas saskarnē;
- **pieprasījumu apstrādi** – analizē jautājumu un ģenerē atbilstošu atbildi;
- **atbilžu attēlošanu lietotājam** – nodrošina skaidru un saprotamu atbildes parādīšanu;
- **kļūdu paziņošanu** – informē, ja jautājums nav izprasts vai atbilde nevar tikt sniegta.

2. PRASĪBU SPECIFIKĀCIJA

2.1. Ieejas, izejas un ārējās informācijas apraksts

2.1.1. Ieejas informācijas apraksts

Sistēma apstrādās dažāda veida ievades informāciju, ko ievadīs lietotāji, pārvaldnieki un citas sistēmas, lai nodrošinātu pilnvērtīgu platformas darbību. Visi dati tiek validēti un pārbaudīti pēc formāta un obligātuma nosacījumiem.

1. Lietotāja informācija (šī informācija tiek ievadīta reģistrācijas vai profila rediģēšanas procesā):

- E-pasts – unikāla e-pasta adrese līdz 50 rakstzīmēm, formātā local@domain.tld . Piemērs: janis.ozolins@gmail.com.
- Parole – teksta lauks ar garumu no 8 līdz 30 rakstzīmēm. Parolei jāsaturs vismaz viens lielais burts, viens mazais burts, viens cipars un viens speciālais simbols. Parole sistēmā tiek glabāta šifrētā (hashētā) veidā.
- Piegādes adrese – teksta lauks līdz 100 rakstzīmēm, kas ietver ielu, mājas numuru, pilsētu, pasta indeksu un valsti. Piemērs: Brīvības iela 12-5, Rīga, LV-1010, Latvija.
- Lomas tips – izvēles lauks ar noteiktām vērtībām: viesis, reģistrēts lietotājs vai pārvaldnieks.

Apstrādes gaitā sistēma vispirms parbauda ievadīto datu formātu un obligātos laukus. Tiek pārbaudīts, vai ievadītā e-pasta adrese jau nav reģistrēta datubāzē. Parole tiek pārbaudīta pret drošības prasībām. Ja validācija ir veiksmīga, sistēma izveido jaunu ierakstu tabulā lietotājs, paroli šifrējot ar drošu hash algoritmu. Pēc veiksmīgas ieraksta izveides lietotājam tiek nosūtīts e-pasts. Visbeidzot sistēma automātiski pieslēdz lietotāju (vai pārvirza uz pieteikšanās lapu) un parāda paziņojumu par veiksmīgu reģistrāciju.

2. Produktu informācija (šie dati tiek ievadīti, kad pārvaldnieks pievieno jaunu preci katalogam):

- Produkta nosaukums – teksta lauks līdz 50 rakstzīmēm. Piemērs: ASUS ROG Strix G16.
- Cena – decimālskaitlis ar divām zīmēm aiz komata, norādīts eiro valūtā. Minimālā vērtība 0,01. Piemērs: 1299,99.
- Kategorija – izvēles lauks, piemēram: Klēpjatori, Komponentes, Monitori, Perifērija.
- Apraksts – teksta lauks līdz 1000 rakstzīmēm, kurā tiek sniegta detalizēta informācija par produktu.
- Attēls – JPG vai PNG formāta attēls ar maksimālo izmēru 5 MB.
- Pieejamais daudzums – vesels skaitlis, kas norāda noliktavā pieejamo preču skaitu.
- Ražotājs un modelis – teksta lauks līdz 50 rakstzīmēm. Piemērs: Intel Core i7-12700K.

3. Izsoles dati (ja lietotājs vai pārvaldnieks veido izsoli):

- Izsoles objekta nosaukums – teksta lauks līdz 50 rakstzīmēm.
- Sākuma cena – decimālskaitlis ar minimālo vērtību 0,01.
- Solis – decimālskaitlis, kas nosaka minimālo cenas palielinājumu izsoles laikā.
- Izsoles sākuma datums un laiks – datuma un laika lauks formātā YYYY-MM-DD HH:MM.
- Izsoles beigu datums un laiks – datuma un laika lauks, kas vienmēr ir vēlāks par izsoles sākuma laiku.
- Apraksts – teksta lauks līdz 500 rakstzīmēm ar informāciju par izsoles objektu.

4. Meklēšanas un filtrēšanas dati (lietotājs var ievadīt):

- Atslēgvārdu meklēšanas laukā.
- Filtra parametrus (kategorija, cena, ražotājs, jauns vai lietots).

5. Komentāru ievade (lietotājs var pievienot saturu):

- Komentārs – līdz 500 rakstzīmēm.

6. MI asistenta ievade:

- Jautājums vai pieprasījums – līdz 300 rakstzīmēm.
- Tēmas konteksts (piemēram, “Izvēlēties procesoru”, “Salīdzināt grafikas kartes”).

2.1.2. Izejas informācijas apraksts

PDF izdruka ar pilnu sistēmas datu eksportu ir pieejama pārvaldniekam. Šī izdruka apkopo vairākas sistēmas datu kopas vienā dokumentā. Augšējā daļā tiek norādīts datums un laiks, kad fails tika ģenerēts. Dokuments tiek veidots A4 formātā, lai būtu iespējams attēlot vairāk kolonnas. Pielikumos ir attēloti PDF izdruku piemēri. PDF dokumentā tiek iekļautas šādas datu tabulas:

1) Tabulā tiek attēlota **informācija par sistēmas lietotājiem**:

- lietotāja ID;
- vārds;
- e-pasta adrese;
- lietotāja loma;
- apbalvojumi (awards);
- konta izveides datums.

2) Tabulā tiek attēlota **informācija par produktiem**:

- produkta ID;
- nosaukums;
- cena;
- kategorija;
- izveides datums.

3) Tabulā tiek attēlota **informācija par pasūtījumiem**:

- pasūtījuma ID;
- lietotāja vārds;
- pasūtījuma statuss;
- kopējā summa;
- izveides datums.

4) Tabulā tiek attēlota **informācija par pasūtījumu vienībām**:

- ieraksta ID;
- pasūtījuma ID;
- produkta nosaukums;
- cena;
- daudzums;
- statuss;
- izveides datums.

5) Tabulā tiek attēlota **informācija par ģimenes kontiem:**

- ģimenes ID;
- ģimenes nosaukums;
- vecāka (parent) vārds;
- ielūguma kods;
- izveides datums.

6) Tabulā tiek attēlota **informācija par komentāriem:**

- komentāra ID;
- lietotāja vārds;
- komentāra saturs;
- izveides datums.

7) Tabulā tiek attēlota **informācija par izsolēm:**

- izsoles ID;
- nosaukums;
- sākuma cena;
- sākuma laiks;
- beigu laiks;
- izveides datums.

8) Tabulā tiek attēlota **informācija par solījumiem:**

- solījuma ID;
- izsoles nosaukums;
- lietotāja vārds;
- solījuma summa;
- izveides datums.

9) Papildus nosacījumi:

- Katrā tabulā tiek attēloti līdz 1000 ierakstiem.
- Dati tiek sakārtoti pēc ID augošā secībā.
- **PDF** fails tiek lejupielādēts ar automātiski ģenerētu nosaukumu, kurā iekļauts datums un laiks.

Sistēma ģenerē un nodrošina lietotājiem dažādu veidu izejas informāciju atkarībā no lietotāja darbībām un piešķirtajām tiesībām. Izejas informācija tiek attēlota tiešsaistē platformas lietotāja saskarnē. Papildus tam noteikta informācija tiek nosūtīta lietotājiem e-pasta vai *push* paziņojumu veidā.

Katrs izejas informācijas veids atšķiras pēc mērķa, satura struktūras, formāta un paredzētā lietotāja tipa. Piemēram, lietotājiem tiek attēlota informācija par precēm, pasūtījumiem un izsoļu norisi, savukārt pārvaldniekiem – sistēmas darbības pārskati, lietotāju aktivitātes dati un darījumu informācija. Šāda pieeja nodrošina, ka katrs lietotājs saņem tikai sev aktuālu un nepieciešamu informāciju, atbilstoši sistēmas funkcionalitātei.

1. Pasūtījuma apstiprinājums un detalizēta informācija.

Kad lietotājs veic pirkumu, sistēma ģenerē detalizētu pasūtījuma apstiprinājumu, kas ir pieejams vairākos veidos:

- **Ekrāna skatā:** uzreiz pēc maksājuma apstiprināšanas.
- **E-pasta ziņojumā:** automātiski nosūtīts lietotājam. Pasūtījuma apstiprinājuma struktūra izskatās sekojoši:
 - Pasūtījuma ID – unikāls identifikators (piem.: #ORD-2025-1042).
 - Pasūtījuma izveides datums un laiks.
 - Lietotāja dati: vārds, e-pasts, piegādes adrese.
- **Produktu saraksts tabulas veidā:**
 - Produkta nosaukums.
 - Preces kods/ID.
 - Vienības cena.
 - Pasūtītais daudzums.
 - Starpsumma.
- Kopējā summa.
- Maksājuma statuss (apmaksāts / neapmaksāts / gaida apstiprinājumu).

Papildu funkcionalitāte: Lietotājs var apskatīt iepriekšējo pasūtījumu vēsturi.

2. Preču katalogs un meklēšanas rezultāti.

Sistēma atgriež rezultātus lietotājam atkarībā no viņa ievadītajiem kritērijiem (meklēšanas vārds, filtrs, kategorija). Izvades struktūra ir dinamiska un pielāgota ekrāna izmēram (mobilais, planšete, dators).

- **Katram produktam izvades sarakstā jāparāda:**
 - produkta nosaukumu;
 - galveno attēlu (*thumbnail*);
 - cenu (ieskaitot akcijas cenu, ja piemērojams).
- **Detalizētās produkta lapas izvade ietver:**
 - pilnu aprakstu un tehniskās specifikācijas (piem., CPU tips, RAM, ekrāna izmērs u.c.);
 - papildu attēlu galeriju;
 - saistītos produktus (ieteikumi).

3. Izsoles rezultātu lapa.

Kad izsole ir beigusies, sistēma ģenerē speciālu izvades lapu ar pilnu izsoles informāciju, kas ir pieejama tiešsaistē.

- **Izsoles rezultāta struktūra izskatās sekojoši:**
 - Izsoles ID un nosaukums.
 - Beigu datums un laiks.
 - Uzvaras summa (galīgā cena).

4. Bloga sadaļas un komentāru izvade.

Sistēma ģenerē saturu bloga sadaļai un komentāru blokiem. Katram ierakstam vai komentāram tiek parādīti šādi dati:

- raksta nosaukums;
- raksta saturs (teksts, attēli, video, saites).

Komentāru sadaļā tiek izvietots:

- lietotājvārds;
- komentāra teksts;
- publicēšanas datums un laiks.

5. MI asistenta atbildes un ieteikumi.

Kad lietotājs uzdod jautājumu MI asistentam, sistēma sniedz atbildi reāllaikā, kas var būt vienkāršs teksts vai dinamiska struktūra ar papildinformāciju:

- Tiešā atbilde uz jautājumu.
- Produkta ieteikumu saraksts ar saitēm uz preču lapām.
- Salīdzināšanas tabulas (ja lietotājs jautā, piemēram, par diviem produktiem).
- Pirkšanas padomi vai rokasgrāmatas saites.
- Saistīti bloga raksti vai biežāk uzdotie jautājumi.

6. Lietotāja profila informācijas izvade.

Lietotājs savā profilā redz personalizētu informāciju, kas tiek automātiski ģenerēta:

- Konta informācija (vārds, uzvārds, e-pasts, piegādes adreses).
- Pasūtījumu vēsture (saraksts ar visiem pasūtījumiem un summām).
- Maksājumu vēsture.

7. Administrācijas pārskati un statistika.

Sistēma ģenerē detalizētus pārskatus pārvaldniekiem, kas tiek rādīti tabulu un diagrammu veidā:

- Lietotāju aktivitātes pārskats (reģistrācijas, pirkumi, izsoles).
- Pārdošanas apjoma pārskats.
- Produktu, pasūtījumu, lietotāju un ģimeņu saraksts.
- Komentāru un atsauksmju moderācijas pārskats.
- PDF izdruka ar informāciju par visiem ierakstiem no datubāzes

8. Paziņojumi un notifikācijas.

Sistēma ģenerē arī īsos paziņojumus, kas tiek rādīti dažādos kontekstos:

- **Tiešsaistes paziņojumi:** “Produkta pievienošana grozam veiksmīga”, “Maksājums apstiprināts” u.c.
- **E-pasta paziņojumi:** par reģistrāciju, pasūtījuma statusu.
- **Push paziņojumi (mobilajās ierīcēs):** kad izsole tuvojas beigām vai par jauniem bloga ierakstiem.

2.2. Funkcionālās prasības

1. tabula

1. Lietotāju kontu pārvaldībai:

Identifikators	PS 2.2.1
Ievads	
Funkcija paredzēta lietotāju kontu izveidei, autentifikācijai un pārvaldībai.	
Ievaddati	
1. E-pasts (unikāls) 2. Parole 3. Kontaktinformācija	
Apstrāde	
1. Validē ievaddatus 2. Pārbauda e-pasta unikālumu 3. Autorizē vai reģistrē lietotāju	
Izvaddati	
1. Veiksmīga reģistrācija/pieslēgšanās 2. Kļūdas ziņojums	
Kļūdas paziņojumi	
1. Nepareizs e-pasts vai parole 2. E-pasts jau eksistē 3. Parole neatbilst prasībām	
Izsaucamās prasības	
Pieejams visiem lietotājiem ar internetu	

- 1.1. Sistēmai jānodrošina iespēja **reģistrēt jaunu lietotāju**, izmantojot e-pasta adresi, paroli un pamatinformāciju (kontaktdati).
- 1.2. Sistēmai jāļauj lietotājam **autorizēties** ar reģistrēto e-pastu un paroli vai **pieslēgties ar trešo pušu kontu**.
- 1.3. Sistēmai jānodrošina iespēja **redīgēt lietotāja profila informāciju** (vārdu, adresi, kontaktus, piegādes informāciju).
- 1.4. Sistēmai jānodrošina iespēja **dzēst kontu**, ja lietotājs to pieprasa, ievērojot datu aizsardzības prasības.
- 1.5. Sistēmai jābrīdina lietotāju, ja **parole neatbilst drošības prasībām** (piem., minimālais garums, lielie un mazie burti, cipari).
- 1.6. Sistēmai jāneļauj izveidot kontu, ja **lietotājs ar ievadīto e-pastu jau pastāv** un jāizvada kļūdas ziņojums.
- 1.7. Sistēmai jāļauj pārvaldniekam **pārvaldīt lietotāju kontus** – dzēst, bloķēt, mainīt lomas un piekļuves tiesības.

2. Produktu katalogam un filtrēšanai:

Identifikators	PS 2.2.2
Ievads	
Funkcija paredzēta produktu pārlūkošanai, meklēšanai un filtrēšanai.	
Ievaddati	
1. Meklēšanas vaicājums 2. Filtri (cena)	
Apstrāde	
1. Meklē produktus datubāzē 2. Pielieto filtrus un kārtošanu	
Izvaddati	
1. Produktu saraksts 2. Detalizēta produkta informācija	
Kļūdas paziņojumi	
1. Nav atrastu produktu	
Izsaucamās prasības	
Pieejams visiem lietotājiem ar internetu	

- 2.1. Sistēmai jāļauj visiem lietotājiem **pārlūkot preču katalogu** pēc kategorijām un apakšskategorijām.
- 2.2. Sistēmai jāļauj lietotājiem **meklēt preces** pēc nosaukuma, atslēgvārdiem vai specifikācijām.
- 2.3. Sistēmai jāļauj lietotājiem **filtrēt produktus** pēc cenas, ražotāja, procesora, atmiņas un citiem kritērijiem.
- 2.4. Sistēmai jānodrošina iespēja **kārtot rezultātus** pēc cenas, popularitātes, reitinga vai jaunuma.
- 2.5. Sistēmai jāatspoguļo katra produkta **detalizēta informācija** – apraksts, specifikācijas, cena, attēli un pieejamība.
- 2.6. Sistēmai jānodrošina **automātiskie ieteikumi** meklēšanas laikā, pamatojoties uz populāriem vaicājumiem vai līdzīgiem produktiem.

3. Grozam un pasūtījumu apstrādei:

Identifikators	PS 2.2.3
Ievads	
Funkcija nodrošina produktu pievienošanu grozam un pasūtījuma veikšanu.	
Ievaddati	
1. Produkta ID 2. Daudzums 3. Piegādes un maksājuma dati	
Apstrāde	
1. Saglabā groza saturu 2. Aprēķina cenu 3. Apstrādā maksājumu	
Izvaddati	
1. Pasūtījuma apstiprinājums 2. Pasūtījuma statuss	
Kļūdas paziņojumi	
1. Maksājums neizdevās 2. Nepietiekams daudzums noliktavā	
Izsaucamās prasības	
Pieejams visiem autorizētiem lietotājiem ar internetu	

- 3.1. Sistēmai jāļauj lietotājam **pievienot produktu grozam** ar vienu klikšķi no kataloga vai produkta lapas.
- 3.2. Sistēmai jāļauj **rediģēt grozu**, mainot produktu daudzumu vai dzēšot nevajadzīgus produktus.
- 3.3. Sistēmai jānodrošina iespēja **apskatīt groza kopsavilkumu** – kopējo cenu, piegādes izmaksas.
- 3.4. Sistēmai jāļauj lietotājam **noformēt pasūtījumu**, ievadot piegādes un maksājuma informāciju.
- 3.5. Sistēmai jāintegrējas ar ārējiem maksājumu pakalpojumiem (Stripe).
- 3.6. Sistēmai jāatjauno pasūtījuma statuss (**apmaksāts, gaida apmaksu, nosūtīts, piegādāts**) reāllaikā.

4. Izsoļu funkcionalitātei:

Identifikators	PS 2.2.4
Ievads	
Funkcija paredzēta produktu izsolēm un solījumu veikšanai.	
Ievaddati	
1. Produkts 2. Solījuma summa	
Apstrāde	
1. Validē solījumu 2. Atjauno augstāko cenu 3. Reģistrē dalībnieku	
Izvaddati	
1. Jaunā cena 2. Izsoles statuss	
Kļūdas paziņojumi	
1. Solījums par zemu 2. Izsole beigusies	
Izsaucamās prasības	
Pieejams visiem reģistrētiem lietotājiem ar internetu	

- 4.1. Sistēmai jānodrošina iespēja **pievienot produktu izsolei** tikai reģistrētiem un autorizētiem lietotājiem.
- 4.2. Sistēmai jāļauj lietotājiem **veikt solījumus izsolē** reāllaikā, ievērojot minimālo soli.
- 4.3. Sistēmai jāļauj pārvaldniekam **apstiprināt vai noraidīt izsolē pievienotos produktus**.

5. Komentāri un blogs

- 5.1. Sistēmai jāļauj lietotājiem **lasīt bloga rakstus** un apskatīt komentārus.
- 5.2. Sistēmai jāļauj pārvaldniekam **moderēt komentārus** – dzēst, rediģēt vai atzīmēt kā neatbilstošus.
- 5.3. Sistēmai jāļauj pārvaldniekam vai redaktoram **publicēt jaunu bloga rakstu**, pievienot attēlus un formatētu tekstu.
- 5.4. Sistēmai jānodrošina **komentāru kārtošana** pēc datuma, populāruma vai atbilstības.

6. Mākslīgā intelekta asistentam (DeepSeek AI):

Identifikators	PS 2.2.6
Ievads	
Funkcija nodrošina AI asistenta darbību lietotāju jautājumu apstrādei.	
Ievaddati	
1. Lietotāja jautājums	
Apstrāde	
1. Analizē tekstu	
2. Meklē atbildi datubāzē vai ģenerē	
Izvaddati	
1. Atbilde	
2. Ieteikumi	
Kļūdas paziņojumi	
1. Neizdevās saprast jautājumu	
Izsaucamās prasības	
Pieejams visiem lietotājiem ar internetu	

- 6.1. Sistēmai jāpieņem lietotāja **dabiski uzdotus jautājumus** par produktiem, pasūtījumiem vai tehnoloģijām.
- 6.2. Sistēmai jāanalizē jautājuma konteksts un jāsniedz **personalizēta atbilde** balstīta uz produkta datiem vai lietotāja vēsturi.
- 6.3. Sistēmai jāsniedz **produktu ieteikumi**, pamatojoties uz lietotāja interesēm vai iepriekšējiem pirkumiem.
- 6.4. Sistēmai jāsniedz **biežāk uzdoto jautājumu atbildes** un jānovirza lietotājs uz atbilstošām sadaļām.
- 6.5. Sistēmai jānodrošina kļūdas apstrāde – ja jautājumu nav iespējams saprast, jāizvada informatīvs ziņojums.

7. Administrācijas panelim:

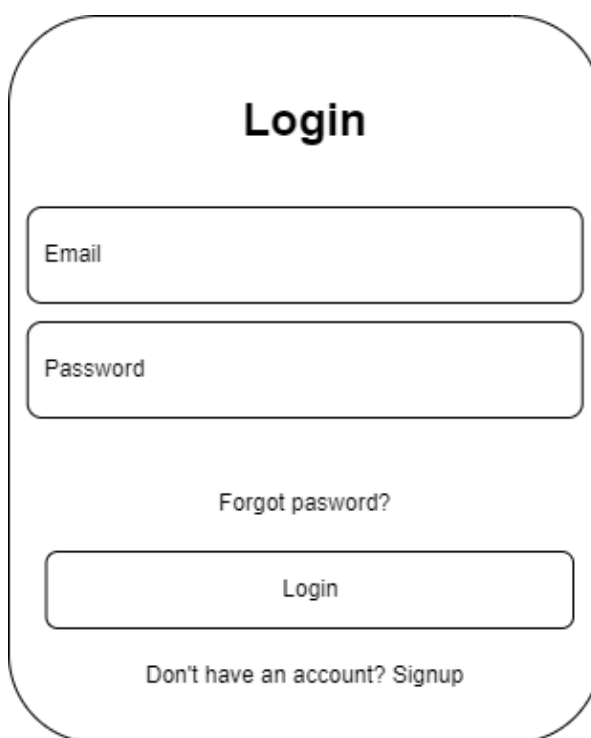
Identifikators	PS 2.2.7
Ievads	
Funkcija paredzēta sistēmas administrēšanai un pārvaldībai.	
Ievaddati	
1. Lietotāju dati 2. Produktu dati 3. Pasūtījumi	
Apstrāde	
1. Redīgē, dzēš vai pievieno datus 2. Ģenerē statistiku	
Izvaddati	
1. Atjaunināti dati 2. Pārskati	
Kļūdas paziņojumi	
1. Nepietiekamas tiesības	
Izsaucamās prasības	
Pieejams tikai administratoriem ar internetu	

- 7.1. Sistēmai jāļauj pārvaldniekam **pārvaldīt lietotājus** – skatīt, redīgēt, bloķēt un dzēst kontus.
- 7.2. Sistēmai jāļauj pārvaldniekam **pievienot, labot un dzēst produktus** no kataloga.
- 7.3. Sistēmai jāļauj pārvaldniekam **pārvaldīt izsoles** – apstiprināt, dzēst vai apturēt.
- 7.4. Sistēmai jāparāda **statistikas pārskati** – pārdošanas apjomi, populārākie produkti, aktīvākie lietotāji.
- 7.5. Sistēmai jānodrošina **piekļuves kontrole**, lai tikai autorizēti pārvaldnieki piekļūtu jutīgām funkcijām.
- 7.6. Sistēmai jāļauj pārvaldniekam **apskatīt maksājumu un transakciju vēsturi** un pārbaudīt to statusu.
- 7.7. Sistēmai jānodrošina, ka tā var izveidot **PDF** izdrukas.

2.3. Nefunkcionālās prasības

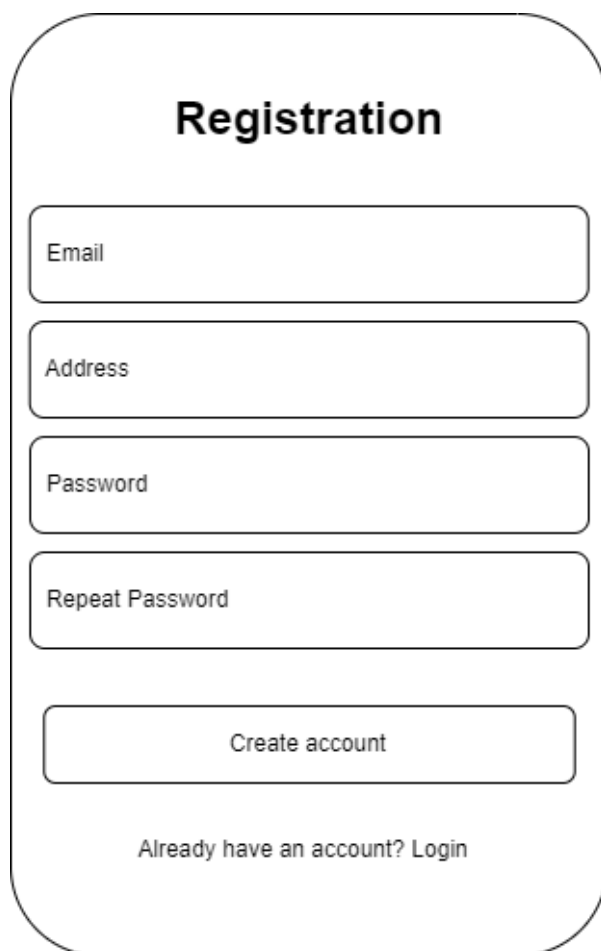
1. Sistēmai jābūt pieejamai angļu valodā.
2. Vietnei jābūt **responsīvai** – pielāgotai darbam datorā, planšetē un mobilajā ierīcē.
3. Darbības ātrumam jānodrošina <2 sek. lapas ielādes laiks normālos apstākļos.
4. Drošība: paroles jāglabā šifrēti, datu pārraidei jāizmanto HTTPS, jānodrošina CSRF un XSS aizsardzība.
5. Sistēmai jābūt saderīgai ar jaunākajām Chrome, Firefox, Edge un Safari versijām.

6. Lietotāja saskarnei jābūt modernai, intuitīvai un vizuāli saskaņotai (tumšais un gaišais režīms, vienota krāsu palete).
7. Sistēmai jāspēj apkalpot vismaz 300 vienlaicīgus lietotājus.
8. Jābūt paziņojumiem par visām kritiskajām darbībām – reģistrāciju, maksājumiem, solišanu, profila izmaiņām.
9. MI asistenta atbildēm jāparādās ne ilgāk kā 7 sekunžu laikā.
10. Platformai jābūt paplašināmai – nākotnē var pievienot jaunas sadaļas, piemēram, “servisa pakalpojumi” vai “lietoto detaļu tirgus”.
11. Lietotāja pieslēgšanās formai jāizskatās sekojoši (skat. 1. un 2. att.).



The image shows a sketch of a login form within a rounded rectangular container. At the top, the word "Login" is centered in a bold, sans-serif font. Below it are two stacked rectangular input fields. The first field is labeled "Email" and the second is labeled "Password", both in a small, sans-serif font. Below the password field, the text "Forgot password?" is centered. Underneath that is a wide rectangular button labeled "Login" in a small, sans-serif font. At the bottom of the form, the text "Don't have an account? Signup" is centered in a small, sans-serif font.

1. att. Lietotāja pieslēgšanās formas skice



Registration

Email

Address

Password

Repeat Password

Create account

Already have an account? Login

2. att. Lietotāja registrēšanas formas skice

12. Lietotāja konta rediģēšanas formai jāizskatās sekojoši (skat. 3.att.).



Information

Email <Email> Name <Name>

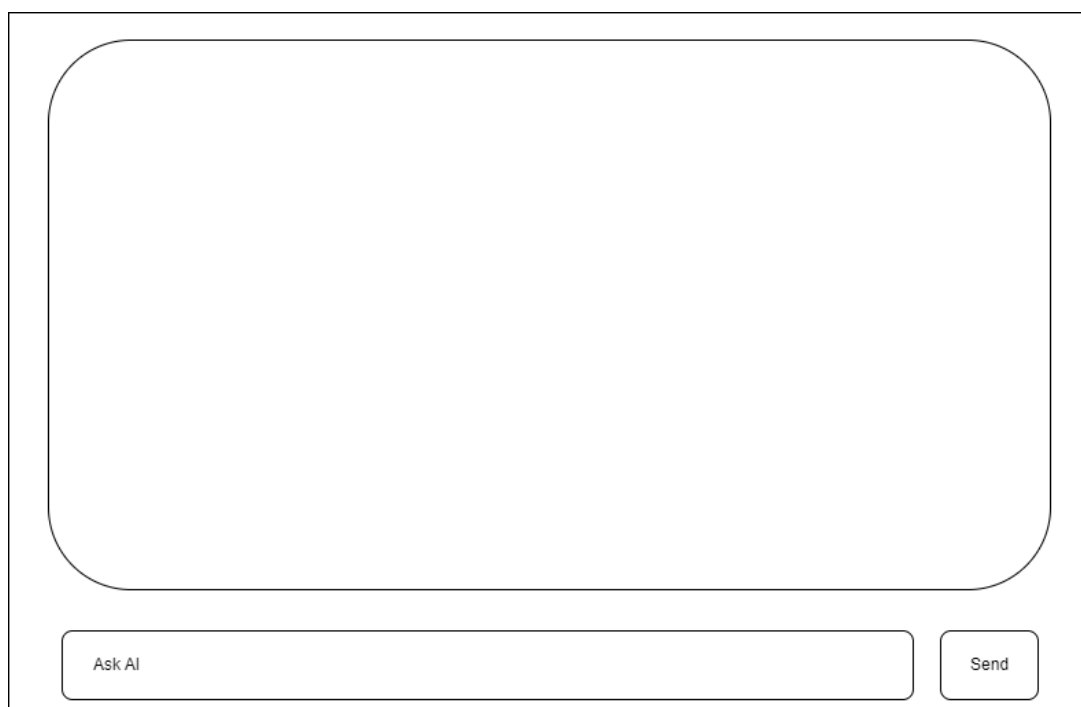
Address <Address>

Click to edit

Delete account

3. att. Lietotāja datu rediģēšanas formas skice

13. MI asistentam jāizskatās sekojoši (skat. 4. att.).



The image shows a wireframe sketch of a user interface for an MI assistant. It consists of a large, empty rectangular area with rounded corners, intended for displaying chat messages. Below this area is a horizontal input bar. On the left side of the input bar is a text field containing the placeholder text "Ask AI". On the right side of the input bar is a button labeled "Send".

4. att. MI asistenta skice

3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS

Programmatūras izstrādē tiek izmantotas PHP, HTML, CSS, JavaScript un MySQL, jo šīs tehnoloģijas nodrošina stabilu, plaši izmantotu un labi dokumentētu vidi tīmekļa lietojumprogrammu izstrādei. Projekta realizācijai tiek izmantoti mūsdienīgi ietvari Laravel un Vue.js, kas ļauj izstrādāt drošu, mērogojamu un lietotājam draudzīgu e-komercijas platformu ar dinamisku lietotāja saskarni

1. Servera puses izstrādei (*backend*):

- **Laravel 11.x (PHP ietvars)** ir izvēlēts kā galvenais servera puses izstrādes ietvars, jo tas balstās uz MVC arhitektūras principiem, kas nodrošina koda strukturētību, uzturējamību un paplašināmību. Laravel piedāvā iebūvētas drošības funkcijas, piemēram, CSRF aizsardzību, piekļuves kontroli un validācijas mehānismus, kas ir īpaši svarīgi e-komercijas sistēmās. Turklāt ietvars nodrošina ērtu REST API izstrādi, kas ir nepieciešama frontend un backend savstarpējai komunikācijai.
- **PHP 8.2+** tiek izmantots kā servera puses programmēšanas valoda, jo tā ir plaši izplatīta, labi piemērota tīmekļa lietojumprogrammām un pilnībā saderīga ar Laravel ietvaru. Jaunākās PHP versijas nodrošina uzlabotu veiktspēju, tipēšanu un kļūdu apstrādi, kas palielina sistēmas stabilitāti un drošību.
- **MySQL 8.0** ir izvēlēta kā relāciju datu bāzes pārvaldības sistēma, jo tā nodrošina augstu veiktspēju, datu integritāti un uzticamību. MySQL ir piemērota strukturētu datu, piemēram, lietotāju, produktu, pasūtījumu un izsoļu informācijas, glabāšanai. Datu bāzes integrācija ar Laravel Eloquent ORM atvieglo datu apstrādi un samazina kļūdu iespējamību.
- **phpMyAdmin** tiek izmantots kā grafiska datu bāzes administrēšanas vide, jo tā ļauj ērti pārvaldīt datu struktūras, veikt vaicājumus un analizēt datus bez nepieciešamības izmantot komandrindu.
- **Laravel Sanctum** ir izvēlēts lietotāju autentifikācijas nodrošināšanai, jo tas piedāvā drošu token-based autentifikāciju, kas ir piemērota Single Page Application (SPA) risinājumiem un API aizsardzībai.

2. Lietotāja puses izstrādei (*frontend*):

- **Vue.js 3.x** ir izvēlēts kā frontend ietvars, jo tas nodrošina reaktīvu un komponentēs balstītu lietotāja saskarnes izstrādi. Vue.js ir viegli apgūstams, elastīgs un labi integrējams ar Laravel API, kas ļauj izveidot ātru un intuitīvu lietotāja interfeisu e-komercijas platformai.
- **HTML5** tiek izmantots tīmekļa lapu struktūras veidošanai, jo tas nodrošina semantisku atzīmēšanu un uzlabo lapas pieejamību un saderību ar dažādām ierīcēm.
- **CSS3** tiek izmantots lietotāja saskarnes vizuālajam noformējumam un responsīva dizaina nodrošināšanai, kas ļauj platformu ērti lietot gan datoros, gan mobilajās ierīcēs.
- **JavaScript (ES6+)** tiek izmantots klienta puses loģikas realizācijai, nodrošinot dinamisku satura atjaunošanu, lietotāja darbību apstrādi un mijiedarbību ar backend API.

3. Izstrādes vides un rīki:

- **XAMPP** tiek izmantots kā lokālās izstrādes vides risinājums, jo tas apvieno Apache, PHP un MySQL vienā paketē, nodrošinot ātru un ērtu izstrādes uzsākšanu bez sarežģītas konfigurācijas.
- **PhpStorm 2024.3.2.1** ir izvēlēta kā integrētā izstrādes vide, jo tā piedāvā plašu funkcionalitāti PHP un JavaScript izstrādei, tostarp koda automātisko papildināšanu, kļūdu analīzi un integrāciju ar Git versiju kontroles sistēmu.
- **Composer** tiek izmantots PHP atkarību pārvaldībai, jo tas ļauj efektīvi instalēt un uzturēt Laravel un citu bibliotēku komponentes.
- **Node.js un NPM** tiek izmantoti frontend atkarību pārvaldībai un Vue.js komponentu izstrādei, nodrošinot modernu un efektīvu frontend izstrādes procesu.
- **Vite** ir izvēlēts kā frontend būvēšanas rīks, jo tas nodrošina ātru projektu kompilāciju un efektīvu resursu ielādi izstrādes un produkcijas vidē.
- **Git un GitHub** tiek izmantoti versiju kontrolei un koda pārvaldībai, kas ļauj sekot līdzi izmaiņām, atgriezties pie iepriekšējām versijām un nodrošināt drošu koda uzglabāšanu.

4. Projekta struktūrai:

- Backend – Laravel API ar JSON atbildēm;
- Frontend – Vue.js Single Page Application (SPA);
- Datu bāze – MySQL;
- Autentifikācija – Sanctum token-based authentication.

5. Uzdevumu risināšanas līdzekļu pamatojums.

Programmatūras izstrāde tiks veikta, izmantojot PHP, HTML, CSS, JavaScript, MySQL. Projekts tiks realizēts ar moderniem ietvariem - Laravel un Vue.js, lai nodrošinātu dinamisku lietotāja interfeisu. Galvenie rīki un tehnoloģijas:

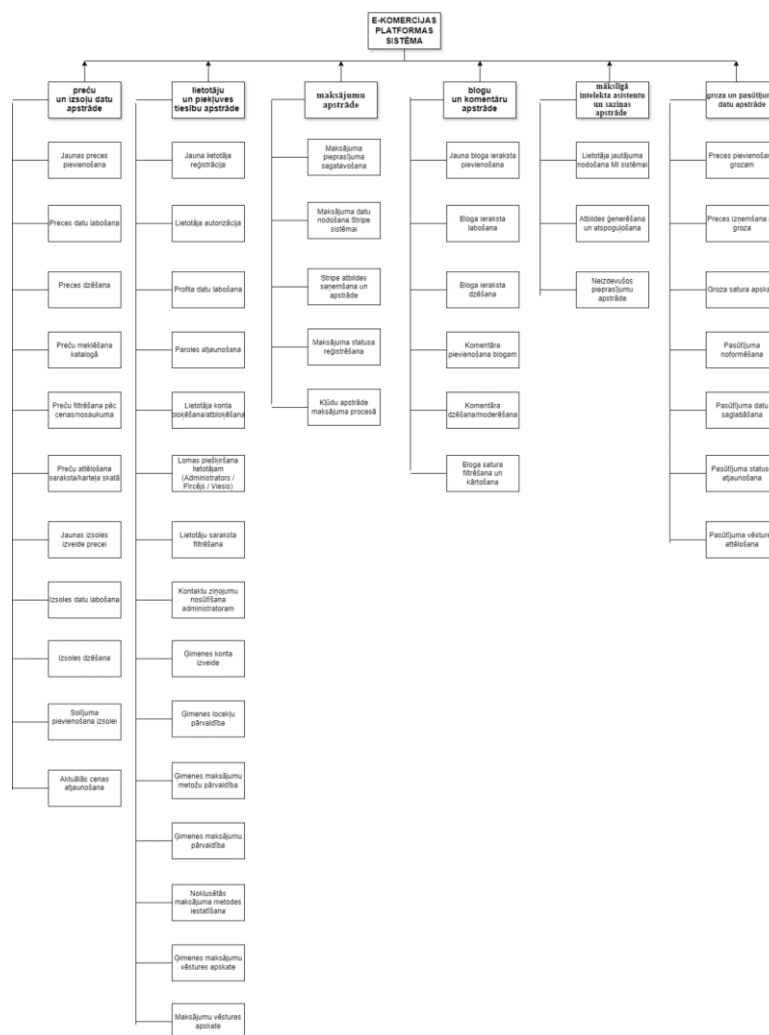
- Laravel (PHP ietvars) – nodrošina drošu servera puses loģiku, autentifikāciju, datubāzes pārvaldību un API izstrādi;
- Vue.js (JavaScript ietvars) – lietotāja interfeisa veidošanai, kas ir responsīvs un viegli uztverams;
- MySQL datubāze – lietotāju, produktu un pasūtījumu datu glabāšanai;
- phpMyAdmin – grafiska datu bāzes administrācijas vides izmantošanai;
- HTML un CSS – vizuālā dizaina izstrādei;
- XAMPP – serveru un pārvaldnieka panelis vienā lietotājprogrammā, kas ir ērts un ātrs variants izstrādātājiem. Ar to ir vieglāk pārvaldīt datubāzi ar phpMyAdmin;
- PhpStorm 2024.3.2.1 – ļoti zināms kodu redaktors komandai, līdz ar to šī ir ērtāka platforma PHP risinājumiem;
- GitHub –versiju kontroles un pārvaldībai.

4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA

4.1. Sistēmas struktūras modelis

4.1.1. Sistēmas arhitektūra

Sistēma ir veidota kā modulāra platforma, kas sastāv no sešām galvenajām apakšsistēmām (skat. 5. att., vai pielikumos, 2. pielikums), kas savstarpēji sadarbojas, lai nodrošinātu pilnīgu funkcionalitāti e-komercijas un izsoļu platformai. Šīs apakšsistēmas ietver preču un izsoļu datu apstrādi, lietotāju, ģimenes kontu un piegāves tiesību pārvaldību, maksājumu apstrādi, blogu un komentāru administrēšanu, mākslīgā intelekta asistenta un saziņas funkcionalitāti, kā arī groza un pasūtījumu loģiku. Katra no apakšsistēmām ir specializēta noteiktu uzdevumu veikšanai, nodrošinot sistēmas darbības efektivitāti, drošību un lietotāju ērtības.



5. att. Funkcionālās dekompozīcijas diagramma

1. Preču un izsoļu pārvaldība nodrošina visu preču kataloga un izsoļu mehānisma funkcionalitāti. Šī apakšsistēma atbalsta preču pievienošanu, rediģēšanu un dzēšanu, ļaujot pārvaldniekiem un pārdevējiem ērti uzturēt aktuālu preču sarakstu. Turklāt apakšsistēma nodrošina preču filtrēšanas un meklēšanas funkcionalitāti, kas ļauj lietotājiem ātri atrast nepieciešamos produktus pēc dažādiem kritērijiem, piemēram, kategorijas, cenas vai popularitātes. Izsoļu mehānisms tiek pārvaldīts šajā apakšsistēmā, ļaujot izveidot jaunas izsoles, noteikt izsoles parametrus un nodrošināt to regulāru atjaunināšanu un uzraudzību. Tā ietver:

- preču pievienošanu, rediģēšanu un dzēšanu;
- preču filtrēšanu un meklēšanu;
- izsoļu izveidi un uzturēšanu.

2. Lietotāju un piekļuves pārvaldība nodrošina visaptverošu lietotāju autentifikācijas un autorizācijas sistēmu, kā arī lomu administrēšanu. Šī apakšsistēma atbild par lietotāju reģistrāciju, pieslēgšanos, vienlaikus ļaujot lietotājiem rediģēt savus profilus. Lietotāji tiek klasificēti pēc lomām, piemēram, viesis, reģistrēts lietotājs vai pārvaldnieks, kas ļauj pielāgot piekļuves tiesības atbilstoši lietotāja statusam. Īpaša uzmanība tiek pievērsta ģimenes kontu pārvaldībai, kas ļauj izveidot kontus vairākām personām vienā ģimenē, pievienot ģimenes locekļus, konfigurēt piekļuves tiesības apakšlietotājiem, pievienot maksājumu metodes un veikt centralizētus ģimenes maksājumus, kā arī pārskatīt visu ģimenes konta maksājumu vēsturi. Tā ietver:

- lietotāja reģistrāciju un pieslēgšanos;
- paroles atjaunošanu;
- profila rediģēšanu;
- lietotāju lomu piešķiršanu (viesis, reģistrēts lietotājs, pārvaldnieks);
- ģimenes kontu pārvaldību un piekļuves tiesību konfigurēšanu apakšlietotājiem;
- ģimenes konta izveidi;
- ģimenes locekļu pievienošanu;
- maksājuma metodes pievienošanu;
- ģimenes maksājumu veikšanu;
- ģimenes maksājumu vēstures apskati.

3. Groza un pasūtījumu pārvaldība nodrošina pirkuma procesa loģiku no preču izvēles līdz pasūtījuma izpildei. Lietotāji var pievienot preces grozam, noformēt pasūtījumu un sekot tam visā tā izpildes procesā. Sistēma saglabā pasūtījumu vēsturi, ļaujot lietotājiem jebkurā brīdī pārskatīt savus iepriekšējos pirkumus. Ģimenes kontu gadījumā pasūtījumu informācija tiek

centralizēti pārvaldīta, kas ļauj uzraudzīt un analizēt ģimenes kopējos pirkumus, vienkāršojot finanšu kontroli un pārvaldību. Tā ietver:

- preču pievienošanu grozam;
- pasūtījuma noformēšanu;
- pasūtījuma statusa uzskaiti;
- pasūtījumu vēstures saglabāšanu;
- ģimenes konta ietvaros veikto pasūtījumu centralizētu pārskatīšanu.

4. Maksājumu apstrādes apakšsistēma nodrošina drošu finanšu darījumu izpildi un uzraudzību. Sistēma integrējas ar Stripe maksājumu platformu, nodrošinot gan darījumu veikšanu, gan to validāciju. Apakšsistēma pārbauda darījumu statusu, sinhronizē to ar sistēmu un saglabā maksājumu informāciju datu bāzē, nodrošinot pilnīgu uzskaiti un atbilstību drošības prasībām. Tā ietver:

- Stripe maksājumu sistēmas integrāciju;
- maksājuma datu validāciju;
- darījuma statusa pārbaudi un sinhronizāciju;
- maksājumu informācijas saglabāšanu datu bāzē.

5. Bloga un komentāru pārvaldība sniedz iespēju publicēt informatīvu saturu un veicina lietotāju iesaisti platformā. Pārvaldnieki var publicēt bloga ierakstus. Papildus tiek nodrošināta komentāru moderēšana pārvaldnieka līmenī, lai saglabātu platformas saturu kvalitatīvu un atbilstošu. Tā ietver:

- bloga ierakstu publicēšanu;
- komentāru pievienošanu un rediģēšanu;
- komentāru moderēšanu pārvaldnieka līmenī.

6. Mākslīgā intelekta asistenta un saziņas apakšsistēma nodrošina lietotāju atbalstu un interaktīvu komunikāciju. Sistēma ietver DeepSeek mākslīgā intelekta asistenta pieprasījumu apstrādi un atbilžu attēlošanu lietotāja saskarnē, ļaujot lietotājiem saņemt ātru un precīzu atbalstu, kā arī veicinot efektīvu mijiedarbību ar platformu. Tā ietver:

- DeepSeek mākslīgā intelekta asistenta pieprasījumu apstrādes;
- atbilžu attēlošanu lietotāja saskarnē.

4.1.2. Sistēmas ER modelis

Sistēmas ER-modelis sastāv no 13 galvenajām entītijām, kas nodrošina e-komercijas, izsoļu un ģimenes maksājumu funkcionalitāti (3. pielikums):

- Entītijas “**LIETOTĀJS**” atribūtu kopums sevī ietver lietotāja vārdu, e-pasta adresi, paroli, adresi, lomu sistēmā un saiti ar ģimeni. Lietotājs sistēmā var rakstīt komentārus, veikt pasūtījumus, izveidot izsoles, iesniegt likmes, pievienot ģimenes maksājuma metodes un veikt ģimenes transakcijas;
- Entītijas “**ĢIMENE**” atribūti ietver ģimenes nosaukumu, uzaicinājuma kodu, vecāka identifikatoru, Stripe klienta identifikatoru, maksimālo dalībnieku skaitu un aktivitātes statusu. Ģimene apvieno vairākus lietotājus kopīgai maksājumu sistēmai;
- Entītijas “**ĢIMENES_MAKSĀJUMA_METODE**” ietver atribūtus Stripe maksājuma metodes identifikators, kartes pēdējie četri cipari, kartes veids un noklusējuma statuss. Šī entītijas glabā ģimenes kopīgās maksājumu kartes un informāciju par lietotāju, kurš tās pievienojis;
- Entītijas “**ĢIMENES_MAKSĀJUMI**” ietver atribūtus maksājuma identifikators, summa, valūta, apraksts un statuss. Tā saglabā informāciju par maksājumiem, kas veikti ģimenes vārdā;
- Entītijas “**IZSOLE**” atribūti ietver nosaukumu, aprakstu, sākuma cenu, attēlu, sākuma un beigu datumu. Izsoles tiek veidots lietotāja profilā un satur informāciju par aktīvajām vai pabeigtajām izsolēm;
- Entītijas “**LIKTENIS**” satur atribūtus solītā summa un datums, un tā sasaista izsoli ar lietotāju, kurš veicis likmi. Viena izsole var saturēt vairākas likmes, un viens lietotājs var izdarīt vairākus solījumus;
- Entītijas “**KOMENTĀRS**” atribūti ietver tekstu, un tā ir saistīta ar lietotāju, kas to uzrakstījis. Vienam lietotājam var būt vairāki komentāri;
- Entītijas “**PASŪTĪJUMS**” atribūtu kopums ietver kopējo summu, statusu, piegādes adresi un maksājuma identifikatoru. Pasūtījums saglabā informāciju par lietotāja veiktajiem pirkumiem;
- Entītijas “**PRODUKTS**” ietver atribūtus nosaukums, apraksts, cena, attēls un kategorija. Produkti var būt ietverti vairākos pasūtījumos;
- Entītijas “**KATEGORIJA**” atribūti ietver nosaukumu, un katra kategorija satur vairākus produktus, kas pie tās piesaistīti;

- Entītija “**GRĀMATAS**” ietver atribūtus nosaukums, apraksts, faila ceļš un attēls. Šie ieraksti tiek pievienoti pārvaldnieka uzraudzībā un kalpo kā papildu informācijas saturs platformā;
- Entītija “**PRODUKTI_PASŪTĪJUMI**” ir saite starp produktiem un pasūtījumiem, kas ietver atribūtus daudzums un cena, un nodrošina daudz-pret-daudz attiecību starp šīm entītijām.
- Entītija “**PĀRVALDNIKES**” ir saite starp produktiem un grāmatām, nodrošina ar saiti viens-pret-daudziem attiecību starp šīm entītijām. (**ER diagrammā** pārvaldnieks ir saistīts tikai ar tiem datu objektiem, kuros tiek tieši veikta datu izveide vai modificēšana (piemēram, produkti un raksti). Pārējās darbības tiek realizētas **sistēmas loģikas līmenī un netiek attēlotas ER modelī.**)

Saites un kardinalitātes:

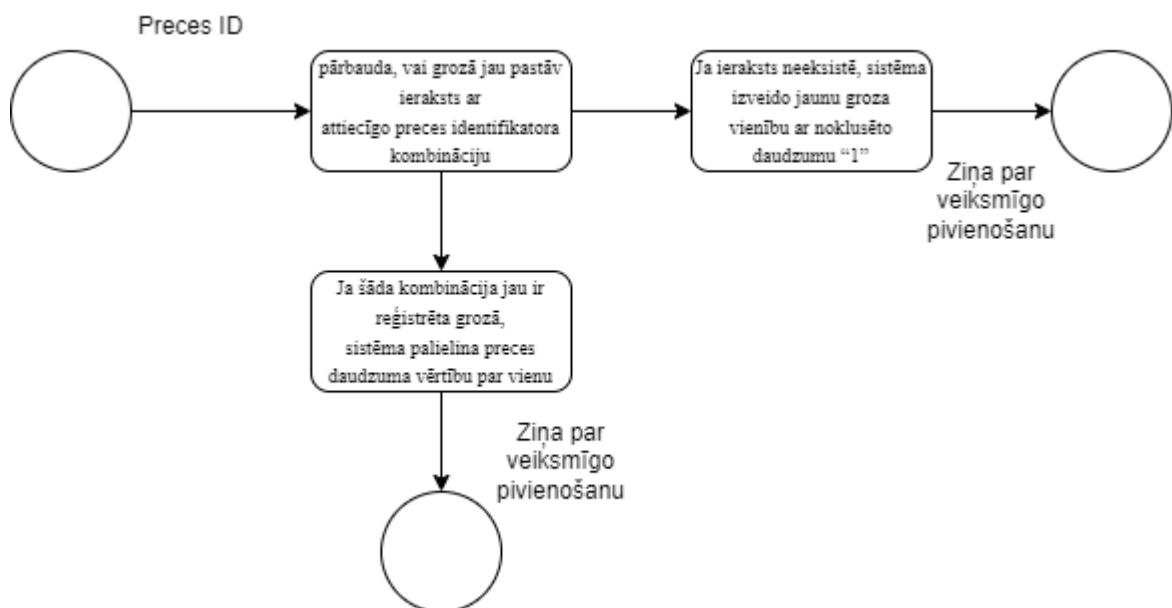
- Starp “LIETOTĀJU” un “ĢIMENI” pastāv saite viens pret daudziem, jo viena ģimene var saturēt vairākus lietotājus, bet lietotājs var piederēt tikai vienai ģimenei;
- Starp “ĢIMENI” un “ĢIMENES_MAKSĀJUMA_METODES” pastāv saite viens pret daudziem, jo viena ģimene var izmantot vairākas maksājuma metodes;
- Starp “LIETOTĀJU” un “ĢIMENES_MAKSĀJUMA_METODES” pastāv saite viens pret daudziem, jo viens lietotājs var pievienot vairākas maksājuma metodes ģimenes vajadzībām;
- Starp “ĢIMENI” un “ĢIMENES_MAKSĀJUMI” pastāv saite viens pret daudziem, jo viena ģimene var veikt vairākas transakcijas;
- Starp “LIETOTĀJU” un “IZSOLI” pastāv bināra saite ar kardinalitāti viens pret daudziem, jo viens lietotājs var izveidot vairākas izsoles, bet katra izsole pieder tikai vienam lietotājam;
- Starp “IZSOLI” un “LIKTENI” pastāv saite viens pret daudziem, jo viena izsole var saturēt vairākas likmes, bet katra likme attiecas uz vienu izsoli;
- Starp “LIETOTĀJU” un “LIKTENI” pastāv saite viens pret daudziem, jo viens lietotājs var izdarīt vairākus solījumus dažādās izsolēs;
- Starp “LIETOTĀJU” un “KOMENTĀRU” pastāv saite viens pret daudziem, jo viens lietotājs var rakstīt vairākus komentārus;
- Starp “LIETOTĀJU” un “PASŪTĪJUMU” pastāv saite viens pret daudziem, jo viens lietotājs var veikt vairākus pasūtījumus, bet katrs pasūtījums ir saistīts ar vienu lietotāju;

- Starp “PASŪTĪJUMU” un “PRODUKTI_PASŪTĪJUMI” pastāv saite viens pret daudziem, jo katrā pasūtījumā var būt vairāki produktu ieraksti;
- Starp “PRODUKTU” un “PRODUKTI_PASŪTĪJUMI” pastāv saite viens pret daudziem, jo viens produkts var būt iekļauts vairākos pasūtījumos;
- Starp “PASŪTĪJUMU” un “ĢIMENES_MAKSĀJUMI” pastāv saite viens pret vienu, jo katrs pasūtījums tiek apmaksāts ar vienu konkrētu ģimenes transakciju, un katra ģimenes transakcija attiecas uz vienu pasūtījumu.
- Starp “PASŪTĪJUMU” un “ĢIMENES_MAKSĀJUMI” pastāv saite viens pret vienu, jo vienam pasūtījumam pieder viena transakcija;
- Starp “PĀRVALDNIIEKS” un “PRODUKTS” pastāv saite viens pret daudziem (1:N), jo viens pārvaldnieks var pievienot un pārvaldīt vairākus produktus, bet katrs produkts ir saistīts ar vienu pārvaldnieku.
- Starp “PĀRVALDNIIEKS” un “RAKSTS” pastāv saite viens pret daudziem (1:N), jo viens pārvaldnieks var izveidot vairākus rakstus, bet katrs raksts pieder vienam pārvaldniekam.

4.2. Funkcionālais sistēmas modelis

4.2.1. Datu plūsmu modelis

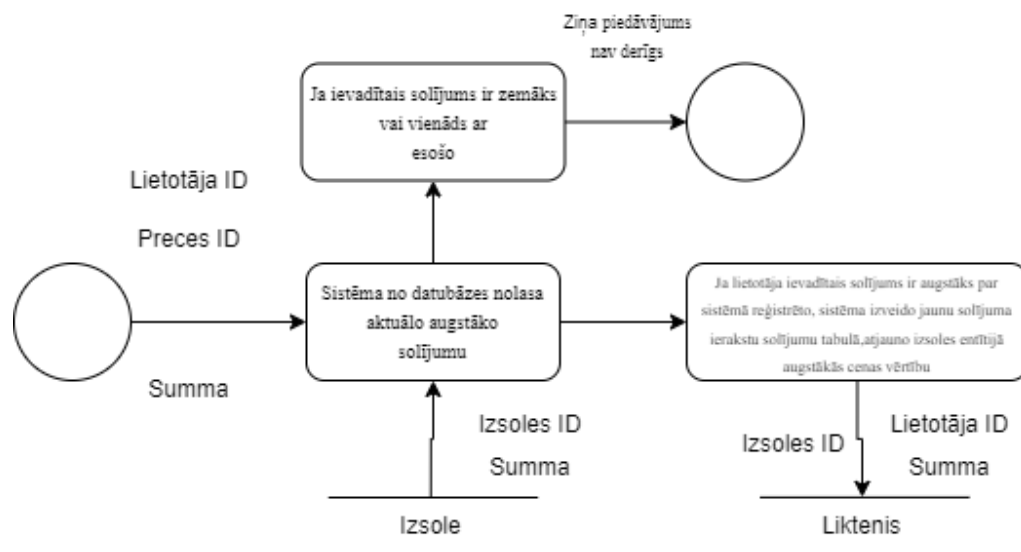
1) Produkta pievienošana iepirkumu grozam (skat. 6. att.).



6. att. Produkta pievienošana iepirkumu grozam

Saskarne nodod preces identifikatoru. Sistēma pārbauda, vai grozā(logiska glabātuve) jau pastāv ieraksts ar attiecīgo preces identifikatora kombināciju. Ja ieraksts neeksistē, sistēma izveido jaunu groza vienību ar noklusēto daudzumu "1". Ja šāda kombinācija jau ir reģistrēta grozā, sistēma palielina preces daudzuma vērtību par vienu un atjauno esošo ierakstu. Pēc datu apstrādes lietotājam tiek atspoguļots apstiprinājuma ziņojums par preces veiksmīgu pievienošanu grozam.

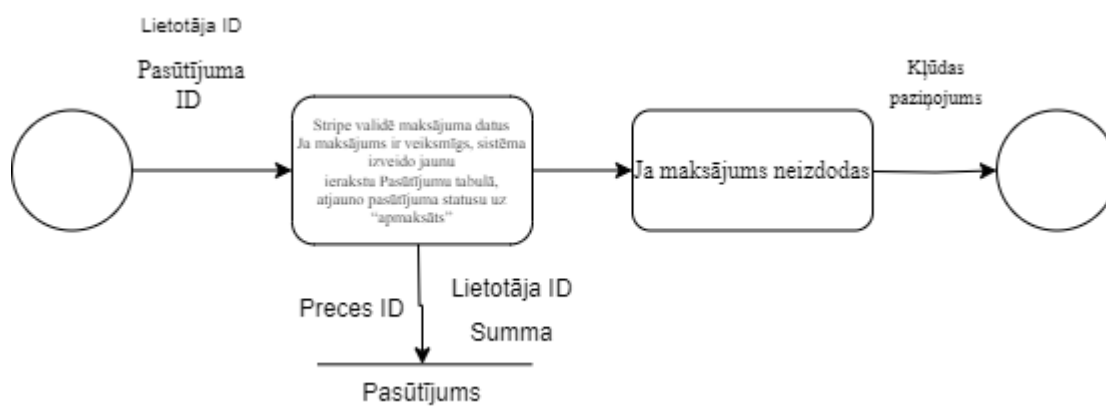
2) Cenas piedāvājuma pievienošana izsolei (skat. 7. att.).



7. att. Cenas piedāvājuma pievienošana izsolei

Saskarne nodod lietotāja identifikatoru, izsoles identifikatoru un jaunā solījuma summu. Sistēma no datubāzes nolasa aktuālo augstāko solījumu konkrētajai izsolei. Ja lietotāja ievadītais solījums ir augstāks par sistēmā reģistrēto, sistēma izveido jaunu solījuma ierakstu solījumu tabulā, pievienojot laika zīmogu un lietotāja identifikatoru, un atjauno izsoles entitijā augstākās cenas vērtību. Ja ievadītais solījums ir zemāks vai vienāds ar esošo, sistēma atgriež informatīvu ziņojumu, ka piedāvājums nav derīgs.

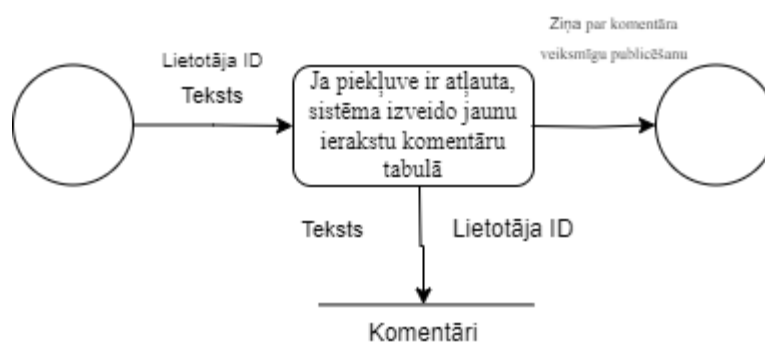
3) Maksājuma apstrāde ar Stripe (skat. 8. att.).



8. att. Maksājuma apstrāde ar Stripe

Saskarne nodod pasūtījuma identifikatoru un lietotāja maksājuma datus. Sistēma sagatavo maksājuma pieprasījumu un nodod to ārējai Stripe maksājumu apstrādes sistēmai. Stripe (ārējais avots) validē maksājuma datus un atgriež sistēmai atbildi ar darījuma statusu. Ja maksājums ir veiksmīgs, sistēma izveido jaunu ierakstu "Pasūtījums" entītijā, saglabājot, statusu un datumu, preces ID, kā arī atjauno pasūtījuma statusu uz "apmaksāts". Ja maksājums neizdodas, lietotājam tiek izvadīts kļūdas paziņojums.

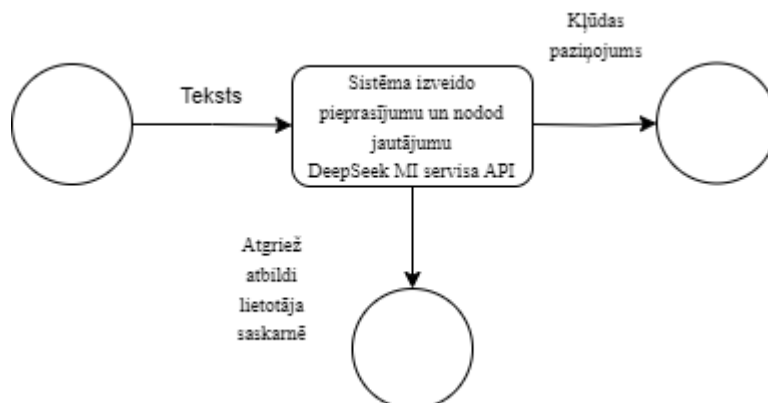
4) Komentāra pievienošana (skat. 9 .att.).



9 .att. Komentāra pievienošana

Saskarne nodod lietotāja identifikatoru un komentāra tekstu. Sistēma pārbauda, vai lietotājam ir tiesības pievienot komentārus. Ja piekļuve ir atļauta, sistēma izveido jaunu ierakstu komentāru tabulā un lietotāja identifikatora, saglabājot arī pievienošanas datumu. Pēc ieraksta izveides lietotājam tiek atspoguļots paziņojums par komentāra veiksmīgu publicēšanu.

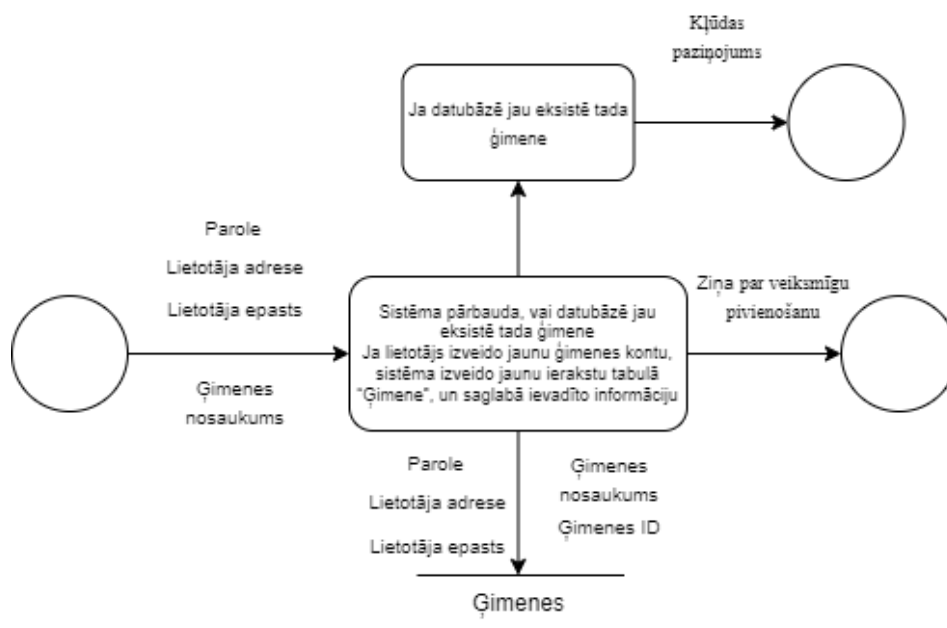
5) MI asistenta pieprasījuma apstrāde (skat. 10. att.).



10. att. MI asistenta pieprasījuma apstrāde

Saskarne nodod lietotāja jautājumu mākslīgā intelekta apakšsistēmai. Sistēma izveido pieprasījumu un nodod jautājumu DeepSeek MI servisa API. Pēc atbildes saņemšanas sistēma pievieno pieprasījuma ierakstam ģenerēto atbildi un atgriež to lietotāja saskarnē. Ja mākslīgā intelekta serviss neatgriež derīgu rezultātu, sistēma izvada informatīvu kļūdas ziņojumu.

6) Jaunas ģimenes autorizācija (skat. 11. att.).



11. att. Jaunas ģimenes autorizācija

Saskarne nodod lietotāja ievadītos datus par ģimenes konta izveidi vai pievienošanos. Sistēma saņem datus un pārbauda, vai datubāzē jau eksistē ģimenes ieraksts.

Ja lietotājs izveido jaunu ģimenes kontu, sistēma izveido jaunu ierakstu tabulā “Ģimene”, piešķir unikālu identifikatoru un saglabā ievadīto informāciju. Pēc tam sistēma piesaista lietotāju šai ģimenei, atjauninot lietotāja ierakstu datubāzē.

5. DATU STRUKTŪRU APRAKSTS

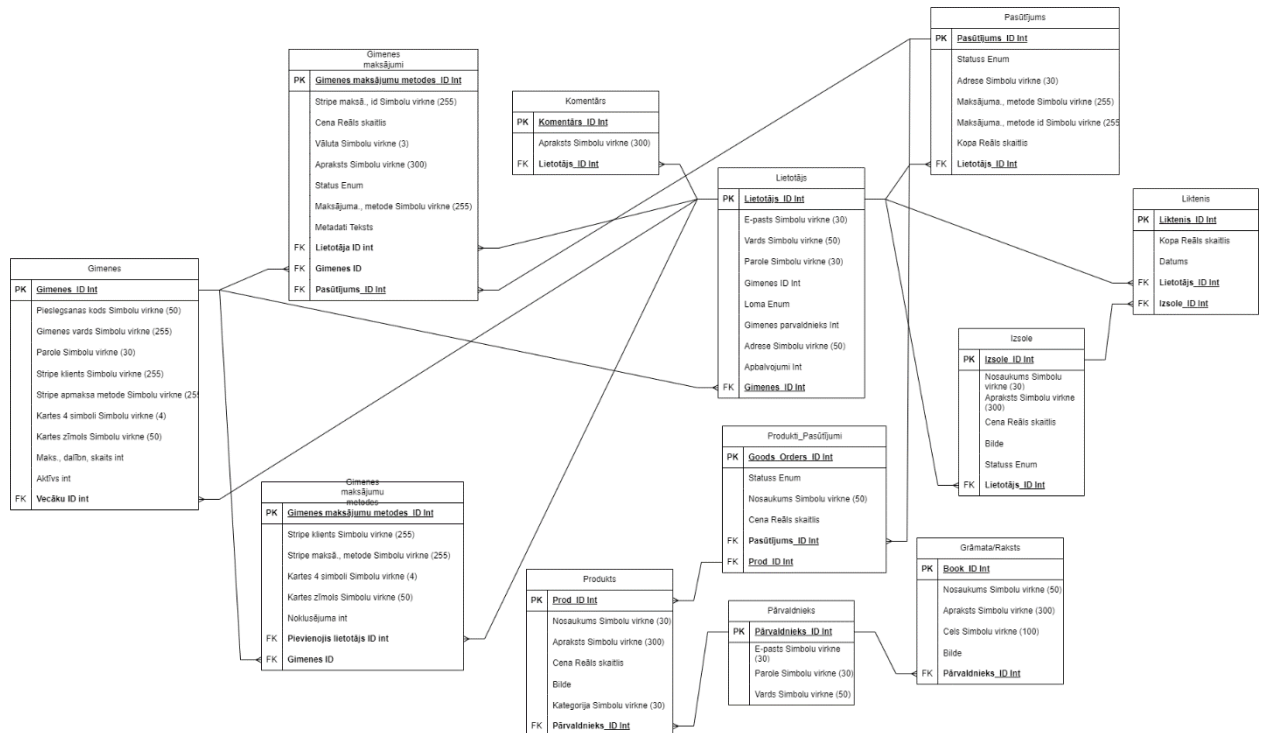
Comtem e-veikala datu bāze ir izstrādāta, lai nodrošinātu efektīvu produktu, pasūtījumu, izsoļu un lietotāju pārvaldību, kā arī kvalitatīvu klientu apkalpošanu. Tā ļauj strukturēti uzglabāt un apstrādāt datus, kas nepieciešami e-komercijas platformas darbībai, tostarp pārdošanas procesa vadībai, produktu uzskaiti, izsoļu norisei un satura administrēšanai

Datubāzes projektēšanas procesā tika izveidotas vairākas savstarpēji saistītas tabulas, un tika definētas to relācijas, kas attēlotas relāciju tabulu diagrammā (skat. 12. att. vai pielikumos, 4. pielikums). Šī diagramma sniedz pārskatu par datu struktūru, tabulu savstarpējām saitēm un izmantotajām primārajām un ārējām atslēgām.

Zemāk ir aprakstītas galvenās datubāzes tabulas:

- Tabula **“Lietotājs”** glabā informāciju par sistēmas lietotājiem. Tā satur lietotāja identifikatoru, e-pastu, vārdu, paroli, lomu, adresi, ģimenes identifikatoru un citus ar lietotāju saistītus datus.
- Tabula **“Ģimene”** glabā informāciju par ģimenes kontiem. Tā ietver ģimenes nosaukumu, piekļuves kodu, Stripe klienta identifikatoru, dalībnieku skaitu un statusu.
- Tabula **“Ģimenes maksājumu metodes”** satur informāciju par ģimenes kopīgajām maksājumu metodēm. Tajā tiek glabāti kartes dati (piemēram, pēdējie cipari), maksājuma veids un saite ar ģimeni.
- Tabula **“Ģimenes maksājumi”** glabā informāciju par visiem maksājumiem, kas veikti ģimenes vārdā, tostarp maksājuma summu, valūtu, statusu un saistīto lietotāju.
- Tabula **“Produkts”** satur informāciju par visām pieejamajām precēm. Tajā ietilpst produkta nosaukums, apraksts, cena, attēls un kategorija.
- Tabula **“Pasūtījums”** glabā informāciju par lietotāju veiktajiem pasūtījumiem. Tā ietver pasūtījuma ID, statusu, kopējo summu, adresi un maksājuma metodi.
- Tabula **“Produkti pasūtījumi”** ir starptabula, kas savieno produktus ar pasūtījumiem. Tā glabā informāciju par pasūtīto produktu daudzumu un cenu konkrētā pasūtījumā.
- Tabula **“Izsole”** glabā informāciju par izsolēm, tostarp nosaukumu, aprakstu, sākuma cenu, sākuma un beigu datumu, kā arī statusu.
- Tabula **“Liktenis”** satur informāciju par lietotāju veiktajiem solījumiem izsolēs. Tajā tiek glabāta solītā summa, datums un saites uz lietotāju un izsoli.
- Tabula **“Komentārs”** glabā lietotāju komentārus, kas pievienoti sistēmā. Tā ietver komentāra tekstu un saiti uz lietotāju.
- Tabula **“Grāmata/Raksts”** glabā bloga vai informatīvos rakstus. Tajā ietilpst nosaukums, apraksts, saturs un saite uz pārdevēju vai autoru.

- Tabula “**Pārvaldnieks**” glabā informāciju par sistēmas pārvaldniekiem.



12. att. Relācijas tabulu diagramma

Datubāze sastāv no vairākām tabulām, kas satur informāciju par lietotājiem, produktiem, pārvaldniekiem, pasūtījumiem, izsolēm, groziem, komentāriem un grāmatām, nodrošinot to savstarpējo sasaisti:

Tabula “**Lietotājs**” satur informāciju par reģistrētajiem sistēmas lietotājiem un sastāv no 11 laukiem. Tā ir centrālā tabula datu bāzē un ir savienota ar tabulām “Komentārs”, “Pasūtījums”, “Izsole” un “Liktenis” ar saiti viens-pret-daudziem, jo viens lietotājs var rakstīt vairākus komentārus, veikt vairākus pasūtījumus, izveidot vairākas izsoles un iesniegt vairākas likmes.

7. tabula

Tabulas “**Lietotājs**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1	ID	bigint unsigned	20	Primārā atslēga, lietotāja ID
2	Ģimenes ID	bigint unsigned	20	Ārējā atslēga uz ģimeni
3	Loma	enum	-	Lietotāja loma sistēmā (parent, child, standalone)
4	Ģimenes pārvaldnieks	tinyint	1	Vai lietotājs ir ģimenes pārvaldnieks
5	Var izmantot ģimenes karti	tinyint	1	Vai lietotājs drīkst izmantot ģimenes maksājumu karti
6	Vārds	varchar	255	Lietotāja vārds
7	E-pasts	varchar	255	Lietotāja e-pasts
8	E-pasts apstiprināts	timestamp	-	Datums, kad e-pasts apstiprināts
9	Adrese	varchar	255	Lietotāja adrese
10	Balvas	int	11	Lietotāja nopelnītās balvas
11	Parole	varchar	255	Šifrēta parole

Tabula “**Komentārs**” sastāv no 3 laukiem un satur lietotāju veidoto komentāru tekstus. Tā ir savienota ar tabulu “Lietotājs” ar saiti daudzi-pret-vienu, jo katrs komentārs pieder vienam lietotājam, bet viens lietotājs var izveidot vairākus komentārus.

8. tabula

Tabulas “**Komentārs**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1	ID	bigint unsigned	20	Primārā atslēga, komentāra ID
2	Teksts	text	-	Komentāra saturs
3	Lietotāja ID	bigint unsigned	20	Ārējā atslēga uz lietotāju

Tabula “**Produkts**” sastāv no 7 laukiem un satur informāciju par e-veikalā piedāvātajām precēm. Tā ir savienota ar tabulu “Satur” ar saiti viens-pret-daudziem, jo viens produkts var būt iekļauts vairākos pasūtījumos, kā arī ar tabulu “Pārvaldnieks” ar saiti daudzi-pret-vienu, jo viens pārvaldnieks var pievienot vairākus produktus.

9. tabula

Tabulas “**Produkts**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1	ID	bigint unsigned	20	Primārā atslēga, produkta ID
2	Nosaukums	varchar	255	Produkta nosaukums
3	Cena	decimal	8,2	Produkta cena
4	Apraksts	text	-	Produkta apraksts
5	Attēls	varchar	255	Produkta attēls
6	Kategorija	varchar	255	Produkta kategorija
7	Pārvaldnieks ID	bigint unsigned	20	Ārējā atslēga uz pārvaldnieku, kas pievienojis produktu

Tabula “**Pārvaldnieks**” sastāv no 5 laukiem un satur informāciju par sistēmas pārvaldniekiem. Tā ir saistīta ar tabulām “Produkti” un “Grāmata” ar saiti viens-pret-daudziem, nodrošinot administratīvā satura pārvaldību un kontroli.

10. tabula

Tabulas “**Pārvaldnieks**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1	ID	bigint unsigned	20	Primārā atslēga, pārvaldnieka ID
2	Vārds	varchar	255	Pārvaldnieka vārds
3	E-pasts	varchar	255	Pārvaldnieka e-pasts
4	Parole	varchar	255	Šifrēta parole

Starptabula “**Produkti_Pasūtījumi**”, kas realizē daudzi-pret-daudziem relāciju starp tabulām “Produkti” un “Pasūtījums”. Tā sastāv no divām ārējām atslēgām un nodrošina iespēju vienā pasūtījumā iekļaut vairākus produktus, kā arī vienam produktam parādīties vairākos pasūtījumos.

11. tabula

Tabulas “**Produkti_Pasūtījumi**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1	Pasūtījuma ID	bigint unsigned	20	Ārējā atslēga uz orders.id
2	Produkta ID	bigint unsigned	20	Ārējā atslēga uz products.id
3	Skaitis	int	11	Pasūtīto vienību skaits
4	Cena	decimal	8,2	Vienības cena
5	Piegādes adrese	text	-	Adrese, uz kuru sūtīt preci

Tabula “**Pasūtījums**” sastāv no 8 laukiem un satur informāciju par lietotāju veiktajiem pirkumiem, tostarp pasūtījuma statusu un kopējo summu. Tā ir savienota ar tabulu “Lietotājs” ar saiti daudzi-pret-vienu, ar tabulu “Satur” ar saiti viens-pret-daudziem, kā arī ar tabulu “Produkti_Pasūtījumi”, kas satur papildu informāciju par pasūtījumā iekļautajām precēm.

12. tabula

Tabulas “**Pasūtījums**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1	ID	bigint unsigned	20	Primārā atslēga, pasūtījuma ID
2	Lietotāja ID	bigint unsigned	20	Ārējā atslēga uz lietotāju
3	Statuss	varchar	255	Pasūtījuma statuss (pending, completed u.c.)
4	Kopējā summa	decimal	10,2	Pasūtījuma kopējā summa
5	Piegādes adrese	text	-	Piegādes adrese
6	Maksājuma metode	varchar	255	Izmantotā maksājuma metode
7	Maksājuma ID	varchar	255	Stripe maksājuma ID
8	Pasūtīts	timestamp	-	Pasūtījuma veikšanas datums

Tabula “**Izsole**” sastāv no 8 laukiem un satur informāciju par izsolēm, tostarp nosaukumu, aprakstu, cenu un statusu. Tā ir savienota ar tabulu “Lietotājs” ar saiti daudzi-pret-vienu, jo katra izsole pieder vienam lietotājam, un ar tabulu “Liktenis” ar saiti viens-pret-daudziem, jo vienai izsolei var būt vairākas likmes.

13. tabula

Tabulas “**Izsole**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1	ID	bigint unsigned	20	Primārā atslēga, izsoles ID
2	Nosaukums	varchar	255	Izsoles nosaukums

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
3	Apraksts	text	-	Izsoles apraksts
4	Sākuma likme	decimal	8,2	Sākuma likmes summa
5	Attēls	varchar	255	Izsoles attēls
6	Sākuma datums	date	-	Izsoles sākuma datums
7	Beigu datums	date	-	Izsoles beigu datums
8	Lietotāja ID	bigint unsigned	20	Ārējā atslēga uz lietotāju (izsoles izveidotājs)

Tabula **“Liktenis”** sastāv no 4 laukiem un satur informāciju par lietotāju veiktajām likmēm izsolēs. Tā ir savienota ar tabulām “Izsole” un “Lietotājs” ar saiti daudzi-pret-vienu, jo katra likme pieder vienam lietotājam un attiecas uz vienu izsoli.

14. tabula

Tabulas **“Liktenis”** struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1	ID	bigint unsigned	20	Primārā atslēga, likmes ID
2	Lietotāja ID	bigint unsigned	20	Ārējā atslēga uz lietotāju
3	Izsoles ID	bigint unsigned	20	Ārējā atslēga uz izsoli
4	Likmju summa	decimal	10,2	Lietotāja likmes summa

Tabula “**Grāmata**” sastāv no 6 laukiem un satur platformā pieejamo papildu informatīvo saturu. Tā ir savienota ar tabulu “Pārvaldnieks” ar saiti daudzi-pret-vienu, jo viens pārvaldnieks var pievienot vairākas grāmatas.

15. tabula

Tabulas “**Grāmata**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1	ID	bigint unsigned	20	Primārā atslēga, grāmatas ID
2	Pārvaldnieka ID	bigint unsigned	20	Ārējā atslēga uz pārvaldnieku
3	Nosaukums	varchar	255	Grāmatas nosaukums
4	Apraksts	text	-	Grāmatas apraksts
5	Faila ceļš	varchar	255	Faila lokācija serverī
6	Attēls	varchar	255	Grāmatas attēls

Tabula “**Ģimenes**” sastāv no 10 laukiem un satur informāciju par sistēmā izveidotajām ģimenēm. Tā glabā ģimenes nosaukumu, ielūguma kodu, maksājumu informāciju (Stripe klienta un maksājuma metodes identifikatorus), kā arī datus par kartes tipu, pēdējiem cipariem un maksimālo dalībnieku skaitu. Tabula ir savienota ar tabulu “Lietotājs” ar saiti viens-pret-daudziem, jo viena ģimene var saturēt vairākus lietotājus, bet katrs lietotājs var piederēt tikai vienai ģimenei. Papildus tam tā ir saistīta ar tabulām “Family_payment_methods” un “Family_transactions” ar saiti viens-pret-daudziem, jo vienai ģimenei var būt vairākas maksājumu metodes un vairākas finanšu transakcijas.

16. tabula

Tabulas “**Ģimenes**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1	ID	bigint unsigned	20	Primārā atslēga, ģimenes ID
2	Nosaukums	varchar	255	Ģimenes nosaukums
3	Ielūguma kods	varchar	50	Kods, lai pievienotos ģimenei
4	Vecāka ID	bigint unsigned	20	Ārējā atslēga uz vecāku (users.id)

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
5	Stripe klienta ID	varchar	255	Stripe maksājumu klienta ID
6	Stripe maksājuma metode	varchar	255	Stripe maksājumu metodes ID
7	Kartes pēdējie 4 cipari	varchar	4	Kartes pēdējie cipari
8	Kartes marka	varchar	50	Kartes marka
9	Maksimālais locekļu skaits	int	11	Maksimālais ģimenes locekļu skaits
10	Aktīva	tinyint	1	Vai ģimene ir aktīva (1 = jā)

Tabula “**Ģimenes maksājumu metodes**” sastāv no 9 laukiem un satur informāciju par ģimenes maksājumu metodēm. Tajā tiek glabāti Stripe maksājuma metodes identifikatori, klienta ID, kartes pēdējie četri cipari, kartes tips un informācija par to, vai konkrētā metode ir noklusējuma metode. Tabula ir savienota ar tabulu “Families” ar saiti daudzi-pret-vienu, jo katra maksājuma metode pieder vienai ģimenei, bet vienai ģimenei var būt vairākas maksājuma metodes. Tā ir saistīta arī ar tabulu “Lietotājs” ar saiti daudzi-pret-vienu, jo katru maksājuma metodi pievieno konkrēts lietotājs.

17. tabula

Tabulas “**Ģimenes maksājumu metodes**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1	ID	bigint unsigned	20	Primārā atslēga, maksājuma metodes ID
2	Ģimenes ID	bigint unsigned	20	Ārējā atslēga uz ģimeni
3	Stripe maksājuma metode	varchar	255	Stripe maksājuma metode
4	Stripe klienta ID	varchar	255	Stripe klienta ID
5	Kartes pēdējie 4 cipari	varchar	4	Kartes pēdējie cipari
6	Kartes marka	varchar	50	Kartes marka

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
7	Noklusētā	tinyint	1	Vai šī maksājuma metode ir noklusētā (1 = jā)
8	Pievienoja lietotājs ID	bigint unsigned	20	Lietotāja ID, kas pievienoja
9	Pievienots	timestamp	-	Pievienošanas datums

Tabula “**Ģimenes maksājumi**” sastāv no 11 laukiem un satur informāciju par ģimenes veiktajām finanšu transakcijām. Tajā tiek uzglabāta maksājuma summa, valūta, apraksts, maksājuma statuss un Stripe maksājuma identifikators, kā arī papildu dati JSON formātā. Tabula ir savienota ar tabulu “Families” ar saiti daudzi-pret-vienu, jo katra transakcija attiecas uz vienu ģimeni, bet viena ģimene var veikt vairākas transakcijas. Tā ir saistīta arī ar tabulu “Lietotājs” ar saiti daudzi-pret-vienu un ar tabulu “Pasūtījums” ar saiti viens-pret-vienu, jo katru transakciju veic konkrēts lietotājs un katra transakcija pieder viņam pasūtījumam.

18. tabula

Tabulas “**Ģimenes maksājumi**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1	ID	bigint unsigned	20	Primārā atslēga, transakcijas ID
2	Ģimenes ID	bigint unsigned	20	Ārējā atslēga uz ģimeni
3	Lietotāja ID	bigint unsigned	20	Ārējā atslēga uz lietotāju, kas veica transakciju
4				
5	Stripe maksājuma ID	varchar	255	Stripe maksājuma ID
6	Summa	decimal	10,2	Transakcijas summa

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
7	Valūta	varchar	3	Valūtas kods (piemēram, USD)
8	Apraksts	text	-	Transakcijas apraksts
9	Statuss	enum	-	Transakcijas statuss (succeeded, failed, pending)
10	Lietotā maksājuma metode	varchar	255	Maksājuma metode, kas izmantota
11	Metadata	longtext	-	Papildu dati JSON formātā

6. LIETOTĀJA CEĻVEDIS

6.1. Sistēmas prasības aparatūrai un programmatūrai

Lai nodrošinātu izstrādātās e-komercijas platformas korektu darbību, lietotāja ierīcei jāatbilst noteiktām minimālajām prasībām.

Aparatūras prasības:

- procesors: vismaz divkodolu procesors (Intel i3 vai ekvivalents);
- operatīvā atmiņa (RAM): vismaz 4 GB;
- brīvā vieta diskā: vismaz 400 MB;
- stabils interneta pieslēgums.

Programmatūras prasības:

- operētājsistēma: Windows 10 vai jaunāka, Linux vai macOS;
- tīmekļa pārlūkprogramma: Google Chrome, Mozilla Firefox, Microsoft Edge vai cita;
- iespējots JavaScript un sīkdatņu (cookies) atbalsts.

6.2. Sistēmas instalācija un palaišana

Lai instalētu un palaistu sistēmu lietotājam vispirms ir jāparūpējas par sekojoša programnodrošinājuma instalēšanu uz izmantojamās ierīces:

- Visual Studio Code vai PhpStorm;
- Laragon (vai cits lokālais izstrādes serveris);
- Node.js;
- Composer;
- Git;
- Xampp;
- Vue.js.

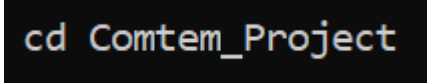
1. Projekta instalācija no GitHub:

1.1. Atvēriet termināli un izpildiet komandu (skat. 13. att.):

```
git clone https://github.com/daniilsonufrijuks/Comtem_Project.git
```

13. att. Git Clone

1.2. Ieejiet projekta mapē (skat. 14. att.):



```
cd Comtem_Project
```

14. att. Projekta mape

1.3. Backend (Laravel) atkarību uzstādīšana:

- composer install
- composer require laravel/cashier
- composer require inertiajs/inertia-laravel
- composer require tightenco/ziggy

1.4. Frontend (Vue.js) atkarību uzstādīšana:

- npm install
- npm install vue @vitejs/plugin-vue
- npm install @fortawesome/fontawesome-free
- npm install font-awesome
- npm install vue-router
- npm install vuex
- npm install @stripe/stripe-js
- npm install autoprefixer
- npm install ziggy-js

1.5. Datubāzes iestatījumus .env failā:

- DB_CONNECTION=mysql
- DB_HOST=127.0.0.1
- DB_PORT=3306
- DB_DATABASE=projekts
- DB_USERNAME=root
- DB_PASSWORD=

1.6. Aplikācijas atslēgas ģenerēšana:

- php artisan key:generate

1.7. Datubāzes migrācijas:

- php artisan migrate

1.8. Apache konfigurācija (XAMPP):

Projekts tiek ievietots direktorijā:

- C:\xampp\htdocs\projekts

1.9. Frontend būvēšana

Production būvei:

- npm run build

2. Stripe integrācija:

- STRIPE_KEY=your_key
- STRIPE_SECRET=your_secret

2.1. Sistēmas servera palaišana:

- php artisan serve

Papildus piezīmes:

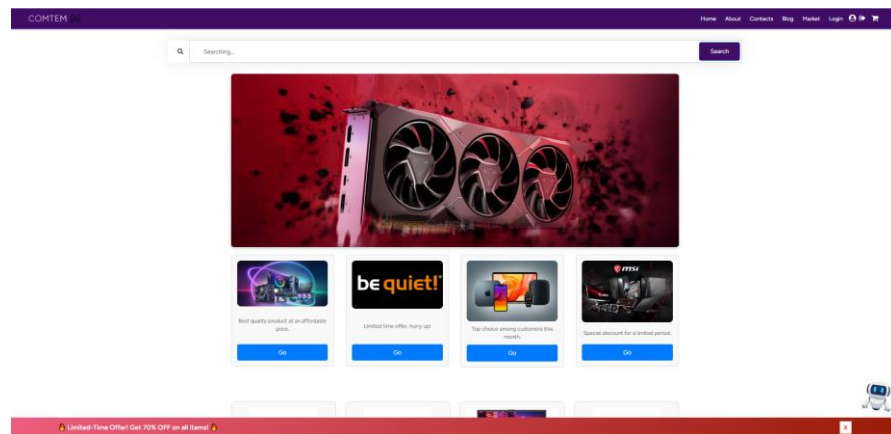
- Tiek nodrošināts, ka MySQL serviss darbojas XAMPP vidē
- Apache konfigurācijā kā root direktorija tiek izmantota public/
- Assetu bundlošanai tiek izmantots Vite
- Tiek palaists Apache serviss XAMPP Control Panel

6.3. Programmas apraksts

Izstrādātā sistēma ir e-komercijas platforma, kas nodrošina datortehnikas iegādi, izsoļu funkcionalitāti un lietotāju sadarbību, izmantojot ģimenes kontus.

Sākulapa.

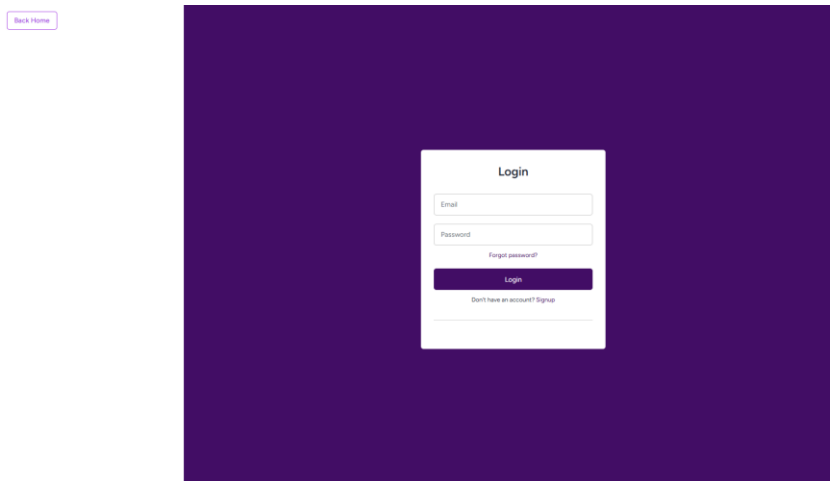
Sākulapā lietotājs var apskatīt pieejamos produktus, jaunākos piedāvājumus, izsoles un bloga rakstus. Ir pieejama arī meklēšanas funkcija.



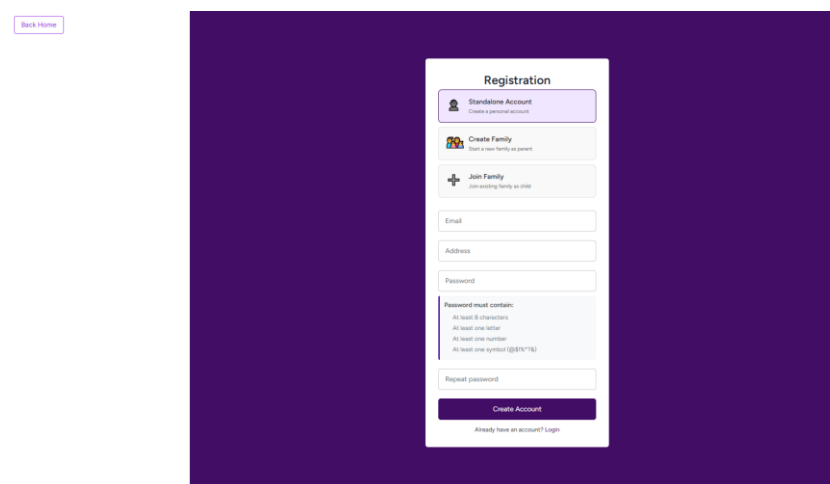
15. att. Sākumlapa

Reģistrācija un pieslēgšanās.

Lietotājs var izveidot kontu, ievadot savu vārdu, e-pastu un paroli. Pēc reģistrācijas lietotājs var pieslēgties sistēmai un izmantot tās funkcionalitāti.



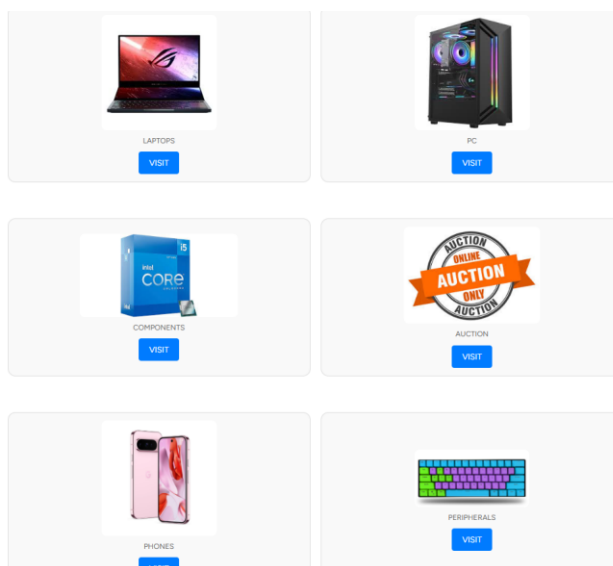
16. att. Pieslēgšanās



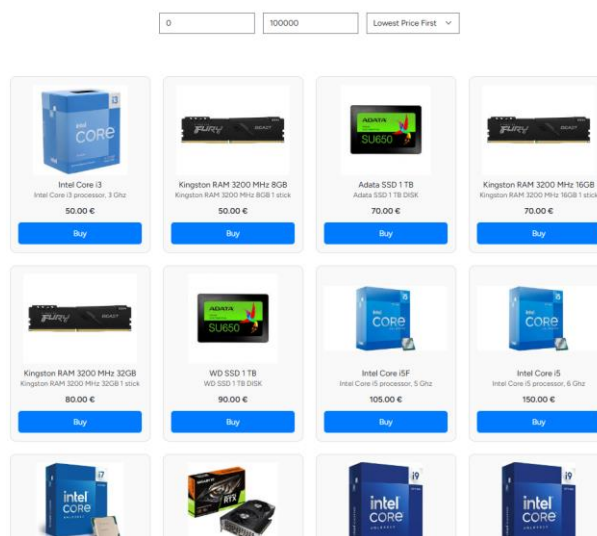
17. att. Reģistrācija

Produktu pārlūkošana.

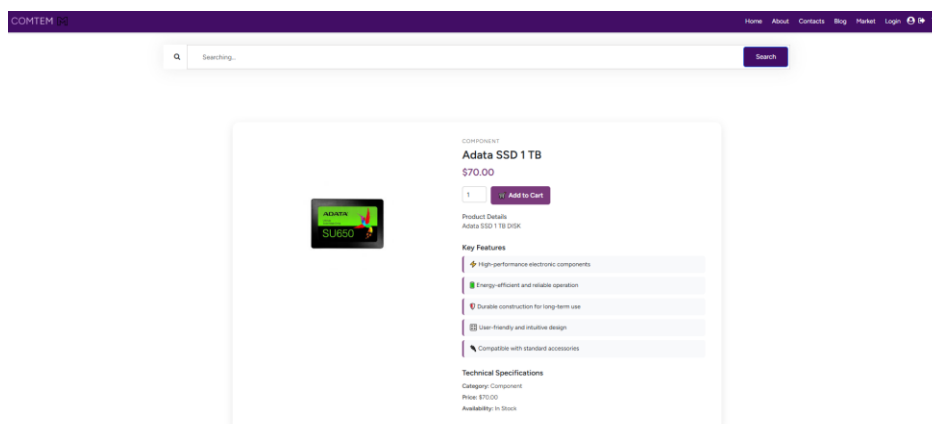
Lietotājs var apskatīt produktu sarakstu, filtrēt tos pēc kategorijām, kā arī apskatīt detalizētu informāciju par katru produktu. Produktus iespējams pievienot grozam.



18. att. Produktu saraksts



19. att. Produktu saraksts

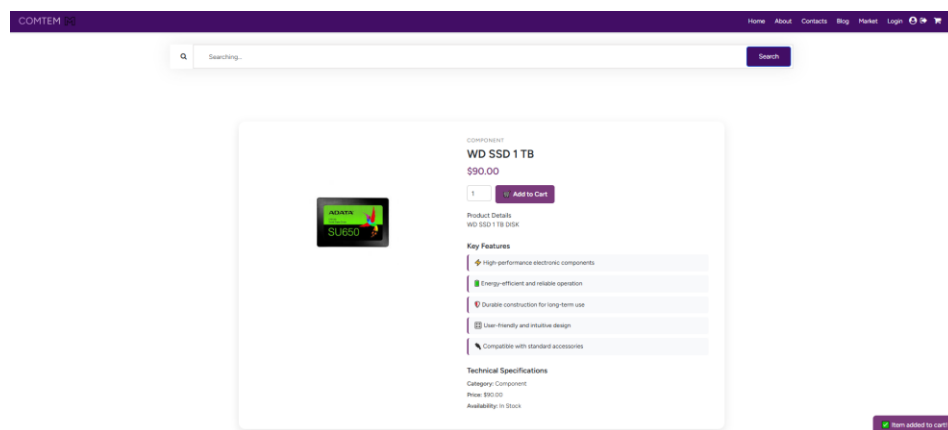


20. att. Produktu apraksts

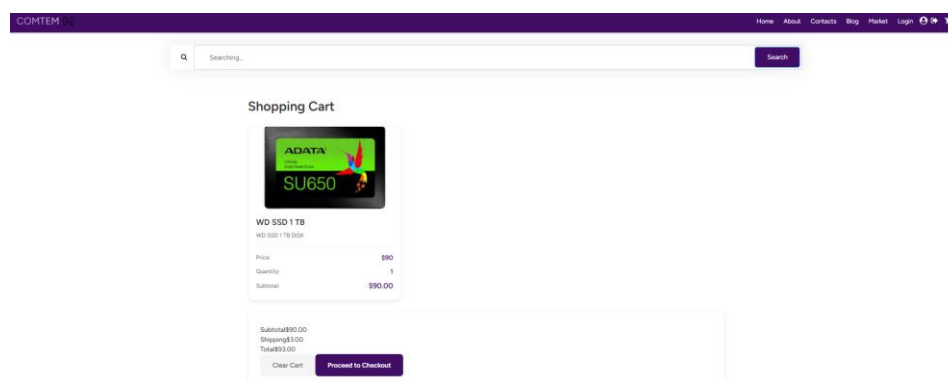
Pasūtījuma veikšana.

Pasūtījuma veikšana notiek vairākos soļos:

- produktu pievienošana grozam;
- groza apskate;
- piegādes informācijas ievade;
- maksājuma metodes izvēle;
- pasūtījuma apstiprināšana.



21. att. Produktu apraksts



22. att. Grozs

← TEST MODE

WD SSD 1TB
US\$90.00

OR

Contact information

Email
botolan@gmail.com

Payment method

☒ Card

Card information

3714 496353 98431

04 / 55 455

Cardholder name

Card

Country or region

Latvia

☐ Save my information for faster checkout
My security on this site and everywhere link is accepted.

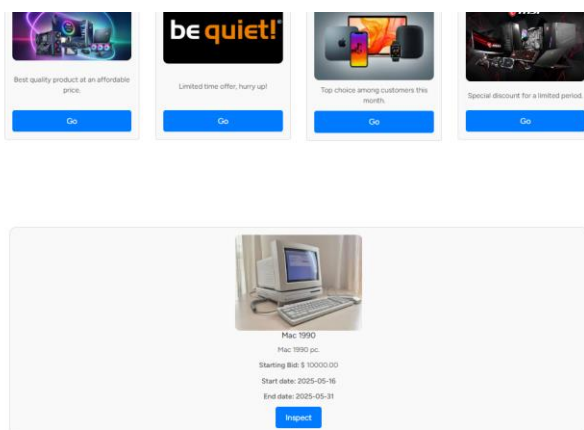
Pay

Powered by 8088 | Terms Privacy

23. att. Maksājums

Izsoļu sistēma.

Sistēma nodrošina izsoļu funkcionalitāti, kur lietotāji var apskatīt aktīvās izsoles, piedalīties tajās un veikt solījumus.



24. att. Izsoļu funkcionalitāte

Ģimenes kots.

Lietotāji var izveidot ģimenes kontu, pievienot citus lietotājus un izmantot kopīgu maksājuma metodi. Tas ļauj pārvaldīt kopējos izdevumus un veikt kopīgus pirkumus.

[Back Home](#)

Registration

Standalone Account
Create a personal account

Create Family
Start your family account

Join Family
Join existing family or club

Check a name for your family

Password must contain:

- At least 8 characters
- At least one letter
- At least one number
- At least one symbol (@!#\$%^&*)

Repeat password

Create Family Account

Already have an account? [Login](#)

25. att. Reģistrācija

COMTEM

Home About Contacts Blog Market Login

Family5chid1
Click person to edit

Information

Email: family5chid1@gmail.com

Name: family5chid1

Address: Valdemāra ielā, Daugavpils 16

Click on a profile of your account, there is no going back. Please be certain.

[Delete Account](#)

Family Payment Management

Family5
Invitation Code: 5PQ2EASIS
Your Role: [Admin](#)

Family Payment Methods

amex ****0000 [Default](#)

Recent Transactions

\$1804.10
Purchase from cart
By: family5chid1

[View All Transactions](#)

26. att. Konts

127.0.0.1:8000 says

You have a family card available. Would you like to use it? (Cancel to use regular checkout)

COMTEM

Home About Contacts Blog Market Login

Shopping Cart

MSI Gaming Station
MSI Gaming Station MSI Gaming Station Intel Core
Processor: 5 GHz Intel® Core™ i9-10900K 8 Cores 16
Fury 32 GB 20000 MHz SSD 1 TB HDD 2 TB Power
Supply Be quiet! P08-W

Price	\$970
Quantity	1
Subtotal	\$970.00

Subtotal\$970.00

Shipping\$3.00

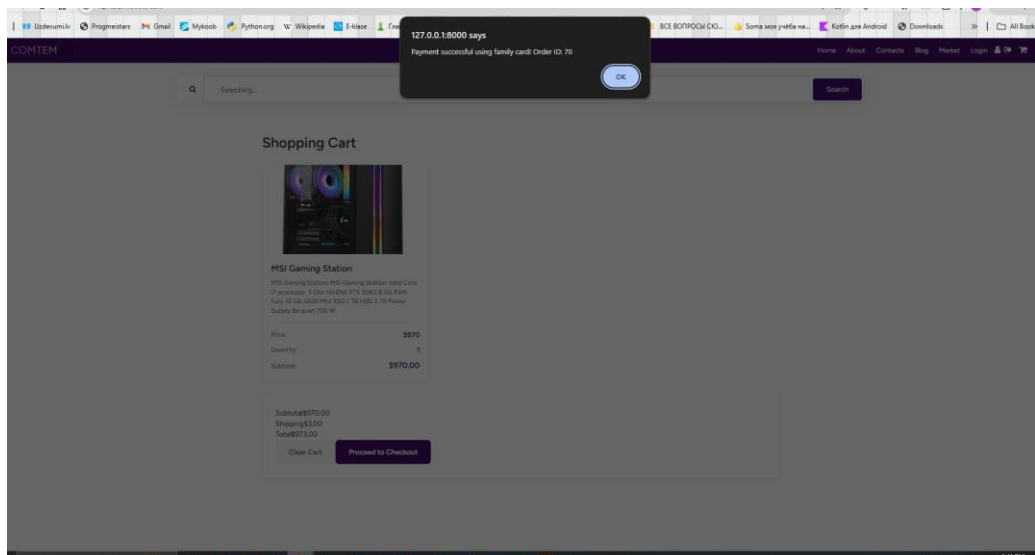
Total\$973.00

[Clear Cart](#)

[Proceed to Checkout](#)

27. att. Maksājums

59



28. att. Ģimenes maksājums

Komentāri.

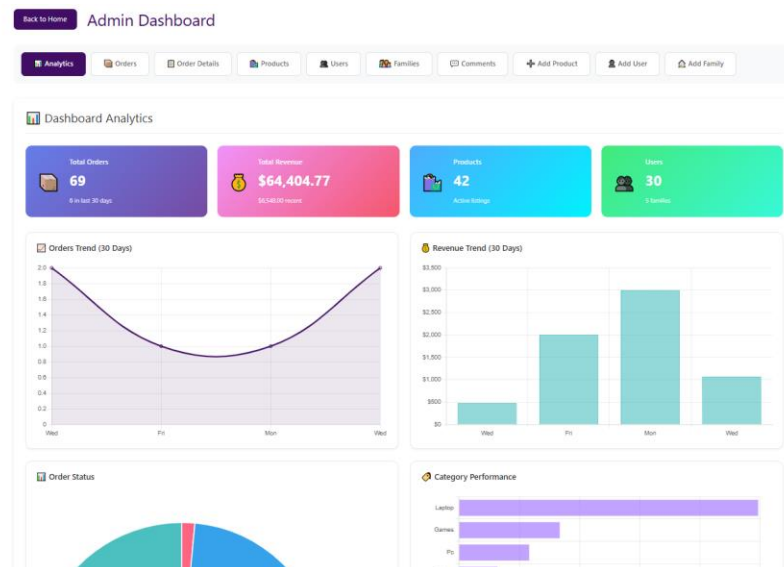
Lietotāji var pievienot komentārus un izteikt savu viedokli par produktiem vai citu saturu sistēmā.

 A screenshot of a web application's comments section. The section is titled 'Comments' and features a text input field with the placeholder 'Write a comment...'. Below the input field is a blue 'Submit' button. The comments are displayed in a list format, each with a user name and a comment text. The first comment is from 'kim' with the text 'Good shop, good laptops!'. The second comment is from 'Guest' with the text 'Nice store!'. The third comment is from 'man' with the text 'Very good shop!'. The fourth comment is from 'Guest' with the text 'Thank you!'. A vertical scrollbar is visible on the right side of the comments list.

29. att. Komentāri

Pārvaldnieka funkcijas.

Pārvaldnieks var pārvaldīt sistēmas saturu, tostarp pievienot un rediģēt produktus, pārvaldīt lietotājus, kā arī uzraudzīt pasūtījumus un izsoles.



30. att. Pārvaldnieka sistēma

Darba pabeigšana

Lai pārtrauktu darbu ar sistēmu, lietotājam ir jāizvēlas funkcija “Izrakstīties”. Pēc izrakstīšanās piekļuve sistēmai tiek pārtraukta līdz nākamajai autorizācijai.



31. att. Darba pabeigšana

7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA

19. tabula

“Reģistrēšanās” testa tabula

Lietotāja loma:	viesis vai reģistrēts lietotājs	
Prasības identifikators	PS 2.2.1	
Prasības apraksts		
Jānodrošina, ka lietotājs var reģistrēties sistēmai.		
<i>Ievaddati/situācijas apraksts</i>	<i>Sagaidāmais rezultāts</i>	<i>Statuss</i>
1. Lietotājs ievada derīgu e-pastu un paroli	Konts tiek veiksmīgi izveidots	pareizi
2. Lietotājs ievada jau eksistējošu e-pastu	Parādās kļūda "E-pasts jau eksistē"	pareizi
3. Parole neatbilst prasībām	Parādās kļūda par paroles drošību	pareizi
4. Nepareizs e-pasta formāts	Parādās validācijas kļūda	pareizi

20. tabula

“Produktu meklēšana un filtrēšana” testa tabula

Lietotāja loma:	viesis vai reģistrēts lietotājs	
Prasības identifikators	PS 2.2.2	
Prasības apraksts		
Jānodrošina, ka lietotājs var meklēt un filtrēt produktus		
Ievaddati/situācijas apraksts	Sagaidāmais rezultāts	Statuss
1. Lietotājs meklē "RTX 4060"	Parādās atbilstoši produkti	pareizi
2. Filtrs: cena līdz 500€	Parādās tikai lētāki produkti	pareizi
3. Meklēšana ar neeksistējošu nosaukumu	Parādās "Nav atrastu produktu"	pareizi
4. Kārtošana pēc cenas	Produkti sakārtoti augošā secībā	pareizi

21. tabula

“Pasūtījuma veikšana” testa tabula

Lietotāja loma:	reģistrēts lietotājs	
Prasības identifikators	PS 2.2.3	
Prasības apraksts		
Jānodrošina, ka lietotājs var veikt pasūtījumu.		
<i>Ievaddati/situācijas apraksts</i>	<i>Sagaidāmais rezultāts</i>	<i>Statuss</i>
1. Produkts pievienots grozam	Produkts parādās grozā	pareizi
2. Lietotājs noformē pasūtījumu	Tiek izveidots pasūtījums	pareizi
3. Maksājums veiksmīgs	Statuss "Apmaksāts"	pareizi
4. Nepietiekams daudzums noliktavā	Parādās kļūda	pareizi

22. tabula

“Solījumu veikšana” testa tabula

Lietotāja loma:	reģistrēts lietotājs	
Prasības identifikators	PS 2.2.4	
Prasības apraksts		
Jānodrošina, ka lietotājs var veikt solījumu.		
Ievaddati/situācijas apraksts	Sagaidāmais rezultāts	Statuss
1. Lietotājs veic lielāku solījumu	Solījums tiek pieņemts	pareizi
2. Solījums mazāks par minimālo	Parādās kļūda	pareizi
3. Izsole beigusies	Nevar veikt solījumu	pareizi

“MI atbildes ģenerēšana” testa tabula

Lietotāja loma:	visi lietotāji	
Prasības identifikators	PS 2.2.6	
Prasības apraksts		
Jānodrošina, ka mākslīgais intelekts var ģenerēt tekstus.		
Ievaddati/situācijas apraksts	Sagaidāmais rezultāts	Statuss
1. Lietotājs uzdod jautājumu	Parādās atbilde	pareizi
2. Jautājums nesaprotams	Parādās kļūdas ziņojums	pareizi
3. Jautājums par produktu	Tiek piedāvāti ieteikumi	pareizi

“Pasūtījuma veikšana” testa tabula

Lietotāja loma:	lietotājs ar administratora tiesībām	
Prasības identifikators	PS 2.2.7	
Prasības apraksts		
Jānodrošina izsoļu pārvaldību administratora līmenī.		
<i>Ievaddati/situācijas apraksts</i>	<i>Sagaidāmais rezultāts</i>	<i>Statuss</i>
1. Administrators pievieno jaunu produktu	produkts tiek pievienots katalogam	pareizi
2. Administrators rediģē produktu	izmaiņas tiek saglabātas	pareizi
3. Administrators dzēš produktu	produkts tiek dzēsts	pareizi
4. Parasts lietotājs mēģina piekļūt admin panelim	piekļuve tiek liegta	pareizi

7.1. Testēšanas mērķis

Programmatūras lietojamības testēšanas mērķis ir novērtēt izstrādātās e-komercijas platformas lietošanas ērtumu, efektivitāti un saprotamību no gala lietotāja skatpunkta. Testēšanas procesā tiek identificētas lietotāja saskarnes nepilnības, navigācijas problēmas un iespējamās kļūdas, kas var ietekmēt sistēmas lietošanas pieredzi.

Papildus tiek izvērtēts, cik ātri un intuitīvi lietotājs spēj apgūt sistēmas funkcionalitāti, kā arī cik efektīvi iespējams veikt galvenos uzdevumus, piemēram, produktu iegādi, piedalīšanos izsolēs un ģimenes konta izmantošanu.

7.2. Testēšanas metodika

Testēšana tika veikta, izmantojot scenārijos balstītu pieeju. Lietotājiem tika piedāvāti konkrēti uzdevumi, kas atbilst reālām sistēmas izmantošanas situācijām.

Testēšanā piedalījās vairāki lietotāji ar atšķirīgu pieredzi darbā ar informācijas sistēmām:

- lietotāji bez tehniskām zināšanām;
- lietotāji ar vidēju pieredzi;
- lietotāji ar labām digitālajām prasmēm.

Testēšana tika veikta individuāli, lietotājiem patstāvīgi izpildot uzdevumus bez palīdzības.

Testēšanas laikā tika novērots:

- uzdevuma izpildes laiks;
- kļūdu skaits;
- lietotāja apjukuma brīži;
- nepieciešamība pēc papildu paskaidrojumiem.

Papildus tika apkopota lietotāju subjektīvā pieredze, izmantojot īsu aptauju pēc testēšanas.

7.3. Testēšanas scenāriji

Testēšanas laikā lietotājiem tika uzdoti šādi uzdevumi:

1. Reģistrēties sistēmā, ievadot nepieciešamos datus;
2. Pieslēgties sistēmai ar izveidoto kontu;
3. Atrast konkrētu produktu, izmantojot meklēšanu vai kategorijas;
4. Pievienot produktu grozam;
5. Veikt pasūtījumu, aizpildot nepieciešamo informāciju;
6. Atrast aktīvu izsoli un veikt solījumu;
7. Izveidot jaunu ģimenes kontu;
8. Pievienoties esošam ģimenes kontam;
9. Izrakstīties no sistēmas.

Katrs scenārijs tika izvērtēts pēc izpildes veiksmīguma un lietotāja pieredzes.

7.4. Testēšanas rezultāti

Testēšanas rezultāti parādīja, ka sistēma kopumā ir viegli lietojama un intuitīva. Lielākā daļa lietotāju spēja veiksmīgi izpildīt visus uzdevumus bez būtiskām grūtībām.

Pozitīvie aspekti:

- vienkārša un saprotama reģistrācijas un pieslēgšanās forma;
- ērta produktu pārlūkošana un meklēšana;
- loģisks pasūtījuma veikšanas process;
- vizuāli pārskatāms dizains.

Konstatētās problēmas:

- dažiem lietotājiem bija grūtības ātri atrast izsoļu sadaļu;
- dažās vietās trūka vizuālu paziņojumu par veiksmīgu darbību;

7.5. Testēšanas rezultātu analīze

Analizējot testēšanas rezultātus, var secināt, ka lielākā daļa problēmu ir saistītas nevis ar sistēmas funkcionalitāti, bet gan ar lietotāja saskarnes uzlabojumiem.

Galvenās problēmu kategorijas:

- nepietiekama lietotāja informēšana par darbību rezultātiem;
- nepietiekami intuitīva jaunu funkciju (piemēram, ģimenes konta) izmantošana.

Šīs problēmas neietekmē sistēmas darbību, bet var ietekmēt lietotāja pieredzi.

7.6. Ieteiktie uzlabojumi

Pamatojoties uz testēšanas rezultātiem, tika izstrādāti šādi uzlabojumi:

- pievienot paskaidrojošus tekstus vai instrukcijas sarežģītākām funkcijām;
- uzlabot pogu un darbību elementu redzamību;
- vienkāršot ģimenes konta izveides un pievienošanās procesu.

7.7. Secinājumi par lietojamību

Veiktā lietojamības testēšana parāda, ka izstrādātā e-komercijas sistēma ir funkcionāla, saprotama un piemērota ikdienas lietošanai. Lietotāji spēj efektīvi veikt galvenās darbības bez būtiskas palīdzības.

Identificētās nepilnības ir saistītas galvenokārt ar lietotāja saskarnes uzlabojumiem, un tās ir viegli novēršamas. Pēc šo uzlabojumu ieviešanas sistēma kļūs vēl lietotājam draudzīgāka un intuitīvāka.

NOBEIGUMS

Kvalifikācijas darba ietvaros tika izstrādāta daudzfunkcionāla e-komercijas platforma datortehnikas un tās komponentu tirdzniecībai. Darba izstrādes gaitā tika analizētas mūsdienu e-komercijas sistēmu prasības, izvēlētas atbilstošas tehnoloģijas un izveidots programmatūras risinājums, kas apvieno vairākas svarīgas funkcionalitātes vienotā platformā.

Izstrādātā sistēma nodrošina lietotājiem iespēju pārlūkot un iegādāties datortehniku, piedalīties izsolēs, izmantot bloga sadaļu informatīva satura iegūšanai, kā arī saņemt palīdzību no mākslīgā intelekta asistenta. Platformā ir ieviesta arī ģimenes konta funkcionalitāte, kas ļauj vairākiem lietotājiem izmantot vienotu maksājumu struktūru un pārvaldīt kopīgus pirkumus. Šāds risinājums uzlabo maksājumu pārskatāmību un ļauj efektīvāk kontrolēt ģimenes izdevumus digitālajā vidē.

Darba gaitā tika izstrādāta sistēmas arhitektūra, datu bāzes struktūra un lietotāju lomu modelis, kas nodrošina drošu un strukturētu sistēmas darbību. Tika definētas funkcionālās un nefunkcionālās prasības, izveidoti sistēmas modeļi un aprakstīta platformas darbības loģika. Platformas realizācijai tika izmantotas modernas tīmekļa izstrādes tehnoloģijas, piemēram, Laravel, Vue.js un MySQL, kas nodrošina sistēmas stabilitāti, drošību un paplašināmību.

Izstrādātā e-komercijas platforma demonstrē, ka vienotā sistēmā iespējams efektīvi apvienot produktu katalogu, izsoļu mehānismu, informatīvo saturu un mākslīgā intelekta atbalstu. Šāds risinājums var uzlabot lietotāju pieredzi un samazināt nepieciešamību izmantot vairākas dažādas tīmekļa vietnes produktu iegādei un informācijas iegūšanai.

Nākotnē platformu būtu iespējams papildināt ar jaunām funkcijām, piemēram, lietoto datortehnikas komponentu tirdzniecību, servisa pakalpojumu rezervāciju, uzlabotu produktu salīdzināšanas sistēmu vai papildu maksājumu metodēm. Tāpat iespējams attīstīt mākslīgā intelekta asistenta iespējas, lai tas sniegtu vēl precīzākus produktu ieteikumus un tehniskās konsultācijas.

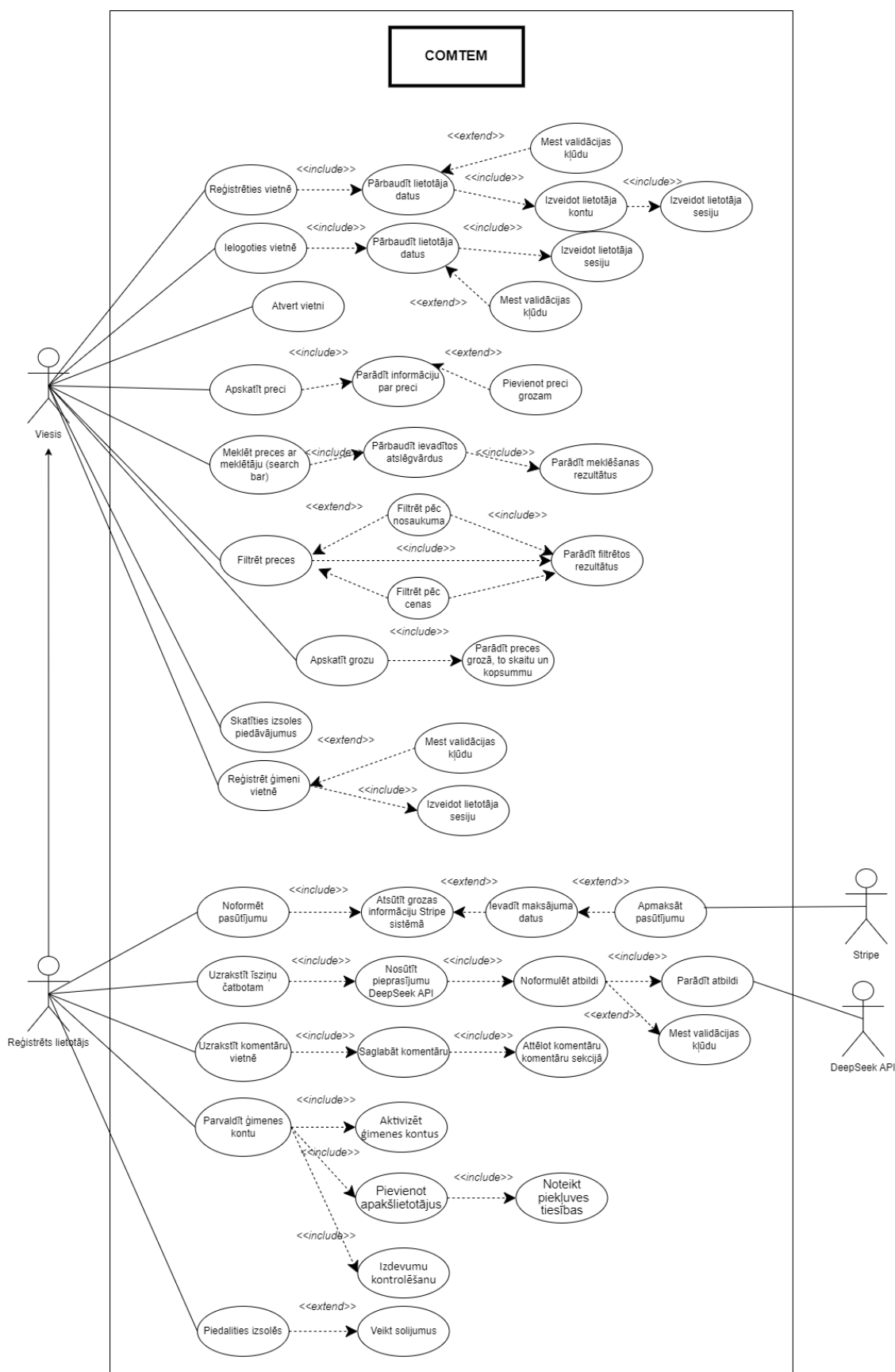
Kopumā kvalifikācijas darba mērķis tika sasniegts – izstrādāta funkcionāla un lietotājam draudzīga e-komercijas platforma, kas demonstrē modernu tīmekļa tehnoloģiju pielietojumu datortehnikas tirdzniecības un informācijas apmaiņas jomā.

INFORMĀCIJAS AVOTI

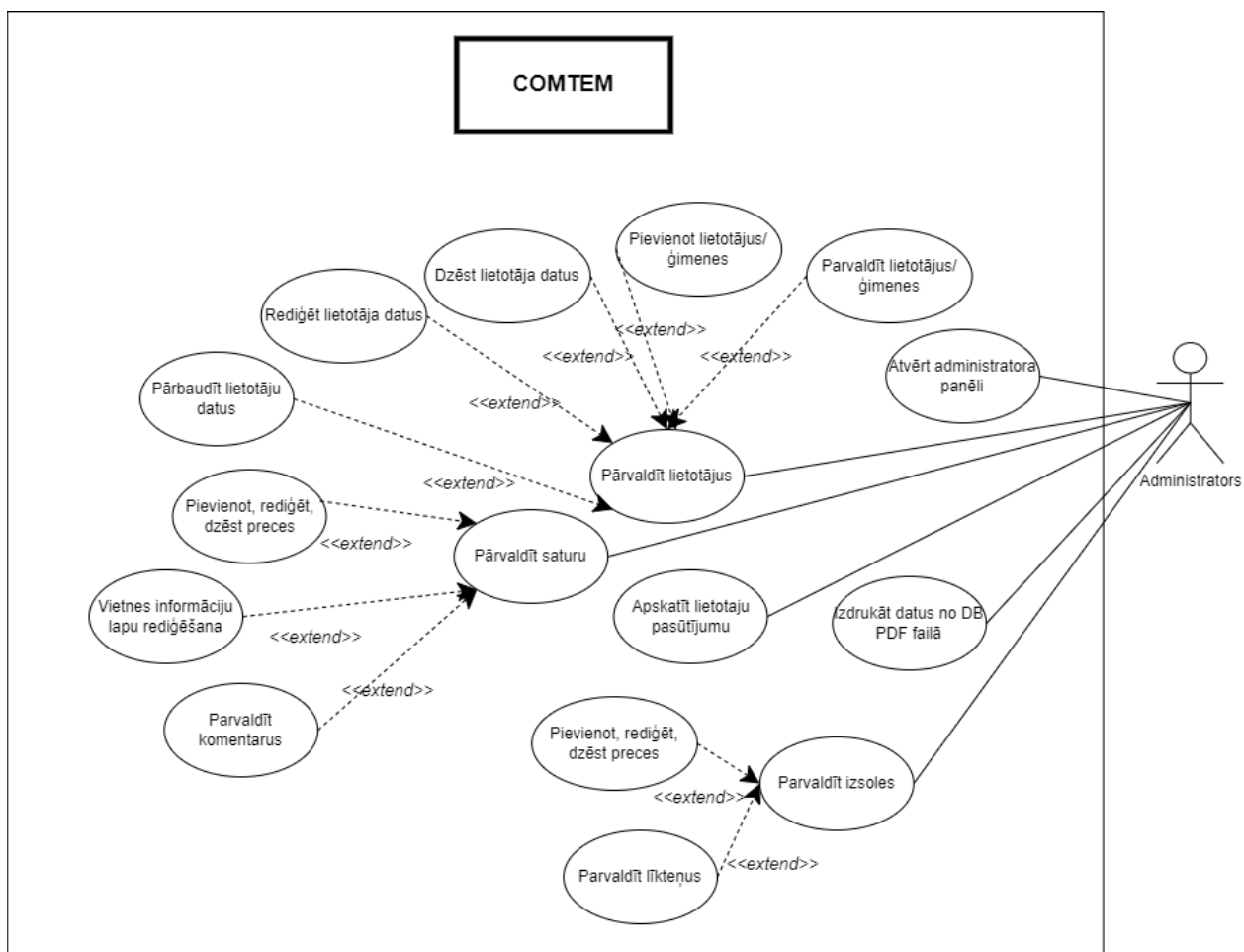
1. Elmasri, R., Navathe, S. B. *Fundamentals of Database Systems*. 7th ed. Pearson Education, 2016.
2. Connolly, T., Begg, C. *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6th ed. Pearson Education, 2015.
3. Date, C. J. *An Introduction to Database Systems*. 8th ed. Pearson Education, 2004.
4. Silberschatz, A., Korth, H. F., Sudarshan, S. *Database System Concepts*. 6th ed. McGraw-Hill, 2011.
5. Chen, P. P.-S. *The Entity-Relationship Model — Toward a Unified View of Data*. ACM Transactions on Database Systems, Vol. 1, No. 1, 1976, pp. 9–36.
6. Sommerville, I. *Software Engineering*. 10th ed. Pearson Education, 2016.
7. Laudon, K. C., Laudon, J. P. *Management Information Systems: Managing the Digital Firm*. 15th ed. Pearson Education, 2018.
8. MySQL Documentation. *MySQL 8.0 Reference Manual*. Pieejams: <https://dev.mysql.com/doc/> (skatīts: 2026. gada janvārī).
9. PostgreSQL Documentation. *PostgreSQL 15 Documentation*. Pieejams: <https://www.postgresql.org/docs/> (skatīts: 2026. gada janvārī).
10. Laravel Documentation. *Laravel Framework Documentation*. Pieejams: <https://laravel.com/docs> (skatīts: 2026. gada janvārī).
11. PHP Documentation. *PHP Manual*. Pieejams: <https://www.php.net/docs.php> (skatīts: 2026. gada janvārī).
12. Mozilla Developer Network (MDN). *JavaScript Guide*. Pieejams: <https://developer.mozilla.org/en-US/docs/Web/JavaScript> (skatīts: 2026. gada janvārī).
13. Vue.js Documentation. *Vue.js Guide*. Pieejams: <https://vuejs.org/guide/> (skatīts: 2026. gada janvārī).
14. Fielding, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine, 2000.
15. Richardson, L., Ruby, S. *RESTful Web Services*. O'Reilly Media, 2007.
16. Newman, S. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2015.
17. Gamma, E., Helm, R., Johnson, R., Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
18. W3Schools. *HTML, CSS, JavaScript Tutorials*. Pieejams: <https://www.w3schools.com/> (skatīts: 2026. gada janvārī).

PIELIKUMI

Lietojumgadījumu diagramma



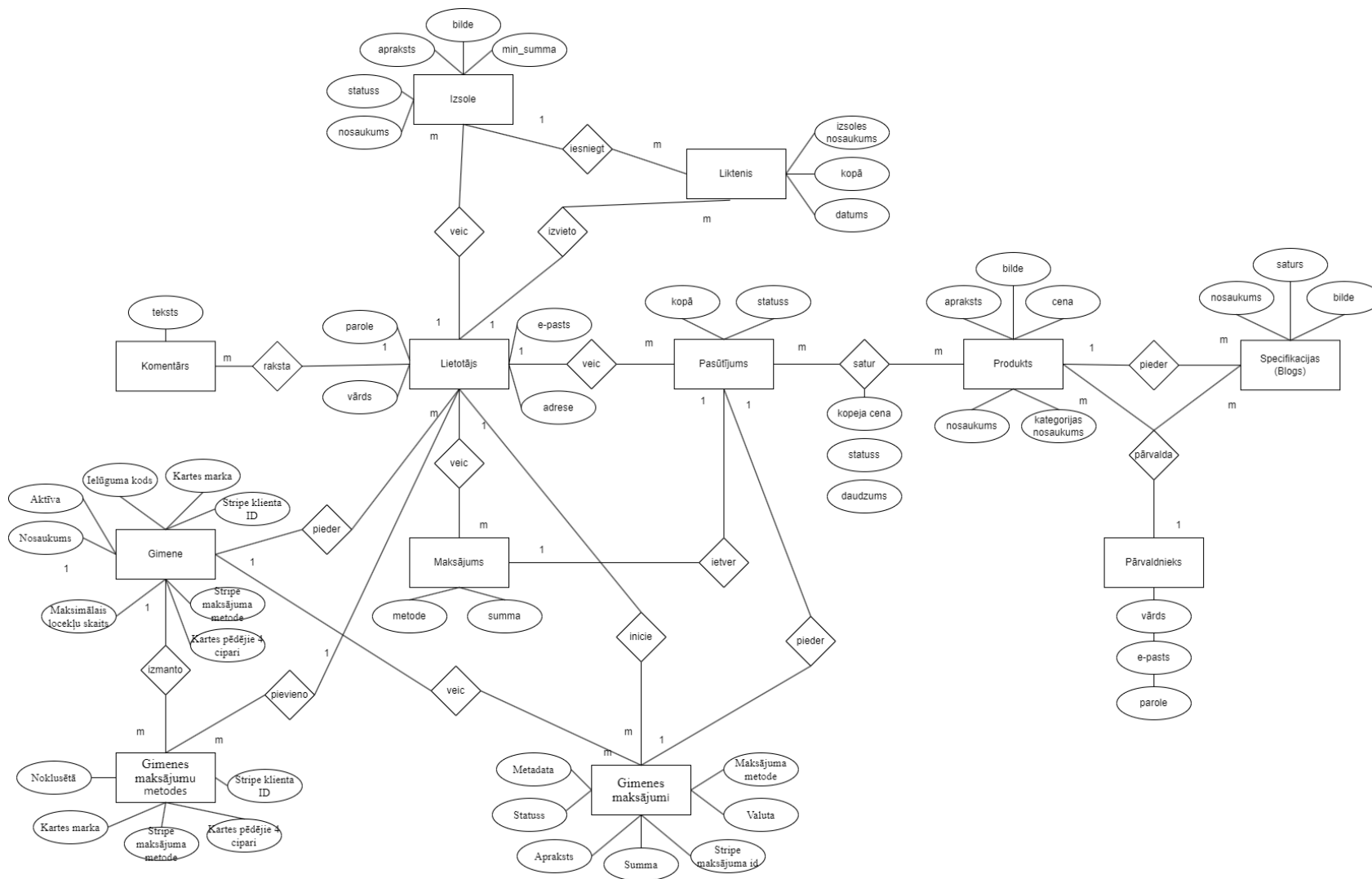
Lietojumgadījumu diagramma



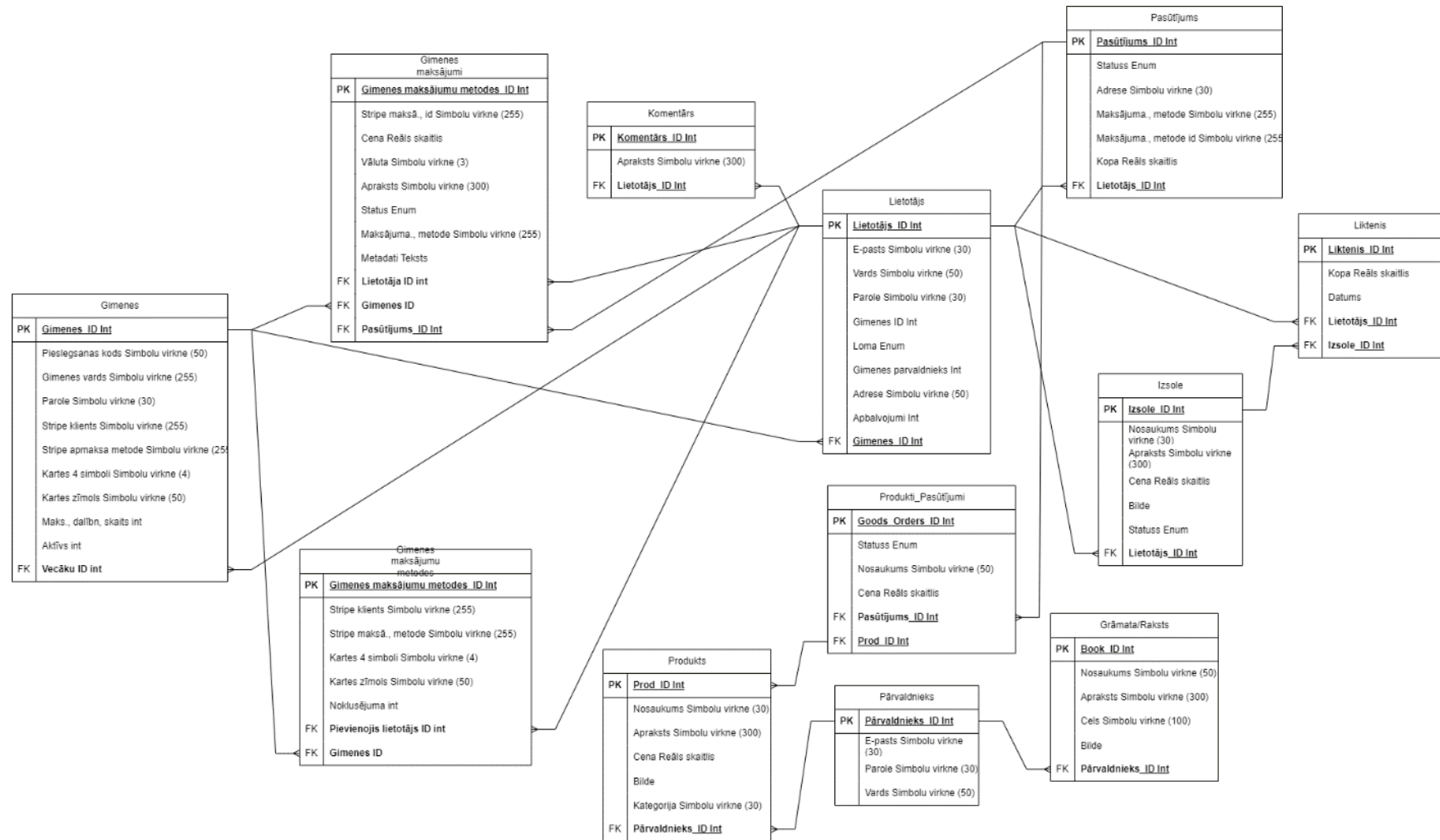
Funkcionālās dekompozīcijas diagramma



ER diagramma



Fiziskā struktūra



Lietotāja kontrolieris

```
class UserController extends Controller
{
  public function userProfile()
  {
    // Check if user is authenticated
    if (!auth()->check()) {
      // Return JSON for API calls, redirect for page requests
      if (request()->expectsJson()) {
        return response()->json([
          'isLoggedIn' => false,
          'user' => null
        ]);
      }
      return redirect()->route('login');
    }

    // Fetch the authenticated user
    $user = auth()->user();
    $user->load(['family']);

    // Return data to the Vue component via Inertia
    return Inertia::render('User', [
      'user' => $user
    ]);
  }

  public function getFamily()
  {
    $user = Auth::user();
    $user->load('family'); // Load the family relationship

    return response()->json([
      'family' => $user->family,
    ]);
  }

  public function getAward()
  {
    $user = Auth::user();
    return response()->json([
      'award' => $user->award,
    ]);
  }

  public function getProfile()
  {
    $user = Auth::user();
    return response()->json([
      'address' => $user->address,
      'name' => $user->name,
      'email' => $user->email,
    ]);
  }
}
```

Izsoles kontrolieris

```

public function indexAuctions(): \Illuminate\Http\JsonResponse
{
    $auctions = Auction::with(['user:id,name,email'])
        ->latest()
        ->get();
    return response()->json($auctions);
}

public function showOrders(Request $request): \Illuminate\Http\JsonResponse
{
    $orders = Orders::with(['user:id,name,email'])
        ->select(['id', 'user_id', 'status', 'total', 'ordered_at', 'created_at'])
        ->latest()
        ->get();

    return response()->json($orders);
}

public function showOrderItems(): \Illuminate\Http\JsonResponse
{
    $orderItems = OrderGoods::select([
        'id', 'order_id', 'name', 'price', 'status', 'created_at'
    ])->latest()->get();

    return response()->json($orderItems);
}

public function showProducts(Request $request): \Illuminate\Http\JsonResponse
{
    $products = Products::select(['id', 'name', 'price', 'description', 'image',
    'category', 'created_at'])
        ->latest()
        ->get();

    return response()->json($products);
}

public function showUsers(Request $request): \Illuminate\Http\JsonResponse
{
    $users = User::withCount('orders')
        ->with(['family:id,family_name'])
        ->select(['id', 'name', 'email', 'role', 'family_id', 'created_at', 'awards'])
        ->latest()
        ->get();

    return response()->json($users);
}

public function showFamilies(Request $request): \Illuminate\Http\JsonResponse
{
    $families = Family::withCount('users')
        ->with(['parent:id,name,email', 'users:id,name,email,role'])
        ->select(['id', 'family_name', 'parent_id', 'invitation_code', 'created_at'])
        ->latest()
        ->get();

    return response()->json($families);
}

// Get bids for a specific auction
public function showBids($auctionId): \Illuminate\Http\JsonResponse
{
    $bids = Bids::with('user:id,name,email')
        ->where('item_id', $auctionId)
        ->orderBy('bid_amount', 'desc')
        ->get();
}

```

```
    return response()->json($bids);
}
```

7. pielikums

Pasūtījumu kontrolieris

```
class OrderController extends Controller
{
    public function store(Request $request): \Illuminate\Http\JsonResponse
    {
        $request->validate([
            'items' => 'required|array',
            'items.*.name' => 'required|string',
            'items.*.price' => 'required|numeric',
            'items.*.description' => 'nullable|string',
            'items.*.image' => 'nullable|string',
            'items.*.category' => 'required|string',
            'items.*.total_price' => 'required|numeric',
            'items.*.shipping_address' => 'nullable|string',
            'items.*.id' => 'required|integer',
            'items.*.quantity' => 'required|integer|min:1',
            'total' => 'required|numeric',
            'award' => 'required|integer',
            'shipping' => 'required|numeric',
            'shipping_address' => 'nullable|string',
            'payment_method' => 'nullable|string',
            'payment_intent_id' => 'nullable|string',
            'status' => 'nullable|string',
        ]);

        $user = $request->user();
        $award = $user->awards ?? 0;

        // SHIPPING LOGIC
        $shippingCost = $award > 100 ? 0 : 3;

        // ITEMS TOTAL
        $itemsTotal = collect($request->items)->sum(function ($item) {
            return $item['price'] * $item['quantity'];
        });

        // FINAL TOTAL
        $finalTotal = $itemsTotal + $shippingCost;

        try {
            // Create order
            $order = Orders::create([
                'user_id' => auth()->id(),
                'total' => $finalTotal,
                'shipping_address' => $request->shipping_address ?? null,
                'payment_method' => $request->payment_method ?? 'stripe',
                'payment_intent_id' => $request->payment_intent_id ?? null,
                'status' => $request->status ?? 'pending',
            ]);

            foreach ($request->items as $item) {
                OrderGoods::create([
                    'order_id' => $order->id,
                    'product_id' => $item['id'],
                    'status' => 'pending',
                    'name' => $item['name'],
                    'price' => $item['price'] * $item['quantity'],
                    'quantity' => $item['quantity'],
                ]);
            }
        }
    }
}
```

```

    }

    return response()->json([
        'success' => true,
        'message' => 'Order placed successfully!',
        'order' => $order,
        'order_id' => $order->id,
    ]);

} catch (\Exception $e) {
    \Log::error('Order creation error: ' . $e->getMessage());
    return response()->json([
        'error' => 'Failed to create order: ' . $e->getMessage(),
    ], 500);
}

}

public function userOrders()
{
    $user = auth()->user();
    if (!$user) {
        return response()->json(['message' => 'Unauthorized'], 401);
    }

    $orders = DB::table('orders')
        ->join('goods_orders', 'orders.id', '=', 'goods_orders.order_id')
        ->where('orders.user_id', $user->id)
        ->select(
            'orders.id as order_id',
            'orders.status as order_status',
            'orders.total as order_total',
            'orders.created_at',
            'goods_orders.id as item_id',
            'goods_orders.product_id',
            'goods_orders.quantity',
            'goods_orders.name as item_name',
            'goods_orders.price as item_price',
        )
        ->orderBy('orders.ordered_at', 'desc')
        ->get();

    return response()->json($orders);
}
}

```

Preču kontrolieris (piemērs)

```
class ProductsController extends Controller
{
  public function getComponentsProducts(Request $request):
  \Illuminate\Http\JsonResponse
  {
    // Base query for "Component" category
    $query = Products::where('category', 'Component');

    // Apply filters based on query parameters
    // Handle price_min only if not null
    // Check if price_min is provided
    // Apply filters only if values are not null or empty
    if ($request->filled('price_min')) {
      $query->where('price', '>=', $request->input('price_min'));
    }

    if ($request->filled('price_max')) {
      $query->where('price', '<=', $request->input('price_max'));
    }

    // Fetch the filtered results
    $products = $query->get(['id', 'name', 'price', 'description', 'image']);

    return response()->json($products);
  }
}
```

Autentifikācijas kontrolieri

```
class AuthenticatedSessionController extends Controller
{
  /**
   * Display the login view.
   */
  public function create(): Response
  {
    return Inertia::render('Auth/Login', [
      'canResetPassword' => Route::has('password.request'),
      'status' => session('status'),
    ]);
  }

  /**
   * Handle an incoming authentication request.
   */
  public function store(LoginRequest $request): RedirectResponse
  {
    // If admin exists, attempt login with admin credentials
    $admin = Admins::where('name', $request->email)->first();

    if ($admin && Auth::guard('admin')->attempt(['name' => $request->email,
    'password' => $request->password])) {
      // Regenerate the session after successful admin login
      $request->session()->regenerate();
      return redirect()->route('admin.dashboard'); // Redirect to the admin
      dashboard
    }

    // Else, proceed with the normal user login (based on email)
    if (Auth::guard('web')->attempt(['email' => $request->email, 'password' =>
    $request->password])) {

```

```

        // Regenerate the session after successful user login
        $request->session()->regenerate();
        return redirect()->intended(route('home', absolute: false)); // Redirect
to home page
    }
    // If login fails, return with an error
    return back()->withErrors([
        'email' => 'These credentials do not match our records.',
    ]);
}

/**
 * Destroy an authenticated session.
 */
public function destroy(Request $request): RedirectResponse
{
    Auth::guard('web')->logout();

    $request->session()->invalidate();

    $request->session()->regenerateToken();

    return redirect('/');
}
}

class RegisteredUserController extends Controller
{
    /**
     * Display the registration view.
     */
    public function create(): Response
    {
        return Inertia::render('Auth/Registration');
    }

    /**
     * Handle an incoming registration request.
     *
     * @throws \Illuminate\Validation\ValidationException
     */
    public function store(Request $request): RedirectResponse
    {
        // Determine account type based on inputs
        $accountType = 'standalone';
        if ($request->filled('invitation_code')) {
            $accountType = 'join_family';
        } elseif ($request->filled('family_name')) {
            $accountType = 'create_family';
        }

        // Validate based on account type
        $validationRules = [
            'email' => 'required|string|lowercase|email|max:255|unique:' .
User::class,
            'password' => ['required', 'confirmed', Rules\Password::defaults()],
            'address' => 'required|string|max:255',
        ];

        if ($accountType === 'join_family') {
            $validationRules['invitation_code'] =
'required|string|exists:families,invitation_code';
        } elseif ($accountType === 'create_family') {
            $validationRules['family_name'] = 'required|string|max:255';
        }

        $request->validate($validationRules);

        // Extract the name from the email

```

```

$name = strstr($request->email, '@', true);

// Handle different account types
if ($accountType === 'join_family') {
    // Joining existing family
    return $this->joinFamily($request, $name);
} elseif ($accountType === 'create_family') {
    // Creating new family
    return $this->createFamily($request, $name);
} else {
    // Standalone account (no family)
    return $this->createStandaloneAccount($request, $name);
}
}

/**
 * Create a standalone account (no family)
 */
private function createStandaloneAccount(Request $request, string $name):
RedirectResponse
{
    $user = User::create([
        'name' => $name,
        'email' => $request->email,
        'address' => $request->address,
        'password' => Hash::make($request->password),
        'role' => 'standalone', // New role for standalone users
        'is_family_admin' => false,
        'can_use_family_card' => false,
        'family_id' => null,
    ]);

    Auth::login($user);

    $this->sendWelcomeEmail($user);

    return redirect(route('quiz', absolute: false));
}

/**
 * Create a new family account
 */
private function createFamily(Request $request, string $name): RedirectResponse
{
    // Create the user first
    $user = User::create([
        'name' => $name,
        'email' => $request->email,
        'address' => $request->address,
        'password' => Hash::make($request->password),
        'role' => 'parent',
        'is_family_admin' => true,
        'can_use_family_card' => true,
    ]);

    // Create family
    $family = Family::create([
        'family_name' => $request->family_name,
        'invitation_code' => Family::generateInvitationCode(),
        'parent_id' => $user->id,
        'max_members' => 5, // Default value
        'is_active' => true,
    ]);

    // Update user with family_id
    $user->update(['family_id' => $family->id]);

    Auth::login($user);

    $this->sendWelcomeEmail($user);
}

```



```

        return redirect(route('quiz', absolute: false));
    }

    /**
     * Join an existing family
     */
    private function joinFamily(Request $request, string $name): RedirectResponse
    {
        $family = Family::where('invitation_code', strtoupper($request->invitation_code))->first();

        if (!$family) {
            return back()->withErrors([
                'invitation_code' => 'Invalid invitation code.'
            ]);
        }

        if ($family->getAvailableSlots() <= 0) {
            return back()->withErrors([
                'invitation_code' => 'This family has reached its maximum member
limit.'
            ]);
        }

        $user = User::create([
            'name' => $name,
            'email' => $request->email,
            'address' => $request->address,
            'password' => Hash::make($request->password),
            'family_id' => $family->id,
            'role' => 'child',
            'is_family_admin' => false,
            'can_use_family_card' => false, // Parent must enable this
        ]);

        Auth::login($user);

        $this->sendWelcomeEmail($user);

        return redirect(route('quiz', absolute: false));
    }

    /**
     * Send welcome email
     */
    private function sendWelcomeEmail(User $user): void
    {
        try {
            Mail::to($user->email)->send(new WelcomeEmail($user));
        } catch (\Exception $e) {
            // Log error but don't stop registration
            \Log::error('Failed to send welcome email: ' . $e->getMessage());
        }
    }
}

```

Komentaru kontrolieris

```
class CommentController extends Controller
{
    public function index()
    {
        return Comment::with('user:id,name')->latest()->get();
    }

    public function store(Request $request)
    {
        $request->validate([
            'body' => 'required|string|max:1000',
        ]);

        $comment = Comment::create([
            'body' => $request->body,
            'user_id' => auth()->id(),
        ]);

        return $comment->load('user:id,name');
    }
}
```

Parvāldnieka kontrolieris

```
class AdminController extends Controller
{
    public function dashboard(): \Inertia\Response
    {
        $orders = Orders::with(['user', 'orderGoods'])
            ->select(['id', 'user_id', 'status', 'total', 'ordered_at', 'created_at',
            'payment_method', 'shipping_address'])
            ->latest()
            ->limit(100)
            ->get();

        $products = Products::select(['id', 'name', 'price', 'description', 'image',
        'category', 'admin_id', 'created_at'])
            ->latest()
            ->limit(100)
            ->get();

        $ordersj = Orders::join('users', 'orders.user_id', '=', 'users.id')
            ->join('goods_orders', 'orders.id', '=', 'goods_orders.order_id')
            ->select(
                'orders.id as order_id',
                'orders.status as order_status',
                'orders.total',
                'orders.created_at',
                'users.name as customer_name',
                'users.email as customer_email',
                'goods_orders.name as item_name',
                'goods_orders.price as item_price',
                'goods_orders.status as item_status',
            )
            ->latest()
            ->limit(100)
            ->get();

        $comments = Comment::with('user:id,name,email')
            ->latest()
            ->limit(100)
    }
}
```

```

        ->get();

        $users = User::withCount('orders')
            ->select(['id', 'name', 'email', 'role', 'family_id', 'created_at',
'awards'])
            ->latest()
            ->limit(100)
            ->get();

        $families = Family::withCount('users')
            ->with(['parent:id,name,email'])
            ->select(['id', 'family_name', 'parent_id', 'invitation_code',
'created_at'])
            ->latest()
            ->limit(50)
            ->get();

        $auctions = Auction::with(['user:id,name,email'])
            ->latest()
            ->limit(100)
            ->get();

        return Inertia::render('Admin', [
            'orders' => $orders,
            'products' => $products,
            'ordersj' => $ordersj,
            'users' => $users,
            'families' => $families,
            'comments' => $comments,
            'auctions' => $auctions,
        ]);
    }

    // Get all auctions (API)
    public function indexAuctions(): \Illuminate\Http\JsonResponse
    {
        $auctions = Auction::with(['user:id,name,email'])
            ->latest()
            ->get();
        return response()->json($auctions);
    }

    public function showOrders(Request $request): \Illuminate\Http\JsonResponse
    {
        $orders = Orders::with(['user:id,name,email'])
            ->select(['id', 'user_id', 'status', 'total', 'ordered_at', 'created_at'])
            ->latest()
            ->get();

        return response()->json($orders);
    }

    public function showOrderItems(): \Illuminate\Http\JsonResponse
    {
        $orderItems = OrderGoods::select([
            'id', 'order_id', 'name', 'price', 'status', 'created_at'
        ])->latest()->get();

        return response()->json($orderItems);
    }

    public function showProducts(Request $request): \Illuminate\Http\JsonResponse
    {
        $products = Products::select(['id', 'name', 'price', 'description', 'image',
'category', 'created_at'])
            ->latest()
            ->get();

        return response()->json($products);
    }
}

```

```

public function showUsers(Request $request): \Illuminate\Http\JsonResponse
{
    $users = User::withCount('orders')
        ->with(['family:id,family_name'])
        ->select(['id', 'name', 'email', 'role', 'family_id', 'created_at',
'awards'])
        ->latest()
        ->get();

    return response()->json($users);
}

public function showFamilies(Request $request): \Illuminate\Http\JsonResponse
{
    $families = Family::withCount('users')
        ->with(['parent:id,name,email', 'users:id,name,email,role'])
        ->select(['id', 'family_name', 'parent_id', 'invitation_code',
'created_at'])
        ->latest()
        ->get();

    return response()->json($families);
}

// Get bids for a specific auction
public function showBids($auctionId): \Illuminate\Http\JsonResponse
{
    $bids = Bids::with('user:id,name,email')
        ->where('item_id', $auctionId)
        ->orderBy('bid_amount', 'desc')
        ->get();

    return response()->json($bids);
}

// Get all bids
public function indexBids(): \Illuminate\Http\JsonResponse
{
    $bids = Bids::with(['user:id,name,email', 'auction:id,name'])
        ->latest()
        ->get();

    return response()->json($bids);
}

// store products
public function storeProduct(Request $request)
{
    $request->validate([
        'name' => 'required|string|max:255',
        'price' => 'required|numeric|min:0',
        'category' => 'required|string|max:255',
        'description' => 'nullable|string',
        'image' => 'required|image|mimes:jpeg,png,jpg,gif|max:2048',
    ]);

    try {
        $imagePath = null;
        if ($request->hasFile('image')) {
            $image = $request->file('image');
            $imageName = time() . '_' . $image->getClientOriginalName();
            $image->move(public_path('images/front'), $imageName);
            $imagePath = 'images/front/' . $imageName;
        }

        Products::create([
            'name' => $request->name,
            'price' => $request->price,
            'category' => $request->category,
            'description' => $request->description,
            'image' => $imagePath,
        ]);
    } catch (\Exception $e) {
        return response()->json(['error' => $e->getMessage()], 500);
    }
}

```

```

        'admin_id' => auth()->id(),
    ]);

    return redirect()->back()->with('success', 'Product added successfully!');

} catch (\Exception $e) {
    Log::error('Error adding product: ' . $e->getMessage());
    return redirect()->back()->with('error', 'Failed to add product: ' . $e-
>getMessage());
}

}

// store users
public function storeUser(Request $request)
{
    $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|string|email|max:255|unique:users',
        'password' => 'required|string|min:8',
        'role' => 'required|in:user,parent,child,standalone',
        'awards' => 'nullable|integer|min:0',
        'family_id' => 'nullable|exists:families,id',
    ]);

    try {
        // Ensure role is valid for your database
        $validRoles = ['standalone', 'parent', 'child', 'admin'];
        $role = in_array($request->role, $validRoles) ? $request->role : 'user';

        $userData = [
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
            'role' => $role,
            'awards' => $request->awards ?? 0,
            'family_id' => $request->family_id ?? null,
            'email_verified_at' => now(),
            'created_at' => now(),
            'updated_at' => now(),
        ];

        \Log::info('User data to create:', $userData);

        User::create($userData);

        return redirect()->back()->with('success', 'User added successfully!');

    } catch (\Exception $e) {
        \Log::error('Full error creating user: ' . $e->getMessage());
        return redirect()->back()->with('error', 'Failed to add user: ' . $e-
>getMessage());
    }
}

// store families
public function storeFamily(Request $request)
{
    $request->validate([
        'family_name' => 'required|string|max:255|unique:families,family_name',
        'parent_id' => 'required|exists:users,id',
    ]);

    try {
        $invitationCode = strtoupper(substr(md5(uniqid()), 0, 8));

        $family = Family::create([
            'family_name' => $request->family_name,
            'parent_id' => $request->parent_id,
            'invitation_code' => $invitationCode,
        ]);
    }
}

```

```

        User::where('id', $request->parent_id)->update([
            'family_id' => $family->id,
            'role' => 'parent',
        ]);

        return redirect()->back()->with('success', 'Family created successfully!
Invitation code: ' . $invitationCode);

    } catch (\Exception $e) {
        Log::error('Error creating family: ' . $e->getMessage());
        return redirect()->back()->with('error', 'Failed to create family: ' . $e-
>getMessage());
    }
}

// Store new auction
public function storeAuction(Request $request): \Illuminate\Http\JsonResponse
{
    $request->validate([
        'name' => 'required|string|max:255',
        'description' => 'required|string',
        'starting_bid' => 'required|numeric|min:0',
        'img' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',
        'start_time' => 'required|date',
        'end_time' => 'required|date|after:start_time',
    ]);

    try {
        $imgPath = null;
        if ($request->hasFile('img')) {
            $img = $request->file('img');
            $imgName = time() . '_' . $img->getClientOriginalName();
            $img->move(public_path('images/auctions'), $imgName);
            $imgPath = 'images/auctions/' . $imgName;
        }

        $auction = Auction::create([
            'name' => $request->name,
            'description' => $request->description,
            'starting_bid' => $request->starting_bid,
            'img' => $imgPath,
            'start_time' => $request->start_time,
            'end_time' => $request->end_time,
            'user_id' => auth()->id(),
        ]);

        return response()->json([
            'success' => true,
            'message' => 'Auction created successfully!',
            'auction' => $auction->load('user:id,name,email')
        ]);
    } catch (\Exception $e) {
        Log::error('Error creating auction: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to create auction.'], 500);
    }
}

// Update auction
public function updateAuction(Request $request, $id):
\Illuminate\Http\JsonResponse
{
    $request->validate([
        'name' => 'required|string|max:255',
        'description' => 'required|string',
        'starting_bid' => 'required|numeric|min:0',
        'img' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',
        'start_time' => 'required|date',
        'end_time' => 'required|date|after:start_time',
    ]);

    try {

```

```

        $auction = Auction::findOrFail($id);

        $updateData = $request->only(['name', 'description', 'starting_bid',
'start_time', 'end_time']);

        if ($request->hasFile('img')) {
            // Delete old image
            if ($auction->img && file_exists(public_path($auction->img))) {
                unlink(public_path($auction->img));
            }
            $img = $request->file('img');
            $imgName = time() . '_' . $img->getClientOriginalName();
            $img->move(public_path('images/auctions'), $imgName);
            $updateData['img'] = 'images/auctions/' . $imgName;
        }

        $auction->update($updateData);

        return response()->json([
            'success' => true,
            'message' => 'Auction updated successfully!',
            'auction' => $auction->load('user:id,name,email')
        ]);
    } catch (\Exception $e) {
        Log::error('Error updating auction: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to update auction.'], 500);
    }
}

// Delete auction
public function destroyAuction($id): \Illuminate\Http\JsonResponse
{
    try {
        $auction = Auction::findOrFail($id);
        // Delete associated bids first
        Bid::where('item_id', $id)->delete();
        // Delete image if exists
        if ($auction->img && file_exists(public_path($auction->img))) {
            unlink(public_path($auction->img));
        }
        $auction->delete();

        return response()->json(['success' => 'Auction deleted successfully!']);
    } catch (\Exception $e) {
        Log::error('Error deleting auction: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to delete auction.'], 500);
    }
}

// delete products
public function destroyProduct($id)
{
    try {
        $product = Products::findOrFail($id);
        if ($product->image && file_exists(public_path($product->image))) {
            unlink(public_path($product->image));
        }
        $product->delete();

        return response()->json(['success' => 'Product deleted successfully!']);
    } catch (\Exception $e) {
        Log::error('Error deleting product: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to delete product.'], 500);
    }
}

// delete orders
public function destroyOrder($id)
{
    try {

```

```

        $order = Orders::findOrFail($id);
        // Delete associated order goods
        OrderGoods::where('order_id', $id)->delete();
        $order->delete();

        return response()->json(['success' => 'Order deleted successfully!']);
    } catch (\Exception $e) {
        Log::error('Error deleting order: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to delete order.'], 500);
    }
}

// delete products = orders
public function destroyOrderItem($id)
{
    try {
        $orderItem = OrderGoods::findOrFail($id);
        $orderItem->delete();
        return response()->json(['success' => 'Order item deleted
successfully!']);
    } catch (\Exception $e) {
        Log::error('Error deleting order item: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to delete order item.'], 500);
    }
}

// delete user
public function deleteUser($id)
{
    try {
        $user = User::findOrFail($id);
        if ($user->orders()->exists()) {
            return response()->json(['error' => 'Cannot delete user with existing
orders.'], 400);
        }
        $user->delete();
        return response()->json(['success' => 'User deleted successfully!']);
    } catch (\Exception $e) {
        Log::error('Error deleting user: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to delete user.'], 500);
    }
}

// delete family
public function destroyFamily($id)
{
    try {
        $family = Family::findOrFail($id);

        // Remove family_id from all users in this family
        User::where('family_id', $id)->update(['family_id' => null, 'role' =>
'user']);

        $family->delete();
        return response()->json(['success' => 'Family deleted successfully!']);
    } catch (\Exception $e) {
        Log::error('Error deleting family: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to delete family.'], 500);
    }
}

// update product
public function updateProduct(Request $request, $id)
{
    $request->validate([
        'name' => 'required|string|max:255',
        'price' => 'required|numeric|min:0',
        'category' => 'required|string|max:255',
        'description' => 'nullable|string',
        'image' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',
    ]);
}

```



```

    try {
        $product = Products::findOrFail($id);

        $updateData = [
            'name' => $request->name,
            'price' => $request->price,
            'category' => $request->category,
            'description' => $request->description,
        ];

        if ($request->hasFile('image')) {
            if ($product->image && file_exists(public_path($product->image))) {
                unlink(public_path($product->image));
            }

            $image = $request->file('image');
            $imageName = time() . '_' . $image->getClientOriginalName();
            $image->move(public_path('images/front/'), $imageName);
            $updateData['image'] = 'images/front/' . $imageName;
        }

        $product->update($updateData);

        return redirect()->back()->with('success', 'Product updated successfully!');
    } catch (\Exception $e) {
        Log::error('Error updating product: ' . $e->getMessage());
        return redirect()->back()->with('error', 'Failed to update product: ' . $e->getMessage());
    }
}

// update orders
public function updateOrder(Request $request, $id)
{
    $request->validate([
        'status' => 'required|in:pending,processing,completed,cancelled',
        'total' => 'nullable|numeric|min:0',
    ]);

    try {
        // Find the order
        $order = Orders::findOrFail($id);

        // Update order data
        $order->status = $request->status;
        if ($request->has('total')) {
            $order->total = $request->total;
        }
        $order->save();

        // Update all related order items with the same status
        OrderGoods::where('order_id', $order->id)
            ->update(['status' => $request->status]);

        return response()->json([
            'success' => true,
            'message' => 'Order updated successfully!'
        ]);
    } catch (\Exception $e) {
        Log::error('Error updating order: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to update order.'], 500);
    }
}

// update user
public function updateUser(Request $request, $id)

```

```

{
    $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|string|email|max:255|unique:users,email,' . $id,
        'password' => 'nullable|string|min:8',
        'role' => 'required|in:user,parent,child,admin',
        'awards' => 'nullable|integer|min:0',
        'family_id' => 'nullable|exists:families,id',
    ]);

    try {
        $user = User::findOrFail($id);

        $updateData = [
            'name' => $request->name,
            'email' => $request->email,
            'role' => $request->role,
            'awards' => $request->awards ?? $user->awards,
            'family_id' => $request->family_id,
        ];

        if ($request->filled('password')) {
            $updateData['password'] = Hash::make($request->password);
        }

        $user->update($updateData);

        return response()->json(['success' => true, 'message' => 'User updated successfully!']);
    } catch (\Exception $e) {
        Log::error('Error updating user: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to update user.'], 500);
    }
}

// update families
public function updateFamily(Request $request, $id): \Illuminate\Http\JsonResponse
{
    $request->validate([
        'family_name' => 'required|string|max:255|unique:families,family_name,' .
$id,
        'parent_id' => 'required|exists:users,id',
    ]);

    try {
        $family = Family::findOrFail($id);

        // Update old parent role
        $oldParent = User::find($family->parent_id);
        if ($oldParent && $oldParent->id != $request->parent_id) {
            $oldParent->update(['role' => 'user']);
        }

        // Update family
        $family->update([
            'family_name' => $request->family_name,
            'parent_id' => $request->parent_id,
        ]);

        // Update new parent
        $newParent = User::find($request->parent_id);
        if ($newParent) {
            $newParent->update([
                'family_id' => $family->id,
                'role' => 'parent',
            ]);
        }

        return response()->json([
            'success' => true,

```

```

        'message' => 'Family updated successfully!',
        'family' => $family
    });

    } catch (\Exception $e) {
        Log::error('Error updating family: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to update family: ' . $e-
>getMessage()], 500);
    }
}

/**
 * Get all comments for API
 */
public function showComments(Request $request): \Illuminate\Http\JsonResponse
{
    $comments = Comment::with('user:id,name,email')
        ->select(['id', 'body', 'user_id', 'created_at', 'updated_at'])
        ->latest()
        ->get();

    return response()->json($comments);
}

/**
 * Store a new comment
 */
public function storeComment(Request $request)
{
    $request->validate([
        'body' => 'required|string|max:1000',
    ]);

    try {
        $comment = Comment::create([
            'body' => $request->body,
            'user_id' => auth()->id(),
        ]);

        // Load the user relationship for the response
        $comment->load('user:id,name,email');

        if ($request->wantsJson()) {
            return response()->json([
                'success' => true,
                'message' => 'Comment added successfully!',
                'comment' => $comment
            ]);
        }

        return redirect()->back()->with('success', 'Comment added successfully!');

    } catch (\Exception $e) {
        Log::error('Error adding comment: ' . $e->getMessage());

        if ($request->wantsJson()) {
            return response()->json(['error' => 'Failed to add comment: ' . $e-
>getMessage()], 500);
        }

        return redirect()->back()->with('error', 'Failed to add comment: ' . $e-
>getMessage());
    }
}

/**
 * Update an existing comment
 */
public function updateComment(Request $request, $id)
{
    $request->validate([

```

```

        'body' => 'required|string|max:1000',
    ]);

    try {
        $comment = Comment::findOrFail($id);

        // Check if user is authorized (admin or comment owner)
        if (auth()->user()->role !== 'admin' && $comment->user_id !== auth()-
>id()) {
            return response()->json(['error' => 'Unauthorized'], 403);
        }

        $comment->update([
            'body' => $request->body,
        ]);

        $comment->load('user:id,name,email');

        return response()->json([
            'success' => true,
            'message' => 'Comment updated successfully!',
            'comment' => $comment
        ]);

    } catch (\Exception $e) {
        Log::error('Error updating comment: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to update comment: ' . $e-
>getMessage()], 500);
    }
}

/**
 * Delete a comment
 */
public function destroyComment($id)
{
    try {
        $comment = Comment::findOrFail($id);

        $comment->delete();

        return response()->json([
            'success' => true,
            'message' => 'Comment deleted successfully!'
        ]);

    } catch (\Exception $e) {
        Log::error('Error deleting comment: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to delete comment: ' . $e-
>getMessage()], 500);
    }
}

public function getAnalytics(): \Illuminate\Http\JsonResponse
{
    try {
        // Basic stats
        $totalOrders = Orders::count();
        $totalRevenue = Orders::sum('total');
        $totalProducts = Products::count();
        $totalUsers = User::count();
        $totalFamilies = Family::count();

        $totalComments = Comment::count();

        // Comments per day for last 30 days
        $commentsPerDay = Comment::selectRaw('DATE(created_at) as date, COUNT(*)
as count')
            ->where('created_at', '>=', now()->subDays(30))
            ->groupBy('date')

```

```

        ->orderBy('date')
        ->get();

    // Most active commenters
    $stopCommenters = Comment::join('users', 'comments.user_id', '=',
'users.id')
        ->selectRaw('users.id, users.name, users.email, COUNT(comments.id) as
comment_count')
        ->groupBy('users.id', 'users.name', 'users.email')
        ->orderByDesc('comment_count')
        ->limit(10)
        ->get();

    // Recent orders (last 30 days)
    $recentOrders = Orders::where('created_at', '>=', now()->subDays(30))
        ->count();

    // Recent revenue (last 30 days)
    $recentRevenue = Orders::where('created_at', '>=', now()->subDays(30))
        ->sum('total');

    // Orders per day for last 30 days
    $ordersPerDay = Orders::selectRaw('DATE(created_at) as date, COUNT(*) as
count')
        ->where('created_at', '>=', now()->subDays(30))
        ->groupBy('date')
        ->orderBy('date')
        ->get();

    // Revenue per day for last 30 days
    $revenuePerDay = Orders::selectRaw('DATE(created_at) as date, SUM(total)
as revenue')
        ->where('created_at', '>=', now()->subDays(30))
        ->groupBy('date')
        ->orderBy('date')
        ->get();

    // Top products by sales
    $stopProducts = OrderGoods::selectRaw('name, COUNT(*) as quantity_sold,
SUM(price) as revenue')
        ->groupBy('name')
        ->orderByDesc('revenue')
        ->limit(10)
        ->get();

    // Order status distribution
    $statusDistribution = Orders::selectRaw('status, COUNT(*) as count')
        ->groupBy('status')
        ->get();

    // Payment method distribution
    $paymentMethodDistribution = Orders::selectRaw('payment_method, COUNT(*)
as count')
        ->whereNotNull('payment_method')
        ->groupBy('payment_method')
        ->get();

    // Top customers by spending
    $stopCustomers = Orders::join('users', 'orders.user_id', '=', 'users.id')
        ->selectRaw('users.id, users.name, users.email, COUNT(orders.id) as
order_count, SUM(orders.total) as total_spent')
        ->groupBy('users.id', 'users.name', 'users.email')
        ->orderByDesc('total_spent')
        ->limit(10)
        ->get();

    // Category performance
    $categoryPerformance = OrderGoods::join('products',
'goods_orders.product_id', '=', 'products.id')
        ->selectRaw('
products.category,

```

```

        COUNT(*) as item_count,
        SUM(goods_orders.price * goods_orders.quantity) as revenue')
    ->whereNotNull('goods_orders.product_id')
    ->groupBy('products.category')
    ->orderByDesc('revenue')
    ->get();

    // Monthly trends (last 6 months)
    $monthlyTrends = Orders::selectRaw('DATE_FORMAT(created_at, "%Y-%m") as
month, COUNT(*) as order_count, SUM(total) as revenue')
    ->where('created_at', '>=', now()->subMonths(6))
    ->groupBy('month')
    ->orderBy('month')
    ->get();

    // User growth
    $userGrowth = User::selectRaw('DATE(created_at) as date, COUNT(*) as
new_users')
    ->where('created_at', '>=', now()->subDays(30))
    ->groupBy('date')
    ->orderBy('date')
    ->get();

    // Family stats
    $averageFamilySize = Family::withCount('users')->get()-
>avg('users_count');
    $largestFamily = Family::withCount('users')->orderByDesc('users_count')-
>first();

    return response()->json([
        // Basic stats
        'totalOrders' => $totalOrders,
        'totalRevenue' => $totalRevenue,
        'totalProducts' => $totalProducts,
        'totalUsers' => $totalUsers,
        'totalFamilies' => $totalFamilies,

        // Recent activity
        'recentOrders' => $recentOrders,
        'recentRevenue' => $recentRevenue,

        // Time series data
        'ordersPerDay' => $ordersPerDay,
        'revenuePerDay' => $revenuePerDay,
        'monthlyTrends' => $monthlyTrends,
        'userGrowth' => $userGrowth,

        // Product analytics
        'topProducts' => $topProducts,
        'categoryPerformance' => $categoryPerformance,

        // Customer analytics
        'topCustomers' => $topCustomers,
        'statusDistribution' => $statusDistribution,
        'paymentMethodDistribution' => $paymentMethodDistribution,

        // Family analytics
        'averageFamilySize' => $averageFamilySize,
        'largestFamily' => $largestFamily,

        'totalComments' => $totalComments,
        'commentsPerDay' => $commentsPerDay,
        'topCommenters' => $topCommenters,
    ]);

    } catch (\Exception $e) {
        Log::error('Error getting analytics: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to get analytics.'], 500);
    }
}

```

```

public function exportOrders(Request $request)
{
    $request->validate([
        'start_date' => 'nullable|date',
        'end_date' => 'nullable|date|after_or_equal:start_date',
    ]);

    try {
        $query = Orders::with(['user:id,name,email', 'orderGoods']);

        if ($request->has('start_date')) {
            $query->where('created_at', '>=', $request->start_date);
        }

        if ($request->has('end_date')) {
            $query->where('created_at', '<=', $request->end_date . ' 23:59:59');
        }

        $orders = $query->latest()->get();

        // Format for CSV/Excel export
        $exportData = [];
        foreach ($orders as $order) {
            $exportData[] = [
                'Order ID' => $order->id,
                'Customer' => $order->user->name ?? 'N/A',
                'Email' => $order->user->email ?? 'N/A',
                'Status' => $order->status,
                'Total' => $order->total,
                'Payment Method' => $order->payment_method ?? 'N/A',
                'Shipping Address' => $order->shipping_address ?? 'N/A',
                'Order Date' => $order->created_at,
                'Items Count' => $order->orderGoods->count(),
            ];
        }

        return response()->json([
            'success' => true,
            'data' => $exportData,
            'count' => count($exportData),
        ]);

    } catch (\Exception $e) {
        Log::error('Error exporting orders: ' . $e->getMessage());
        return response()->json(['error' => 'Failed to export orders.'], 500);
    }
}

// export data to pdf
public function exportAllToPdf()
{
    // Fetch data (limit to 1000 rows per table to avoid memory issues)
    $users = User::select('id', 'name', 'email', 'role', 'awards', 'created_at')
        ->orderBy('id')->limit(1000)->get();

    $products = Products::select('id', 'name', 'price', 'category', 'created_at')
        ->orderBy('id')->limit(1000)->get();

    $orders = Orders::select('id', 'user_id', 'status', 'total', 'created_at')
        ->with('user:id,name')
        ->orderBy('id')->limit(1000)->get();

    $orderGoods = OrderGoods::select('id', 'order_id', 'name', 'price',
        'quantity', 'status', 'created_at')
        ->orderBy('id')->limit(1000)->get();

    $families = Family::select('id', 'family_name', 'parent_id',
        'invitation_code', 'created_at')
        ->with('parent:id,name')
        ->orderBy('id')->limit(1000)->get();
}

```

```

    $comments = Comment::select('id', 'user_id', 'body', 'created_at')
        ->with('user:id,name')
        ->orderBy('id')->limit(1000)->get();

    $auctions = Auction::select('id', 'name', 'starting_bid', 'start_time',
'end_time', 'created_at')
        ->orderBy('id')->limit(1000)->get();

    $bids = Bids::select('id', 'item_id', 'user_id', 'bid_amount', 'created_at')
        ->with(['user:id,name', 'auction:id,name'])
        ->orderBy('id')->limit(1000)->get();

    $data = compact('users', 'products', 'orders', 'orderGoods', 'families',
'comments', 'auctions', 'bids');

    $pdf = Pdf::loadView('export-all', $data);
    $pdf->setPaper('A4', 'landscape'); // landscape fits more columns

    return $pdf->download('database_export_' . now()->format('Y-m-d_His') .
'.pdf');
}
}

```

12. pielikums

MI (DeepSeek) kontrolieris

```

class ChatController extends Controller
{
    private $productKeywords = [
        'product', 'products', 'buy', 'purchase', 'price', 'cost', 'sale', 'deal',
        'specification', 'specs', 'feature', 'features', 'stock',
        'available', 'availability', 'order', 'pc', 'computer',
        'laptop', 'laptops', 'notebook', 'notebooks', 'ultrabook',
        'macbook', 'dell', 'hp', 'lenovo', 'asus', 'acer', 'msi',
        'gaming', 'business', 'student', 'cpu', 'processor', 'intel', 'amd', 'core',
        'gpu', 'graphics', 'video card', 'graphics card', 'nvidia', 'raadeon',
        'ram', 'memory', 'storage', 'ssd', 'hard drive', 'hdd',
        'motherboard', 'mainboard', 'mb', 'power supply', 'psu',
        'case', 'chassis', 'cooling', 'fan', 'heatsink', 'thermal',
        'monitor', 'display', 'screen', 'keyboard', 'keyboards', 'mouse', 'mice',
        'workstation', 'budget', 'cheap', 'expensive', 'discount', 'offer',
        'component', 'components', 'part', 'parts', 'hardware'
    ];

    private $categoryMappings = [
        'cpu' => [
            'keywords' => ['cpu', 'processor', 'intel', 'amd', 'core i3', 'core i5',
'core i7', 'core i9', 'ryzen'],
            'search_fields' => [
                ['category', 'LIKE', '%component%'],
                ['name', 'LIKE', '%intel%'],
                ['name', 'LIKE', '%amd%'],
                ['name', 'LIKE', '%core%'],
                ['name', 'LIKE', '%ryzen%'],
                ['description', 'LIKE', '%processor%']
            ],
            'category_name' => 'CPU/Processor'
        ],
        'gpu' => [
            'keywords' => ['gpu', 'graphics', 'video card', 'graphics card', 'nvidia',
'geforce', 'rtx', 'gtx', 'raadeon'],
            'search_fields' => [
                ['category', 'LIKE', '%component%'],
                ['name', 'LIKE', '%nvidia%'],

```



```

        ['name', 'LIKE', '%geforce%'],
        ['name', 'LIKE', '%rtx%'],
        ['name', 'LIKE', '%gtx%'],
        ['name', 'LIKE', '%radeon%'],
        ['description', 'LIKE', '%graphics%']
    ],
    'category_name' => 'GPU/Graphics Card'
],
'ram' => [
    'keywords' => ['ram', 'memory', 'ddr4', 'ddr5', 'memory stick'],
    'search_fields' => [
        ['category', 'LIKE', '%component%'],
        ['name', 'LIKE', '%ram%'],
        ['name', 'LIKE', '%memory%'],
        ['name', 'LIKE', '%ddr%']
    ],
    'category_name' => 'RAM/Memory'
],
'storage' => [
    'keywords' => ['ssd', 'hard drive', 'hdd', 'storage', 'nvme', 'sata'],
    'search_fields' => [
        ['category', 'LIKE', '%component%'],
        ['name', 'LIKE', '%ssd%'],
        ['name', 'LIKE', '%hdd%'],
        ['name', 'LIKE', '%hard drive%'],
        ['description', 'LIKE', '%storage%']
    ],
    'category_name' => 'Storage'
],
'motherboard' => [
    'keywords' => ['motherboard', 'mainboard', 'mb', 'mobo'],
    'search_fields' => [
        ['category', 'LIKE', '%component%'],
        ['name', 'LIKE', '%motherboard%'],
        ['name', 'LIKE', '%mainboard%']
    ],
    'category_name' => 'Motherboard'
],
'psu' => [
    'keywords' => ['power supply', 'psu', 'power', 'watt'],
    'search_fields' => [
        ['category', 'LIKE', '%component%'],
        ['name', 'LIKE', '%power%'],
        ['name', 'LIKE', '%psu%'],
        ['description', 'LIKE', '%watt%']
    ],
    'category_name' => 'Power Supply'
],
'case' => [
    'keywords' => ['case', 'chassis', 'tower', 'pc case'],
    'search_fields' => [
        ['category', 'LIKE', '%component%'],
        ['name', 'LIKE', '%case%'],
        ['name', 'LIKE', '%chassis%']
    ],
    'category_name' => 'PC Case'
],
'cooling' => [
    'keywords' => ['cooling', 'fan', 'cooler', 'heatsink', 'thermal',
'airflow'],
    'search_fields' => [
        ['category', 'LIKE', '%component%'],
        ['name', 'LIKE', '%fan%'],
        ['name', 'LIKE', '%cooler%'],
        ['description', 'LIKE', '%cooling%']
    ],
    'category_name' => 'Cooling'
],
'monitor' => [
    'keywords' => ['monitor', 'display', 'screen', 'lcd', 'led', '4k',
'144hz'],

```

```

        'search_fields' => [
            ['category', '=', 'Monitor'],
            ['name', 'LIKE', '%monitor%'],
            ['name', 'LIKE', '%display%']
        ],
        'category_name' => 'Monitor'
    ],
    'keyboard' => [
        'keywords' => ['keyboard', 'mechanical keyboard', 'keyboards'],
        'search_fields' => [
            ['category', '=', 'Keyboard'],
            ['name', 'LIKE', '%keyboard%']
        ],
        'category_name' => 'Keyboard'
    ],
    'mouse' => [
        'keywords' => ['mouse', 'gaming mouse', 'mice'],
        'search_fields' => [
            ['category', '=', 'Mouse'],
            ['name', 'LIKE', '%mouse%']
        ],
        'category_name' => 'Mouse'
    ],
    'headphones' => [
        'keywords' => ['headphones', 'headset', 'headphone', 'headsets', 'audio'],
        'search_fields' => [
            ['category', '=', 'Headphones'],
            ['name', 'LIKE', '%headphone%'],
            ['name', 'LIKE', '%headset%']
        ],
        'category_name' => 'Headphones'
    ],
    'desktop' => [
        'keywords' => ['desktop', 'pc', 'computer', 'workstation', 'tower'],
        'search_fields' => [
            ['category', '=', 'Desktop'],
            ['name', 'LIKE', '%desktop%'],
            ['name', 'LIKE', '%pc%'],
            ['name', 'LIKE', '%computer%']
        ],
        'category_name' => 'Desktop PC'
    ],
    'laptop' => [
        'keywords' => ['laptop', 'laptops', 'notebook', 'notebooks', 'ultrabook'],
        'search_fields' => [
            ['category', '=', 'Laptop'],
            ['name', 'LIKE', '%laptop%'],
            ['name', 'LIKE', '%notebook%']
        ],
        'category_name' => 'Laptop'
    ]
];

private $brandMappings = [
    'intel' => ['intel', 'core i3', 'core i5', 'core i7', 'core i9'],
    'amd' => ['amd', 'ryzen'],
    'nvidia' => ['nvidia', 'geforce', 'rtx', 'gtx'],
    'corsair' => ['corsair'],
    'kingston' => ['kingston', 'hyperx'],
    'samsung' => ['samsung'],
    'western digital' => ['western digital', 'wd'],
    'seagate' => ['seagate'],
    'asus' => ['asus', 'rog'],
    'msi' => ['msi'],
    'gigabyte' => ['gigabyte'],
    'evga' => ['evga'],
    'cooler master' => ['cooler master'],
    'noctua' => ['noctua'],
    'logitech' => ['logitech'],
    'razer' => ['razer'],
    'steelseries' => ['steelseries']
];

```

```

];

private function extractProductKeywords($message)
{
    $message = strtolower($message);
    $foundKeywords = [];

    foreach ($this->productKeywords as $keyword) {
        if (str_contains($message, $keyword)) {
            $foundKeywords[] = $keyword;
        }
    }

    return $foundKeywords;
}

private function searchProducts($message)
{
    if (!class_exists('App\Models\Products')) {
        Log::warning('Products model not found');
        return collect();
    }

    $messageLower = strtolower($message);
    $query = Products::query();

    // Check for specific product category queries first
    $matchedCategory = $this->getMatchedCategory($messageLower);

    if ($matchedCategory) {
        // Apply category-specific search
        $this->applyCategorySearch($query, $matchedCategory, $messageLower);
    } else {
        // Generic search based on extracted terms
        $searchTerms = $this->extractSearchTerms($message);
        if (!empty($searchTerms)) {
            $this->applyGenericSearch($query, $searchTerms);
        }
    }

    // Always search for brands mentioned in the query
    $this->applyBrandSearch($query, $messageLower);

    // Apply price range filters if mentioned
    $this->applyPriceFilters($query, $messageLower);

    Log::info('Searching products for query: ' . substr($message, 0, 100));

    return $query->distinct()->limit(15)->get();
}

private function getMatchedCategory($messageLower)
{
    foreach ($this->categoryMappings as $categoryKey => $categoryData) {
        foreach ($categoryData['keywords'] as $keyword) {
            if (str_contains($messageLower, $keyword)) {
                return $categoryKey;
            }
        }
    }

    return null;
}

private function applyCategorySearch($query, $categoryKey, $messageLower)
{
    $categoryData = $this->categoryMappings[$categoryKey];

    $query->where(function($q) use ($categoryData, $messageLower) {
        foreach ($categoryData['search_fields'] as $field) {
            if (count($field) === 3) {
                [$column, $operator, $value] = $field;
            }
        }
    });
}

```

```

        $q->orWhere($column, $operator, $value);
    }
}

// Also search in description for category keywords
foreach ($categoryData['keywords'] as $keyword) {
    if (str_contains($messageLower, $keyword)) {
        $q->orWhere('description', 'LIKE', "%{$keyword}%");
    }
}
});
}

private function applyGenericSearch($query, $searchTerms)
{
    $query->where(function($q) use ($searchTerms) {
        foreach ($searchTerms as $term) {
            $q->orWhere('name', 'LIKE', "%{$term}%")
            ->orWhere('description', 'LIKE', "%{$term}%")
            ->orWhere('category', 'LIKE', "%{$term}%");
        }
    });
}

private function applyBrandSearch($query, $messageLower)
{
    foreach ($this->brandMappings as $brandName => $brandKeywords) {
        foreach ($brandKeywords as $keyword) {
            if (str_contains($messageLower, $keyword)) {
                $query->orWhere(function($q) use ($brandName, $keyword) {
                    $q->where('name', 'LIKE', "%{$brandName}%")
                    ->orWhere('name', 'LIKE', "%{$keyword}%")
                    ->orWhere('description', 'LIKE', "%{$brandName}%")
                    ->orWhere('description', 'LIKE', "%{$keyword}%");
                });
                break;
            }
        }
    }
}

private function applyPriceFilters($query, $messageLower)
{
    // Check for budget/price range mentions
    if (str_contains($messageLower, 'cheap') || str_contains($messageLower, 'budget') || str_contains($messageLower, 'low cost')) {
        $query->where('price', '<=', 100);
    } elseif (str_contains($messageLower, 'mid range') || str_contains($messageLower, 'mid-range')) {
        $query->whereBetween('price', [100, 300]);
    } elseif (str_contains($messageLower, 'high end') || str_contains($messageLower, 'expensive') || str_contains($messageLower, 'premium')) {
        $query->where('price', '>=', 300);
    }

    // Look for specific price mentions like "under $200"
    if (preg_match('/under\s+\$(\d+)/i', $messageLower, $matches)) {
        $query->where('price', '<=', $matches[1]);
    }

    if (preg_match('/over\s+\$(\d+)/i', $messageLower, $matches)) {
        $query->where('price', '>=', $matches[1]);
    }
}

private function extractSearchTerms($message)
{
    $stopWords = ['the', 'a', 'an', 'and', 'or', 'but', 'in', 'on', 'at', 'to', 'for', 'of', 'with', 'by', 'is', 'are', 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had', 'do', 'does', 'did', 'will', 'would', 'should', 'could', 'can', 'may', 'might',

```

```

        'must', 'about', 'tell', 'me', 'your', 'store', 'shop',
        'want', 'looking', 'need', 'find', 'show', 'give'];

    $words = str_word_count(strtolower($message), 1);
    $terms = array_diff($words, $stopWords);

    return array_filter($terms, function($word) {
        return strlen($word) > 2 || in_array($word, ['pc', 'cpu', 'gpu', 'ram',
'ssd', 'hdd', 'psu', 'mb']);
    });
}

private function formatProductContext($products, $userMessage)
{
    if ($products->isEmpty()) {
        Log::info('No products found for the query');

        // Provide helpful suggestions based on the query
        $messageLower = strtolower($userMessage);
        $suggestions = [];

        foreach ($this->categoryMappings as $categoryKey => $categoryData) {
            foreach ($categoryData['keywords'] as $keyword) {
                if (str_contains($messageLower, $keyword)) {
                    $suggestions[] = "We might have
{$categoryData['category_name']} products available. Please check our full catalog.";
                    break;
                }
            }
        }

        if (empty($suggestions)) {
            $suggestions[] = "We have a wide range of PC components including
CPUs, GPUs, RAM, and storage.";
        }

        return "I couldn't find exact products matching your search. " . implode('
', $suggestions) .
            " You can ask about specific components like 'Intel processors',
'Nvidia graphics cards', or 'SSD storage'.";
    }

    Log::info('Found ' . $products->count() . ' products');

    $context = "Here are the products available in our store that match your
query:\n\n";

    foreach ($products as $product) {
        $context .= "**Product Name:** {$product->name}\n";
        $context .= "**Category:** {$product->category}\n";

        if ($product->price) {
            $context .= "**Price:** $" . number_format($product->price, 2) . "\n";
        }

        if ($product->description) {
            // Clean up description - remove extra spaces and newlines
            $cleanDescription = trim(preg_replace('/\s+/', ' ', $product->
description));
            $context .= "**Description:** {$cleanDescription}\n";
        }

        $context .= "---\n";
    }

    $context .= "\nYou have direct access to these real products from our
database. ";
    $context .= "Use this information to answer the user's question accurately. ";
    $context .= "If the user asks about specifications, features, or
recommendations, ";
    $context .= "refer to these specific products and their details.";
}

```

```

        return $context;
    }

    public function chat(Request $request)
    {
        $request->validate([
            'message' => 'required|string|max:500',
        ]);

        $apiKey = config('services.deepseek.api_key');

        if (empty($apiKey) || !str_starts_with($apiKey, 'sk-')) {
            Log::error('DeepSeek API key issue');
            return response()->json([
                'reply' => 'AI service configuration error.'
            ], 500);
        }

        $userMessage = $request->input('message');
        Log::info('User message: ' . $userMessage);

        $isProductQuery = !empty($this->extractProductKeywords($userMessage));
        Log::info('Is product query: ' . ($isProductQuery ? 'Yes' : 'No'));

        $productContext = "";
        $products = collect();

        if ($isProductQuery) {
            $products = $this->searchProducts($userMessage);
            if ($products->isNotEmpty()) {
                Log::info('Found ' . $products->count() . ' products in database');
                $productContext = $this->formatProductContext($products,
                    $userMessage);
            } else {
                Log::info('No products found in database search');
                // Still provide context even if no products found
                $productContext = $this->formatProductContext($products,
                    $userMessage);
            }
        }

        $client = new Client([
            'timeout' => 90,
            'verify' => false
        ]);

        try {
            $systemMessage = 'You are a helpful assistant specialized in PC components, computer hardware, and electronics. ';
            $systemMessage .= "You are integrated with the store's database and have access to real product information. ";
            $systemMessage .= "You can recommend products, compare specifications, and help users make purchasing decisions. ";
            $systemMessage .= "Be specific, technical, and helpful when discussing products.\n\n";

            if ($productContext) {
                $systemMessage .= $productContext . "\n\n";
                $systemMessage .= "IMPORTANT INSTRUCTIONS:\n";
                $systemMessage .= "1. ALWAYS reference specific products by name when they match the user's query\n";
                $systemMessage .= "2. Mention prices when available\n";
                $systemMessage .= "3. Highlight key specifications from the descriptions\n";
                $systemMessage .= "4. If the user asks for recommendations, suggest the most relevant products from the list above\n";
                $systemMessage .= "5. If we don't have exactly what they're looking for, suggest similar alternatives\n";
                $systemMessage .= "6. Be enthusiastic and knowledgeable about our products!\n";
            }
        }
    }

```

```

    } else {
        $systemMessage .= "The user is asking a general question. Help them
with computer hardware knowledge, ";
        $systemMessage .= "recommendations, or suggest they ask about specific
products we might carry like CPUs, GPUs, RAM, storage, or other components.";
    }

    Log::info('System message length: ' . strlen($systemMessage));

    $response = $client->post('https://api.deepseek.com/v1/chat/completions',
[
    'headers' => [
        'Authorization' => 'Bearer ' . $apiKey,
        'Content-Type' => 'application/json',
    ],
    'json' => [
        'model' => 'deepseek-chat',
        'messages' => [
            [
                'role' => 'system',
                'content' => $systemMessage
            ],
            [
                'role' => 'user',
                'content' => $userMessage
            ]
        ],
        'temperature' => 0.7,
        'max_tokens' => 1500,
    ],
]);

$responseData = json_decode($response->getBody()->getContents(), true);

if (!isset($responseData['choices'][0]['message']['content'])) {
    Log::error('DeepSeek API unexpected response structure',
$responseData);
    return response()->json([
        'reply' => "Sorry, I couldn't process that. Please try again.",
    ], 500);
}

return response()->json([
    'reply' => $responseData['choices'][0]['message']['content'],
    'products_found' => $products->count(),
    'is_product_query' => $isProductQuery
]);

} catch (\Exception $e) {
    Log::error('ChatController error', [
        'message' => $e->getMessage(),
        'trace' => $e->getTraceAsString()
    ]);

    return response()->json([
        'reply' => 'Sorry, I am currently unable to process your request.
Please try again later.'
    ], 500);
}

}

// Helper method to get search suggestions for debugging
public function getSearchableCategories()
{
    $categories = [];
    foreach ($this->categoryMappings as $key => $data) {
        $categories[] = [
            'name' => $data['category_name'],
            'keywords' => $data['keywords'],
            'example_query' => "Do you have any {$data['category_name']}
products?"

```

```

    ];
}

return response()->json([
    'categories' => $categories,
    'brands' => array_keys($this->brandMappings)
]);
}
}

```

13. pielikums

PDF izdruka (piemēri)

Products (42 records)

Id	Name	Price	Category	Created At
1	Intel Core i5F	105.00	Component	2025-04-27T09:02:47.000000Z
2	Intel Core i7	200.00	Component	2025-04-27T09:02:47.000000Z

Id	Name	Price	Category	Created At
3	Intel Core i5	150.00	Component	2025-04-27T09:02:47.000000Z
4	Intel Core i9	400.00	Component	2025-04-27T09:02:47.000000Z
5	Intel Core i9	450.00	Component	2025-04-27T09:02:47.000000Z
6	Intel Core i3	50.00	Component	2025-04-27T09:02:47.000000Z
7	Adata SSD 1 TB	70.00	Component	2025-04-27T09:02:47.000000Z
8	WD SSD 1 TB	90.00	Component	2025-04-27T09:02:47.000000Z
9	Nvidia RTX 3060	300.00	Component	2025-04-27T09:02:47.000000Z
10	Nvidia RTX 3080	500.00	Component	2025-04-27T09:02:47.000000Z
11	Nvidia RTX 3090	700.00	Component	2025-04-27T09:02:47.000000Z
12	Kingston RAM 3200 MHz 8GB	50.00	Component	2025-04-27T09:02:47.000000Z
13	Kingston RAM 3200 MHz 16GB	70.00	Component	2025-04-27T09:02:47.000000Z
14	Kingston RAM 3200 MHz 32GB	80.00	Component	2025-04-27T09:02:47.000000Z
15	MSI Gaming Laptop	600.00	Laptop	2025-04-27T09:02:47.000000Z

32. att. PDF izdrukas skice ar produktiem

Orders (69 records)					
Id	User Id	Status	Total	Created At	User
1	1	pending	700.00	2025-04-27T09:03:44.000000Z	{"id":1,"name":"johndoe"}
2	1	pending	950.00	2025-04-27T09:04:40.000000Z	{"id":1,"name":"johndoe"}
3	2	pending	50.00	2025-05-15T10:43:02.000000Z	{"id":2,"name":"igor"}
4	3	pending	700.00	2025-05-25T11:17:17.000000Z	{"id":3,"name":"rvt"}
5	8	pending	1250.00	2025-06-05T07:16:02.000000Z	{"id":8,"name":"kms"}
6	8	pending	970.00	2025-06-05T07:29:26.000000Z	{"id":8,"name":"kms"}
7	8	pending	50.00	2025-06-05T07:33:49.000000Z	{"id":8,"name":"kms"}
8	9	pending	800.00	2025-06-24T10:35:45.000000Z	{"id":9,"name":"g"}
9	9	pending	800.00	2025-06-24T10:37:05.000000Z	{"id":9,"name":"g"}
10	9	pending	800.00	2025-06-24T10:40:09.000000Z	{"id":9,"name":"g"}
11	9	pending	800.00	2025-06-24T10:42:00.000000Z	{"id":9,"name":"g"}
12	9	pending	100.00	2025-06-24T10:46:09.000000Z	{"id":9,"name":"g"}
13	9	pending	150.00	2025-06-25T14:54:49.000000Z	{"id":9,"name":"g"}
14	9	pending	1050.00	2025-06-25T15:01:51.000000Z	{"id":9,"name":"g"}
15	10	pending	1360.00	2025-08-31T08:09:56.000000Z	{"id":10,"name":"mYn"}
16	11	pending	700.05	2025-10-05T13:36:56.000000Z	{"id":11,"name":"man"}
18	12	pending	2903.00	2026-01-17T12:45:30.000000Z	{"id":12,"name":"man2"}
19	12	pending	803.00	2026-01-17T14:37:17.000000Z	{"id":12,"name":"man2"}
20	12	pending	803.00	2026-01-17T15:04:22.000000Z	{"id":12,"name":"man2"}

33. att. PDF izdrukas skice ar pasūtījumiem

Families (5 records)					
Id	Family Name	Parent Id	Invitation Code	Created At	Parent
1	Mikes	14	15848FE7	2026-01-31T14:53:22.000000Z	{"id":14,"name":"family"}
2	Smith	1	10326109	2026-02-01T11:57:33.000000Z	{"id":1,"name":"johndoe"}
3	Johns	19	7BFF26BA	2026-02-07T14:57:32.000000Z	{"id":19,"name":"johns"}
4	family	21	E1C30B0A	2026-03-02T16:33:05.000000Z	{"id":21,"name":"family3"}
5	Family5	23	0FD2EA35	2026-03-15T15:22:07.000000Z	{"id":23,"name":"family5"}

Comments (7 records)					
Id	User Id	Body	Created At	User	
1	-	Nice shop	2025-06-27T12:40:43.000000Z	-	
2	-	Thank you	2025-06-27T12:46:35.000000Z	-	

34. att. PDF izdrukas skice ar ģimenēm un komentāriem

Auctions (1 records)						
Id	Name	Starting Bid	Start Time	End Time	Created At	
1	Mac 1991	2000.00	2025-05-16	2025-05-31	2025-05-25T11:16:23.000000Z	

Bids (3 records)						
Id	Item Id	User Id	Bid Amount	Created At	User	Auction
1	1	10	1.00	2025-08-31T10:44:01.000000Z	{"id":10,"name":"mYn"}	{"id":1,"name":"Mac 1991"}
2	1	10	1.00	2025-08-31T10:44:29.000000Z	{"id":10,"name":"mYn"}	{"id":1,"name":"Mac 1991"}
3	1	27	10002.00	2026-04-18T15:33:25.000000Z	{"id":27,"name":"kim"}	{"id":1,"name":"Mac 1991"}

Exported from Admin Panel | 2026-04-28 10:00:37

35. att. PDF izdrukas skice ar izsolēm un solījumiem